Computational Complexity of Information Sufficiency

Tristan Simas
McGill University
tristan.simas@mail.mcgill.ca

February 23, 2026

© 2026 Tristan Simas. This work is licensed under CC BY 4.0. License: https://creativecommons.org/licenses/by/4.0/
Zenodo DOI: 10.5281/zenodo.18140965

Abstract

We characterize the computational complexity of *information sufficiency* as a *meta-problem*: given an arbitrary decision problem $\mathcal{D}$ with action set A, factored state space $S = X_1 \times \cdots \times X_n$, and utility $U : A \times S \to \mathbb{Q}$, does a candidate coordinate set I carry enough information to act optimally? Formally, I is *sufficient* if $s_I = s'_I \implies \mathrm{Opt}(s) = \mathrm{Opt}(s')$. Because the problems below are parameterized by an arbitrary $\mathcal{D}$, and any problem requiring a choice among alternatives under structured information instantiates the (A, S, U) tuple, the complexity results apply to every problem with factored state space, not to a specific problem domain (e.g., configuration simplification where parameters are coordinates and behavior-preservation is sufficiency).

Information-theoretically, this is task-conditional compression: the target is preservation of the optimal-action map rather than full-state reconstruction. This places the framework at the interface of source coding and zero-error confusability structure [57, 14, 58, 31, 38], while separating short report length from certifiable exactness in the MDL/Kolmogorov sense [54, 24, 35].

The landscape in the formal model:

- **General case:** SUFFICIENCY-CHECK is coNP-complete; ANCHOR-SUFFICIENCY is Σ_2^{P}-complete.
- **Tractable cases:** Polynomial-time for bounded action sets under the explicit-state encoding; separable utilities ($U = f + g$) under any encoding; and tree-structured utility with explicit local factors.
- **Encoding-regime separation:** Polynomial-time under the explicit-state encoding (polynomial in $|S|$). Under ETH, there exist succinctly encoded worst-case instances witnessed by a strengthened Lean-mechanized reduction gadget from the paper's SAT/TAUTOLOGY hardness spine (see Appendix A) with $k^* = n$ for which SUFFICIENCY-CHECK requires $2^{\Omega(n)}$ time.
- **Intermediate query-access lower bounds:** In the finite-state query core, SUFFICIENCY-CHECK has worst-case Opt-oracle query complexity $\Omega(|S|)$ via indistinguishable yes/no pairs for the $I = \emptyset$ subproblem (Boolean instantiation: $\Omega(2^n)$); mechanized Boolean interface refinements show the same obstruction for value-entry and state-batch access, including finite-support randomized robustness and oracle-lattice transfer-/strictness statements.

The tractable cases are stated with explicit encoding assumptions (Section 2.4). Together, these results answer the question "when is decision-relevant information identifiable efficiently?"

within the stated regimes. Complexity classification is a property of the encoding regime measured against a fixed decision question (Remark 2.21): the same sufficiency predicate is polynomial-time under explicit-state access and worst-case exponential under succinct encoding. At the structural level, the apparent $\exists\forall$ form of MINIMUM-SUFFICIENT-SET collapses to a coNPcharacterization via the criterion sufficient$(I) \iff$ Relevant $\subseteq I$. Within this regime-typed framework, under a declared objective contract, over-modeling is admissible only under integrity-preserving attempted-competence-failure conditions (roughly: after an attempted exact procedure, exact competence remains unavailable in the active regime); outside that cell, persistent over-modeling is irrational for the same objective.*[C:P3.59;R:AR]* The report layer is now formally non-binary: abstention can carry a tentative guess and self-reflected confidence, while certified confidence is evidence-gated by typed admissibility; positive certified confidence implies admissibility, and exact reports without exact competence force zero certified confidence.*[C:P3.23,3.29,3.30;C3.25;R:AR]* For progress semantics, the framework distinguishes confidence percentages from runtime witnesses: scalar step-count is always well-defined/falsifiable, while completion percentage is defined only under an explicit declared bound.*[C:P3.26,3.27,3.28;R:AR]* For temporal semantics, interface time is formalized as a discrete $\mathbb{N}$ index; one decision event is exactly one time-unit transition, and run length equals elapsed interface time.*[C:P2.27,2.28,2.29;R:AR]*

The contribution has two levels: (i) a complete complexity landscape for the core decision-relevant problems in the static sufficiency class defined by the model contract (coNP/Σ_2^P completeness and tractable regimes under explicit encoding assumptions), and (ii) a formal regime-typing framework that separates structural complexity from representational hardness and yields theorem-indexed engineering corollaries.

The reduction constructions and key equivalence theorems are machine-checked in Lean 4 (12218 lines, 564 theorem/lemma statements); complexity classifications follow by composition with standard results (see Appendix A).

Keywords: computational complexity, decision theory, polynomial hierarchy, tractability dichotomy, Lean 4

1 Introduction

Consider any decision problem $\mathcal{D}$ with actions A and factored state space $S = X_1 \times \cdots \times X_n$. A coordinate set $I \subseteq \{1, \ldots, n\}$ is *sufficient* if knowing only coordinates in I determines the optimal action:

$$s_I = s'_I \implies \mathrm{Opt}(s) = \mathrm{Opt}(s')$$

This is *coordinate sufficiency*, the formalization of *information sufficiency* (whether available information determines the optimal action) over a factored state space.

In information-theoretic terms, this is a task-conditional coding problem: the object preserved by projection is the decision boundary $s \mapsto \mathrm{Opt}(s)$, not the full source state. The same object appears in classical source-coding and fidelity viewpoints [57, 59, 14, 64], and in zero-error/confusability structure where exact recoverability depends on graph constraints rather than raw description length [58, 31, 38, 15]. The same separation also appears in model selection: short descriptions and certifiable adequacy are distinct [54, 24, 35, 8].

Compact claim-stamp notation. We use compact provenance stamps of the form $[C : \cdots ; R : \cdots]$: claim references are grouped by type ($D/P/C/T/L/R$ for Definition/Proposition/Corollary/Theorem/Lemma/Remark), and regime tags are abbreviated (for example AR=active declared regime, DC=declared contract, CR=declared class/regime, Qf=query-finite, Qb=query-Boolean). Theorem-strength narrative statements are stamped and, where available, paired with compact Lean handle IDs; unstamped narrative remains explanatory or citation-only context.

SUFFICIENCY-CHECK, MINIMUM-SUFFICIENT-SET, and ANCHOR-SUFFICIENCY are *meta-problems*: each is parameterized by an arbitrary decision problem $\mathcal{D}$ and asks a structural question about $\mathcal{D}$'s coordinate space. The complexity results therefore apply to every problem with factored state space, not to a specific problem domain.*[C:T4.6,4.7,4.9;R:S] (L: CC61,CC36,CC5.)*

Remark 1.1 (Universality of the Decision-Problem Tuple). The formal object of this paper is the decision-problem tuple $\mathcal{D} = (A, S, U)$: actions, factored states, utility. Within this model family, any problem requiring a choice among alternatives given structured information instantiates the tuple: the alternatives are A, the structured information is $S = X_1 \times \cdots \times X_n$, and the objective is U. Thus the qualifier "decision problem" does not narrow scope *inside the declared coordinate model*. The meta-problem results below are universal over problems representable with this coordinate structure; the (A, S, U) formalism supplies the language for stating the sufficiency predicate.

This paper characterizes the complexity landscape of these meta-problems within the formal model:

Section 2.4 fixes the computational model and input encodings used for all complexity claims. Section 2.5 gives the model contract and regime tags used to type every strong claim. Completeness claims are class-internal: complete for the static sufficiency class fixed by C1–C4 and the declared access regimes. Adjacent objective classes that violate the bridge conditions are typed separately (Section 2.5).

Problem	**Complexity**	**When Tractable**
SUFFICIENCY-CHECK	coNP-complete	Bounded actions (explicit-state), separable utility, tree-structured utility
MINIMUM-SUFFICIENT-SET	coNP-complete	Same conditions
ANCHOR-SUFFICIENCY	Σ_2^{P}-complete	Open

The tractable cases are stated with explicit encoding assumptions (Section 2.4). Outside those regimes, the succinct model yields hardness.

1.1 Hardness–Recovery Reading Map

Theorems in this paper are intended to be read as hardness/recovery pairs for the same fixed decision question:

1. **Sufficiency checking:** [S] coNP-complete in the general succinct regime (Theorem 4.6), paired with polynomial-time recovery under explicit-state and structured-access regimes (Theorem 6.1).

2. **Minimal sufficient sets:** [S] coNP-complete in general (Theorem 4.7), paired with the collapse criterion and relevance-computation recovery route (Theorems 4.14, 6.1).

3. **Anchor sufficiency:** Σ_2^{P}-complete in the general anchored formulation (Theorem 4.9); no general polynomial-time recovery condition is established in this model.

(L: CC60, CC36, CC34, CC4, CC71, S2P2.)

An operational criterion follows later from the same chain: once decision-site count exceeds the amortization threshold n^*, avoiding structural identification is more expensive than paying the one-time centralization cost (Theorem 9.6).*[C:T9.6;R:LC] (L: HD3, HD21.)*

1.2 Landscape Summary

When is sufficiency checking tractable? We identify three sufficient conditions:

1. **Bounded actions** ($|A| \leq k$) under explicit-state encoding: with constantly many actions, we enumerate action pairs over the explicit utility table.

2. **Separable utility** ($U(a, s) = f(a) + g(s)$): The optimal action depends only on f, making all coordinates irrelevant to the decision.

3. **Tree-structured utility**: With explicit local factors over a tree, dynamic programming yields polynomial algorithms in the input length.

Each condition is stated with its encoding assumption. Outside these regimes, the general problem remains coNP-hard (Theorem 4.6). *[C:T4.6,6.1;R:E,S] (L: CC61,CC69,CC70,CC71.)*

When is it intractable? The general problem is coNP-complete (Theorem 4.6), with a separation between explicit-state tractability and succinct worst-case hardness:

- In the explicit-state model: SUFFICIENCY-CHECK is solvable in polynomial time in $|S|$ by explicitly computing $\mathrm{Opt}(s)$ for all $s \in S$ and checking all pairs (s, s') with equal I-projection. In particular, instances with $k^* = O(\log |S|)$ are tractable in this model.

- In the intermediate query-access model: the finite-state core yields an Opt-oracle lower bound $\Omega(|S|)$ (Boolean instantiation: $\Omega(2^n)$), and the mechanized Boolean interface refinements preserve the obstruction for value-entry and state-batch interfaces, with explicit subproblem-to-full transfer, weighted randomized robustness, and oracle-lattice transfer/strictness theorems (Propositions 5.3–5.15).

- In the succinct model: under ETH there exist worst-case instances produced by our strengthened gadget in which the minimal sufficient set has size $\Omega(n)$ (indeed n) and SUFFICIENCY-CHECK requires $2^{\Omega(n)}$ time.

The explicit ETH lower bound is still a succinct worst-case statement; Proposition 5.3 provides the finite-state query core and Propositions 5.6–5.15 provide theorem-level Boolean-interface refinements. *[C:P5.3,5.6,5.7,5.10,5.14,5.15;R:Qf,Qb] (L: CC50,L9,L11,L8,L21,L20.)*

1.3 Main Theorems

1. **Theorem 4.6:** SUFFICIENCY-CHECK is coNP-complete via reduction from TAUTOLOGY.

2. **Theorem 4.7:** MINIMUM-SUFFICIENT-SET is coNP-complete (the Σ_2^{P} structure collapses).

3. **Theorem 4.9:** ANCHOR-SUFFICIENCY is Σ_2^{P}-complete via reduction from $\exists\forall$-SAT.

4. **Theorem 5.1:** Encoding-regime separation: explicit-state polynomial-time (polynomial in $|S|$), and under ETH a succinct worst-case lower bound witnessed by a hard family with $k^* = n$.

5. **Theorem 6.1:** Polynomial algorithms for bounded actions, separable utility, tree structure.

6. **Theorem 2.35:** physical-system bridge composition (law-objective semantics, behavior-preserving configuration reduction, and required-work lower bound) is proved as one bundled mechanized statement under declared encoding assumptions.

7. **Propositions 2.27–2.30:** the interface-time model is discrete, each decision event advances time by one unit, run-length equals elapsed interface time, and the unit-time law is substrate-invariant under the declared interface-consistency condition.

[C:T4.6,4.7,4.9,5.1,6.1,2.35;P2.27,2.28,2.29,2.30;R:E,S,S+ETH,AR] (L: CC61,CC36,CC5,CC16,CC69,CC70,CC71,CT4,DQ548,DQ537,DQ539,DQ541.)

1.4 Claim-Typing Theorems

1. **Proposition 3.16:** report admissibility is typed by certificate class (ABSTAIN, EXACT, EPSILON(ε)).

2. **Proposition 3.18:** first-class evidence-object existence is equivalent to typed report admissibility.

3. **Propositions 2.5 and 2.6:** the $I = \emptyset$ query is exactly universal constancy of the decision boundary, and non-sufficiency is exactly existence of a counterexample pair.

4. **Proposition 3.21:** exact admissibility is equivalent to exact evidence existence.

5. **Proposition 3.23 and Corollary 3.25:** certified confidence is evidence-gated; for exact reports in no-competence regimes, certified confidence must be zero.

6. **Proposition 3.24:** under signal consistency, no evidence implies zero certified confidence.

7. **Proposition 3.29 and Proposition 3.30:** abstain may carry tentative guesses and self-reflected confidence, but self-confidence alone does not certify exactness.

8. **Propositions 3.26, 3.27, 3.28:** scalar step-count witnesses are always exact/falsifiable, while completion percentages are meaningful only under explicit declared bounds.

9. **Theorem 3.55:** outside declared carve-outs, admissible exact-claim policy requires explicit assumptions.

10. **Proposition 3.60:** under non-collapse, integrity and exact competence cannot both hold universally on hardness-blocked declared regimes.

11. **Proposition 3.61 and Corollary 3.62:** schema-level consequence for declared class/regime objects and exact-report inadmissibility.

[C:T3.55;P3.16,3.18,2.5,2.6,3.21,3.23,3.24,3.29,3.30,3.26,3.27,3.28,3.60,3.61;C3.25,3.62;R:DC,AR] (L: CC75,IC2,DP6,DP7,IC35,IC38,IC40,IC46,IC47,IC42,IC44,IC45,CC30,CC15,CC38.)

1.5 Regime Separation Theorems

1. **Theorem 5.1:** same-question separation between explicit-state and succinct regimes (with [S+ETH] hard-family lower bound).

2. **Proposition 5.3:** finite-state Opt-oracle lower bound in [Q_fin].

3. **Propositions 5.6–5.15:** Boolean-interface refinements and transfer/strictness in [Q_bool].

[C:T5.1;P5.3,5.6,5.10,5.7,5.14,5.15;R:E,S+ETH,Qf,Qb] (L: CC16,CC50,L9,L8,L11,L21,L20,L23.)

1.6 Machine-Checked Proofs

The reduction constructions and key equivalence theorems are machine-checked in Lean 4 [16, 17, 67, 21, 32] (12218 lines, 564 theorem/lemma statements). The formalization verifies that the TAUTOLOGY reduction correctly maps tautologies to sufficient coordinate sets. Complexity class membership (coNP-completeness, Σ_2^P-completeness) follows by composition with standard complexity-theoretic results.

What is new. We contribute (i) formal definitions of decision-theoretic sufficiency in Lean; (ii) machine-checked proofs of reduction correctness for the TAUTOLOGY and ∃∀-SAT reductions; (iii) a complete complexity landscape for coordinate sufficiency with explicit encoding assumptions; and (iv) a formal separation between structural complexity and representational hardness used to derive theorem-indexed engineering corollaries, including theorem-level typed coverage/completeness closures for the declared static class (Theorems 2.36, 3.4). Prior work establishes hardness informally; we formalize the constructions and their regime-typed consequences. *[C:T2.36,3.4,4.6,4.9;R:CR,S] (L: CC14,CC76,CC61,CC5.)*

1.7 Paper Structure

The primary contribution is theoretical: a formalized reduction framework and a complete characterization of the core decision-relevant problems in the formal model (coNP/Σ_2^P completeness and tractable cases stated under explicit encoding assumptions). A second contribution is formal claim typing: Section 3 introduces the structural/representational and integrity/competence splits that type-check the applied corollaries.

Section 2: decision-problem foundations and encoding model. Section 3: structural vs representational hardness; integrity vs competence. Section 4: hardness proofs. Section 5: regime separation. Section 6: tractable cases. Section 7: regime-conditional corollaries. Section 8: dominance theorems for hardness placement. Section 9: conservation law for decision-relevant coordinates. Section 10: related work. Appendix A: Lean listings.

Sections 7–9 are not applied commentary. Every claim in those sections is a formal corollary, regime-typed by the framework of Section 3 and indexed to a machine-checked theorem handle. They complete the complexity landscape by characterizing when exact minimization is rational and when it is not: the rationality matrix (Proposition 3.59), the dominance and conservation theorems (Theorems 8.16, 9.3), and the amortization threshold (Theorem 9.6) are all mechanized in Lean. *[C:P3.59;T8.16,9.3,9.6;R:AR,LC] (L: IC27,HD1,HD4,HD3.)*

2 Formal Foundations

We formalize decision problems with coordinate structure, sufficiency of coordinate sets, and the decision quotient, drawing on classical decision theory [55, 52].

2.1 Decision Problems with Coordinate Structure

Definition 2.1 (Decision Problem)**.** A *decision problem with coordinate structure* is a tuple $\mathcal{D} = (A, X_1, \ldots, X_n, U)$ where:

- A is a finite set of *actions* (alternatives)
- $X_1, \ldots, X_n$ are finite *coordinate spaces*

- $S = X_1 \times \cdots \times X_n$ is the *state space*
- $U : A \times S \to \mathbb{Q}$ is the *utility function*

Definition 2.2 (Projection)**.** For state $s = (s_1, \ldots, s_n) \in S$ and coordinate set $I \subseteq \{1, \ldots, n\}$:

$$s_I := (s_i)_{i \in I}$$

is the *projection* of s onto coordinates in I.

Definition 2.3 (Optimizer Map)**.** For state $s \in S$, the *optimal action set* is:

$$\mathrm{Opt}(s) := \arg\max_{a \in A} U(a, s) = \{a \in A : U(a, s) = \max_{a' \in A} U(a', s)\}$$

2.2 Sufficiency and Relevance

Definition 2.4 (Sufficient Coordinate Set)**.** A coordinate set $I \subseteq \{1, \ldots, n\}$ is *sufficient* for decision problem $\mathcal{D}$ if:

$$\forall s, s' \in S : \quad s_I = s'_I \implies \mathrm{Opt}(s) = \mathrm{Opt}(s')$$

Equivalently, the optimal action depends only on coordinates in I. In this paper, this set-valued invariance of the full optimal-action correspondence is the primary decision-relevance target.

Proposition 2.5 (Empty-Set Sufficiency Equals Constant Decision Boundary)**.** *For any decision problem,*

$$\textit{sufficient}(\emptyset) \iff \forall s, s' \in S,\ \textit{Opt}(s) = \textit{Opt}(s').$$

Hence, the $I = \emptyset$ query is exactly a universal constancy check of the decision boundary. *(L: DP6.)*

Proposition 2.6 (Insufficiency Equals Counterexample Witness)**.** *For any coordinate set I,*

$$\neg\textit{sufficient}(I) \iff \exists s, s' \in S :\ s_I = s'_I \ \wedge\ \textit{Opt}(s) \neq \textit{Opt}(s').$$

In particular,

$$\neg\textit{sufficient}(\emptyset) \iff \exists s, s' \in S :\ \textit{Opt}(s) \neq \textit{Opt}(s').$$

(L: DP7, DP8.)

[C:D2.4;P2.5,2.6;R:DM]

Definition 2.7 (Deterministic Selector)**.** A *deterministic selector* is a map $\sigma : 2^A \to A$ that returns one action from an optimal-action set.

Definition 2.8 (Selector-Level Sufficiency)**.** For fixed selector σ, a coordinate set I is *selector-sufficient* if

$$\forall s, s' \in S : \quad s_I = s'_I \implies \sigma(\mathrm{Opt}(s)) = \sigma(\mathrm{Opt}(s')).$$

Proposition 2.9 (Set-Level Target Dominates Selector Targets)**.** *If I is sufficient in the set-valued sense (Definition 2.4), then I is selector-sufficient for every deterministic selector σ (Definition 2.8).* *(L: DP5)*

Proof. If I is sufficient, then $s_I = s'_I$ implies $\mathrm{Opt}(s) = \mathrm{Opt}(s')$. Applying any deterministic selector σ to equal optimal-action sets yields equal selected actions. ■

Definition 2.10 (ε-Optimal Set and ε-Sufficiency)**.** For $\varepsilon \geq 0$, define

$$\mathrm{Opt}_\varepsilon(s) := \{a \in A : \forall a' \in A,\ U(a', s) \leq U(a, s) + \varepsilon\}.$$

Coordinate set I is *ε-sufficient* if

$$\forall s, s' \in S :\ s_I = s'_I \implies \mathrm{Opt}_\varepsilon(s) = \mathrm{Opt}_\varepsilon(s').$$

Proposition 2.11 (Zero-ε Reduction)**.** *Set-valued sufficiency is exactly the $\varepsilon = 0$ case:*

$$I \text{ sufficient} \iff I \text{ is } 0\text{-sufficient}.$$

(L: DP1, DP4)

Proposition 2.12 (Selector-Level Separation Witness)**.** *The converse of Proposition 2.9 fails in general: there exists a decision problem, selector, and coordinate set I such that I is selector-sufficient but not sufficient in the full set-valued sense. (L: CC54.)*

Definition 2.13 (Minimal Sufficient Set)**.** A sufficient set I is *minimal* if no proper subset $I' \subsetneq I$ is sufficient.

Definition 2.14 (Relevant Coordinate)**.** Coordinate i is *relevant* if there exist states that differ only at coordinate i and induce different optimal action sets:

$$i \text{ is relevant} \iff \exists s, s' \in S :\ \left(\forall j \neq i,\ s_j = s'_j\right) \ \wedge\ \mathrm{Opt}(s) \neq \mathrm{Opt}(s').$$

Proposition 2.15 (Minimal-Set/Relevance Equivalence)**.** *For any minimal sufficient set I and any coordinate i:*

$$i \in I \iff i \text{ is relevant}.$$

Hence every minimal sufficient set is exactly the relevant-coordinate set. (L: DP2, DP3.)

Proof. The "only if" direction follows by minimality: if $i \in I$ were irrelevant, removing i would preserve sufficiency, contradicting minimality. The "if" direction follows from sufficiency: every sufficient set must contain each relevant coordinate. ■

Definition 2.16 (Exact Relevance Identifiability)**.** For a decision problem $\mathcal{D}$ and candidate coordinate set I, we say I is *exactly relevance-identifying* if

$$\forall i \in \{1, \ldots, n\} : \quad i \in I \iff i \text{ is relevant for } \mathcal{D}.$$

Equivalently, I is exactly relevance-identifying iff I equals the full relevant-coordinate set.

Example 2.17 (Weather Decision)**.** Consider deciding whether to carry an umbrella:

- Actions: $A = \{\text{carry}, \text{don't carry}\}$
- Coordinates: $X_1 = \{\text{rain}, \text{no rain}\}$, $X_2 = \{\text{hot}, \text{cold}\}$, $X_3 = \{\text{Monday}, \ldots, \text{Sunday}\}$
- Utility: $U(\text{carry}, s) = -1 + 3 \cdot \mathbf{1}[s_1 = \text{rain}]$, $U(\text{don't carry}, s) = -2 \cdot \mathbf{1}[s_1 = \text{rain}]$

The minimal sufficient set is $I = \{1\}$ (only rain forecast matters). Coordinates 2 and 3 (temperature, day of week) are irrelevant.

2.3 The Decision Quotient

Definition 2.18 (Decision Equivalence)**.** For coordinate set I, states s, s' are *I-equivalent* (written $s \sim_I s'$) if $s_I = s'_I$.

Definition 2.19 (Decision Quotient)**.** The *decision quotient* for state s under coordinate set I is:

$$\mathrm{DQ}_I(s) = \frac{|\{a \in A : a \in \mathrm{Opt}(s') \text{ for some } s' \sim_I s\}|}{|A|}$$

This measures the fraction of actions that are optimal for at least one state consistent with I.

Proposition 2.20 (Sufficiency Characterization)**.** *Coordinate set I is sufficient if and only if $DQ_I(s) = |Opt(s)|/|A|$ for all $s \in S$.* *(L: CC62, CC63)*

Proof. If I is sufficient, then $s \sim_I s' \implies \mathrm{Opt}(s) = \mathrm{Opt}(s')$, so the set of actions optimal for some $s' \sim_I s$ is exactly $\mathrm{Opt}(s)$.

Conversely, if the condition holds, then for any $s \sim_I s'$, the optimal actions form the same set (since $\mathrm{DQ}_I(s) = \mathrm{DQ}_I(s')$ and both equal the relative size of the common optimal set). ■

2.4 Computational Model and Input Encoding

We fix the computational model used by the complexity claims.

Succinct encoding (primary for hardness). This succinct circuit encoding is the standard representation for decision problems in complexity theory; hardness is stated with respect to the input length of the circuit description [5, 47]. An instance is encoded as:

- a finite action set A given explicitly,
- coordinate domains $X_1, \ldots, X_n$ given by their sizes in binary,
- a Boolean or arithmetic circuit C_U that on input (a, s) outputs $U(a, s)$.

The input length is $L = |A| + \sum_i \log |X_i| + |C_U|$. Polynomial time and all complexity classes (coNP, Σ_2^P, ETH) are measured in L. All hardness results in Section 4 use this encoding.

Explicit-state encoding (used for enumeration algorithms and experiments). The utility is given as a full table over $A \times S$. The input length is $L_{\exp} = \Theta(|A||S|)$ (up to the bitlength of utilities). Polynomial time is measured in $L_{\exp}$. Results stated in terms of $|S|$ use this encoding.

Query-access regime (intermediate black-box access). The solver is given oracle access to decision information at queried states (e.g., $\mathrm{Opt}(s)$, or $U(a, s)$ with $\mathrm{Opt}(s)$ reconstructed from finitely many value queries). Complexity is measured by oracle-query count (optionally paired with per-query evaluation cost). This separates representational access from full-table availability and from succinct-circuit input length.

Unless explicitly stated otherwise, "polynomial time" refers to the succinct encoding.

Remark 2.21 (Decision Questions vs Decision Problems). We distinguish between a *decision question* and a *decision problem*. A decision question is the encoding-independent predicate: "is coordinate set I sufficient for $\mathcal{D}$?" This is a mathematical relation, fixed by the model contract (C1–C4). A decision problem is a decision question together with a specific encoding of its inputs. The same

decision question yields different decision problems under different encodings, and these problems may belong to different complexity classes. Formally, each encoding regime e induces a language $L_{\mathcal{D},e}$; complexity classification is a property of $L_{\mathcal{D},e}$, not of $\mathcal{D}$ alone.

Definition 2.22 (Regime Simulation)**.** Fix a decision question Q and two regimes R_1, R_2 for that question. We say R_1 *simulates* R_2 if there exists a polynomial-time transformer that maps any R_2 instance to an R_1 instance while preserving the answer to Q; in oracle settings, this includes a polynomial-time transcript transducer that preserves yes/no outcomes for induced solvers. *(L: cc1, cc52.)*

This paper uses that simulation spine in two explicit forms: restriction-map simulation (subproblem-to-full transfer; Proposition 5.7) and oracle-transducer simulation (batch-to-entry transfer; Proposition 5.14).

2.5 Model Contract and Regime Tags

All theorem statements in this paper are typed by the following model contract:

- **C1 (finite actions):** A is finite.
- **C2 (finite coordinate domains):** each X_i is finite, so $S = \prod_i X_i$ is finite.
- **C3 (evaluable utility):** $U(a, s)$ is computable from the declared input encoding.
- **C4 (fixed decision semantics):** optimality is defined by $\mathrm{Opt}(s) = \arg\max_a U(a, s)$.

We use short regime tags for applied corollaries:

- **[E]** explicit-state encoding,
- **[Q]** query-access (oracle) regime,
- **[Q_fin]** finite-state query-access core (state-oracle),
- **[S]** succinct encoding,
- **[S+ETH]** succinct encoding with ETH,
- **[Q_bool]** mechanized Boolean query submodel,
- **[S_bool]** mechanized Boolean-coordinate submodel.

This tagging is a claim-typing convention: each strong statement is attached to the regime where it is proven.

Remark 2.23 (Uniform Assumption Discipline). All theorem-level claims in this paper are conditional on their stated premises and regime tags; no assumption family is given special status. For each claim, the active assumptions are exactly those appearing in its statement, its regime tag, and its cited mechanized handles.

Remark 2.24 (Modal Convention: Contract-Conditional Necessity). In this paper, normative modal statements denote contract-conditional necessity: once an agent declares an objective/regime/assumption contract, admissible policy is fixed by typed admissibility and integrity under that contract. No unconditional external moral prescription is asserted.

2.6 Discrete-Time Event Semantics

Definition 2.25 (Timed Interface State)**.** For any interface state space S, a timed interface state is a pair

$$x = (s, t) \in S \times \mathbb{N}.$$

The time coordinate is the natural-number component t.

Definition 2.26 (Decision Tick and Decision Event)**.** Fix an interface decision process $(\text{decide}, \text{transition})$ with

$$\text{decide} : S \to A, \qquad \text{transition} : S \times A \to S.$$

Define one tick by

$$\text{tick}(s, t) = \big(\text{transition}(s, \text{decide}(s)),\ t + 1\big).$$

Define $\text{DecisionEvent}(x, y)$ to mean that y is the one-tick successor of x under this rule.

Proposition 2.27 (Time Coordinate Is Discrete)**.** *In Definition 2.25, time is discrete:*

$$\forall x \in S \times \mathbb{N},\ \exists k \in \mathbb{N} \text{ such that } x_t = k.$$

Pointwise time equality is decidable for every $k \in \mathbb{N}$:

$$x_t = k \ \vee\ x_t \neq k.$$

(L: DQ548,DQ547.)

Proposition 2.28 (Decision Event Is One Unit of Time)**.** *Under Definition 2.26, for all timed states x, y:*

$$\text{DecisionEvent}(x, y) \iff y = \text{tick}(x),$$

and therefore

$$\text{DecisionEvent}(x, y) \implies y_t = x_t + 1.$$

(L: DQ535,DQ537,DQ545,DQ546.)

Proposition 2.29 (Run Accounting Identity)**.** *Let $\text{run}(n, x)$ be n repeated ticks from x, and let $\text{trace}(n, x)$ be the emitted decision sequence. Then:*

$$\text{run}(n, x)_t = x_t + n, \qquad \text{run}(n, x)_t - x_t = n, \qquad |\text{trace}(n, x)| = n.$$

Hence decision count equals elapsed time:

$$|\text{trace}(n, x)| = \text{run}(n, x)_t - x_t.$$

(L: DQ540,DQ539,DQ533,DQ534.)

Proposition 2.30 (Substrate Realization Preserves Unit-Time Events)**.** *For any substrate model that is consistent with the declared interface transition rule, each one-step substrate evolution realizes one interface decision event and advances interface time by one unit. The time update law is invariant under substrate kind. (L: DQ542,DQ541,DQ549.)*

2.7 Physical-System Encoding and Theorem Transport

Definition 2.31 (Physical-to-Core Encoding)**.** Fix a physical instance class $\mathcal{P}$ and a core decision class $\mathcal{D}$. A *physical-to-core encoding* is a map

$$E : \mathcal{P} \to \mathcal{D}.$$

All physical claims below are typed relative to a declared encoding map E and declared assumption transfer map from physical assumptions to core assumptions.

Proposition 2.32 (Conditional Physical Claim Transport)**.** *Let $E : \mathcal{P} \to \mathcal{D}$ be an encoding. If*

$$\forall d \in \mathcal{D},\ \mathrm{CoreAssm}(d) \Rightarrow \mathrm{CoreClaim}(d)$$

and

$$\forall p \in \mathcal{P},\ \mathrm{PhysAssm}(p) \Rightarrow \mathrm{CoreAssm}(E(p)),$$

then

$$\forall p \in \mathcal{P},\ \mathrm{PhysAssm}(p) \Rightarrow \mathrm{CoreClaim}(E(p)).$$

(L: CT6, CT5.)

Corollary 2.33 (Physical Counterexample Implies Core Failure on Encoded Slice)**.** *Under the same encoding and assumption-transfer map:*

1. *any encoded physical counterexample induces a core counterexample on the encoded slice;*
2. *if a physical counterexample exists, then the purported core rule is invalid;*
3. *if the lifted claim holds on all physically admissible encoded instances, no such physical counterexample exists.*

(L: CT8, CT7, CT3.)

Proposition 2.34 (Law-Instance Objective Bridge)**.** *For the mechanized law-gap physical instance class, the encoded optimizer claim holds at theorem level and admits no counterexample:*

$$\forall x \in \mathcal{P}_{\mathrm{law}},\ \mathrm{lawGapPhysicalClaim}(x), \qquad \neg\exists x \in \mathcal{P}_{\mathrm{law}},\ \neg\,\mathrm{lawGapPhysicalClaim}(x).$$

(L: CT1, CT2.)

Theorem 2.35 (Physical Bridge Bundle)**.** *In one bundled mechanized statement, the paper's physical bridge layer composes:*

1. *law-induced objective semantics (Opt = feasible actions for law-gap instances),*
2. *behavior-preserving configuration reduction equivalence,*
3. *irreducible required-work lower bound under site-count scaling.*

(L: CT4.)

[C:D2.31;P2.32,2.34;C2.33;T2.35;R:AR]

Theorem 2.36 (Declared Regime Coverage for the Static Class)**.** *Let*

$$\mathcal{R}_{\mathrm{static}} = \{[E],\ [S{+}ETH],\ [Q_fin{:}Opt],\ [Q_bool{:}value\text{-}entry],\ [Q_bool{:}state\text{-}batch]\}.$$

For each declared regime in $\mathcal{R}_{\mathrm{static}}$, the paper has a theorem-level mechanized core claim. Equivalently, regime typing is complete over the declared static family. *(L: CC14, CC51.)*

Scope Lattice (typed classes and transfer boundary).

Layer	Transfer Status	Lean handles (representative)
Static sufficiency class (C1–C4; declared regimes) CC51	Internal landscape complete	CC76
Bridge-admissible adjacent class (one-step deterministic) CC10	Transfer licensed	CC42
Non-admissible represented adjacent classes (horizon, stochastic, transition-coupled) CC8	Transfer blocked by witness	CC9

2.8 Adjacent Objective Regimes and Bridge

Definition 2.37 (Adjacent Sequential Objective Regime)**.** An adjacent sequential objective instance consists of:

- finite action set A,
- finite coordinate state space $S = X_1 \times \cdots \times X_n$,
- horizon $T \in \mathbb{N}_{\geq 1}$ and history-dependent policy class,
- reward process r_t and objective functional $J(\pi)$ (e.g., cumulative reward or regret).

Proposition 2.38 (One-Step Deterministic Bridge)**.** *Consider an instance of Definition 2.37 satisfying:*

1. $T = 1$,
2. *deterministic rewards* $r_1(a, s) = U(a, s)$ *for some evaluable* U,
3. *objective* $J(\pi) = U(\pi(s), s)$ *(single-step maximization),*
4. *no post-decision state update relevant to the objective.*

Then the induced optimization problem is exactly the static decision problem of Definition 2.1, and coordinate sufficiency in the sequential formulation is equivalent to Definition 2.4. (L: cc42)

Proof. Under (1)–(3), optimizing J at state s is identical to choosing an action in $\arg\max_{a \in A} U(a, s) = \mathrm{Opt}(s)$. Condition (4) removes any dependence on future transition effects. Therefore the optimal-policy relation in the adjacent formulation coincides pointwise with Opt from Definition 2.3, and the invariance condition "same projection implies same optimal choice set" is exactly Definition 2.4. ■

Proposition 2.39 (Bridge Transfer Rule)**.** *Under conditions (1)–(4), any sufficiency statement formulated over Definition 2.4 is equivalent between the adjacent sequential formulation and the static model.* (L: cc42)

Proof. By Proposition 2.38, the adjacent sequential objective induces exactly the same sufficiency predicate as Definition 2.4 when (1)–(4) hold. Equivalence of any sufficiency statement then follows by substitution. ■

If any bridge condition fails, direct transfer from this paper's static complexity theorems is not licensed by this rule.

Remark 2.40 (Extension Boundary). Beyond Proposition 2.38 (multi-step horizon, stochastic transitions/rewards, or regret objectives), the governing complexity objects change. Those regimes are natural extensions, but they are distinct formal classes from the static sufficiency class analyzed in this paper.

Proposition 2.41 (Horizon-> 1 Bridge-Failure Witness)**.** *Dropping the one-step condition can break sufficiency transfer: there exists a two-step objective where sufficiency in the immediate-utility static projection does not match sufficiency in the two-step objective.* *(L: CC26, CC25.)*

Proposition 2.42 (Stochastic-Criterion Bridge-Failure Witness)**.** *If optimization is performed against a stochastic criterion not equal to deterministic utility maximization, bridge transfer to Definition 2.1 can fail even when an expected-utility projection is available.* *(L: CC57.)*

Proposition 2.43 (Transition-Coupled Bridge-Failure Witness)**.** *If post-decision transition effects are objective-relevant, bridge transfer to the static one-step class can fail.* *(L: CC73.)*

Theorem 2.44 (Exact Bridge Boundary in the Represented Adjacent Family)**.** *Within the represented adjacent family*

$$\{\textit{one-step deterministic},\ \textit{horizon-extended},\ \textit{stochastic-criterion},\ \textit{transition-coupled}\},$$

direct transfer of static sufficiency claims is licensed if and only if the class is one-step deterministic. Each represented non-one-step class has an explicit mechanized bridge-failure witness. *(L: CC10, CC9, CC8.)*

Definition 2.45 (Agent Snapshot vs Agent Process Typing)**.** Within the represented adjacent family, we type:

- *agent snapshot*: fixed-parameter inference view (mapped to one-step deterministic class),
- *agent process*: online/dynamical update views (horizon-extended, stochastic-criterion, or transition-coupled classes).

Proposition 2.46 (Snapshot/Process Transfer Typing)**.** *In the represented adjacent family, direct transfer of static sufficiency claims is licensed if and only if the agent is typed as an* agent snapshot*. Every process-typed represented class has an explicit mechanized bridge-failure witness.* *(L: CC3, CC48, CC56.)*

Operationally: a trained model evaluated with fixed parameters during inference is an agent snapshot for this typing; once parameters or transition-relevant internals are updated online, the object is process-typed and transfer from static sufficiency theorems is blocked unless the one-step bridge conditions are re-established.*[C:P2.46;R:RA]*

3 Interpretive Foundations: Hardness and Solver Claims

The claims in later applied sections are theorem-indexed consequences of this section and Sections 4–6.

3.1 Structural Complexity vs Representational Hardness

Definition 3.1 (Structural Complexity)**.** For a fixed decision question (e.g., "is I sufficient for $\mathcal{D}$?"; see Remark 2.21), *structural complexity* means its placement in standard complexity classes within the formal model (coNP, Σ_2^P, etc.), as established by class-membership arguments and reductions.

Definition 3.2 (Representational Hardness)**.** For a fixed decision question and an encoding regime E (Section 2.4), *representational hardness* is the worst-case computational cost incurred by solvers whose input access is restricted to E.

In the mechanized architecture model, this is made explicit as a required-work observable and intrinsic lower bound: 'requiredWork' is total realized work, 'hardnessLowerBound' is the intrinsic floor, and 'hardness_is_irreducible_required_work' proves the lower-bound relation for all nonzero use-site counts; 'requiredWork_eq_affine_in_sites' and 'workGrowthDegree' make the site-count growth law explicit.*(L: DQ562,DQ560,DQ561,DQ563,DQ564,DQ565.)*

Remark 3.3 (Interpretation Contract). This paper keeps the decision question fixed and varies the encoding regime explicitly. Thus, later separations are read as changes in representational hardness under fixed structural complexity, not as changes to the underlying sufficiency semantics. Accordingly, "complete landscape" means complete for this static sufficiency class under the declared regimes; adjacent objective classes are distinct typed objects, not unproved remainder cases of the same class.

Theorem 3.4 (Typed Completeness for the Static Sufficiency Class)**.** *For the static sufficiency class fixed by C1–C4 and declared regime family $\mathcal{R}_{\text{static}}$, the paper provides:*

1. *conditional class-label closures for SUFFICIENCY-CHECK, MINIMUM-SUFFICIENT-SET, and ANCHOR-SUFFICIENCY;*
2. *a conditional explicit/succinct dichotomy closure;*
3. *a regime-indexed mechanized core claim for every declared regime; and*
4. *declared-family exhaustiveness in the typed regime algebra.*

(L: CC76.)

3.2 Solver Integrity and Regime Competence

To keep practical corollaries type-safe, we separate *integrity* (what a solver is allowed to assert) from *competence* (what it can cover under a declared regime), following the certifying-algorithms schema [39, 43]. The induced-relation view used below is standard in complexity/computability presentations: algorithms are analyzed as machines deciding languages or computing functions/relations over encodings [47, 5, 21].

Definition 3.5 (Certifying Solver)**.** Fix a decision question $\mathcal{R} \subseteq \mathcal{X} \times \mathcal{Y}$ and an encoding regime E over $\mathcal{X}$. A *certifying solver* is a pair (Q, V) where:

- Q maps each input $x \in \mathcal{X}$ to either $\mathsf{ABSTAIN}$ or a candidate pair (y, w),
- V is a polynomial-time checker (in $|\text{enc}_E(x)|$) with output in $\{0, 1\}$.

Definition 3.6 (Induced Solver Relation)**.** For any deterministic program M computing a partial map $f_M : \mathcal{X} \rightharpoonup \mathcal{Y}$ on encoded inputs, define

$$\mathcal{R}_M := \{(x, y) \in \mathcal{X} \times \mathcal{Y} : f_M(x) \downarrow \ \wedge\ y = f_M(x)\}.$$

Proposition 3.7 (Universality of Solver Framing)**.** *Every deterministic computer program that computes a (possibly partial) map on encoded inputs can be framed as a solver for its induced relation (Definition 3.6). In this formal sense, all computers are solvers of specific problems.* (L: CC77.)

Proof. Immediate from Definition 3.6: each program induces a relation given by its graph on the domain where it halts. ■

Definition 3.8 (Solver Integrity)**.** A certifying solver (Q, V) has *integrity* for relation $\mathcal{R}$ if:

- (assertion soundness) $Q(x) = (y, w) \implies V(x, y, w) = 1$,
- (checker soundness) $V(x, y, w) = 1 \implies (x, y) \in \mathcal{R}$.

The output ABSTAIN (equivalently, UNKNOWN) is first-class and carries no assertion about membership in $\mathcal{R}$.

Definition 3.9 (Competence Under a Regime)**.** Fix a regime $\Gamma = (\mathcal{X}_\Gamma, E_\Gamma, \mathcal{C}_\Gamma)$ with instance family $\mathcal{X}_\Gamma \subseteq \mathcal{X}$, encoding assumptions E_Γ, and resource bound $\mathcal{C}_\Gamma$. A certifying solver (Q, V) is *competent* on Γ for relation $\mathcal{R}$ if:

- it has integrity for $\mathcal{R}$ (Definition 3.8),
- (coverage) $\forall x \in \mathcal{X}_\Gamma,\ Q(x) \neq \mathsf{ABSTAIN}$,
- (resource bound) $\mathrm{runtime}_Q(x) \leq \mathcal{C}_\Gamma(|\mathrm{enc}_{E_\Gamma}(x)|)$ for all $x \in \mathcal{X}_\Gamma$.

Corollary 3.10 (Universality of Integrity Applicability)**.** *Let M be any deterministic program, viewed as a certifying solver pair (Q, V) for some externally fixed target relation $\mathcal{R}$. Then Definition 3.8 applies unchanged. Thus epistemic integrity is architecture-universal over computing systems once they are cast as solvers for declared tasks.* (L: CC31.)

Proof. Definition 3.8 quantifies over a relation $\mathcal{R}$ and a certifying pair (Q, V); it does not assume any special implementation substrate. ■

[C:P3.7;C3.10;R:TR]

Remark 3.11 (External-Task Non-Vacuity). If the target relation is chosen post hoc from the program's own behavior (for example, $\mathcal{R} = \mathcal{R}_M$ from Definition 3.6), integrity can become tautological and competence claims can be vacuous. Non-vacuous competence claims therefore require the target relation, encoding regime, and resource bound to be declared independently of observed outputs.
[C:D3.8,3.9;P3.57,3.60;R:AR]

Definition 3.12 (ε-Objective Relation Family)**.** An *ε-objective relation family* is a map

$$\varepsilon \mapsto \mathcal{R}_\varepsilon \subseteq \mathcal{X} \times \mathcal{Y}$$

that assigns a certified target relation to each tolerance level $\varepsilon \geq 0$. The exact target is $\mathcal{R}_0$.

Definition 3.13 (ε-Competence Under a Regime)**.** Fix a relation family $(\mathcal{R}_\varepsilon)_{\varepsilon \geq 0}$ and regime Γ. A certifying solver (Q, V) is *ε-competent* on Γ if it is competent on Γ for relation $\mathcal{R}_\varepsilon$ in the sense of Definition 3.9.

Proposition 3.14 (Zero-ε Competence Reduction)**.** *If $\mathcal{R}_0 = \mathcal{R}$, then*

$$\varepsilon\text{-competent at } \varepsilon = 0 \iff \text{competent for exact relation } \mathcal{R}.$$

Moreover, ε-competence implies integrity for the corresponding $\mathcal{R}_\varepsilon$. (L: IC33, IC19.)

Definition 3.15 (Typed Claim Report)**.** For a declared objective family and regime, a solver-side report is one of:

- ABSTAIN,
- EXACT (claiming exact competence/certification),
- EPSILON(ε) (claiming ε-competence/certification).

Proposition 3.16 (Typed Claim Admissibility Discipline)**.** *Under Definitions 3.9, 3.13, and 3.15:*

- ABSTAIN *is always admissible;*
- EXACT *is admissible iff exact competence holds;*
- EPSILON(ε) *is admissible iff ε-competence holds.*

Hence claim type and certificate type must match (no cross-typing of uncertified assertions as certified guarantees). *(L: CC75.)*

Definition 3.17 (Evidence Object for a Typed Report)**.** For declared $(\mathcal{R}, (\mathcal{R}_\varepsilon)_{\varepsilon \geq 0}, \Gamma, Q)$ and report type $r \in \{\mathsf{ABSTAIN}, \mathsf{EXACT}, \mathsf{EPSILON}(\varepsilon)\}$, *evidence* is a first-class witness object:

- for ABSTAIN: trivial witness;
- for EXACT: exact-competence witness;
- for EPSILON(ε): ε-competence witness.

(L: IC2.)

Proposition 3.18 (Evidence–Admissibility Equivalence)**.** *For any declared contract and report type r:*

$$\textit{Evidence object for } r \textit{ exists} \iff \textit{Claim } r \textit{ is admissible.}$$

(L: IC17, IC21, IC20.)

[C:D3.17;P3.16,3.18;R:AR]

Security-game reading (derived). The typed-report layer can be read as a standard claim-verification game under a declared contract: a reporting algorithm emits a report token r and optional evidence, and a checker accepts if and only if the evidence is valid for r. Within this game view, existing results already provide the core security properties: *completeness* for admissible report classes (Proposition 3.18), *soundness* against overclaiming (Propositions 3.16, 3.21), and *evidence-gated confidence* (Propositions 3.23, 3.24). So integrity is enforced as verifiable claim discipline, not as unchecked payload declaration. *[C:P3.16,3.18,3.21,3.23,3.24;R:AR]*

Definition 3.19 (Signaled Typed Report (Guess + Confidence))**.** A *signaled typed report* augments the typed report token $r \in \{\mathsf{ABSTAIN}, \mathsf{EXACT}, \mathsf{EPSILON}(\varepsilon)\}$ with:

- an optional guess payload g,
- self-reflected confidence $p_{\text{self}} \in [0, 1]$,
- certified confidence $p_{\text{cert}} \in [0, 1]$.

- scalar execution witness $t_{\text{run}} \in \mathbb{N}$ (steps actually run),
- optional declared bound $B \in \mathbb{N}$.

Here p_{self} is a self-signal, while p_{cert} is an evidence-gated signal under the declared contract. *(L: IC34.)*

Definition 3.20 (Signal Consistency Discipline)**.** For a signaled typed report $(r, g, p_{\text{self}}, p_{\text{cert}}, t_{\text{run}}, B)$, require:

$$p_{\text{cert}} > 0 \Rightarrow \exists \text{ evidence object for } r.$$

Proposition 3.21 (Exact Claims Require Exact Evidence)**.** *In the typed claim discipline,*

$$\mathit{ClaimAdmissible}(\mathsf{EXACT}) \iff \exists \text{ exact evidence object.}$$

Equivalently, admissible exact claims are exactly those with an evidence witness. *(L: IC35, IC36.)*

Definition 3.22 (Bound-Dependent Completion Fraction)**.** Completion-fraction semantics are defined only when a positive declared bound exists and covers observed runtime:

$$\text{CompletionFractionDefined} \iff \exists b > 0 : \; B = b \;\wedge\; t_{\text{run}} \leq b.$$

(L: IC37.)

Proposition 3.23 (Certified-Confidence Gate)**.** *Under Definition 3.20, positive certified confidence implies typed admissibility:*

$$p_{\text{cert}} > 0 \Rightarrow \mathit{ClaimAdmissible}(r).$$

Thus certified confidence is not a free scalar; it is evidence-gated by the same typed discipline as the report itself. *(L: IC38, IC39.)*

Proposition 3.24 (No Evidence Forces Zero Certified Confidence)**.** *Under Definition 3.20, absence of evidence for the reported type forces:*

$$\neg\exists \text{ evidence object for } r \Rightarrow p_{\text{cert}} = 0.$$

(L: IC40.)

Corollary 3.25 (Exact-No-Competence Zero-Certified Constraint)**.** *For exact reports, if exact competence is unavailable on the declared regime/objective, then any signal-consistent report must satisfy:*

$$p_{\text{cert}} = 0.$$

(L: IC41.)

Proposition 3.26 (Scalar Step Witness Is Always Defined and Falsifiable)**.** *For every signaled report, the execution witness is an exact natural number and any equality claim about it is decidable:*

$$\exists k \in \mathbb{N} : \; t_{\text{run}} = k, \qquad \forall k \in \mathbb{N}, \; (t_{\text{run}} = k) \;\vee\; (t_{\text{run}} \neq k).$$

(L: IC42, IC43.)

Proposition 3.27 (No Completion Fraction Without Declared Bound)**.** *If no declared bound is provided ($B = \varnothing$), completion-fraction semantics are undefined:*

$$B = \varnothing \Rightarrow \neg\,\text{CompletionFractionDefined}.$$

(L: IC44.)

Proposition 3.28 (Completion Fraction Under Explicit Bound)**.** *If a positive declared bound is provided and observed runtime is within bound, completion-fraction semantics are defined:*

$$B = b,\ b > 0,\ t_{\text{run}} \le b \Rightarrow \text{CompletionFractionDefined}.$$

(L: IC45.)

Proposition 3.29 (Abstain Can Carry Guess and Self-Confidence)**.** *For any optional guess payload g and any self-reflected confidence $p_{\text{self}} \in [0, 1]$, there exists a signal-consistent abstain report*

$$(\mathsf{ABSTAIN}, g, p_{\text{self}}, 0, 0, \varnothing).$$

Hence the framework is non-binary at the report-signal layer: abstention can carry a tentative answer and self-reflection without upgrading to certified exactness. (L: IC46.)

Proposition 3.30 (Self-Reflected Confidence Is Not Certification)**.** *Self-reflected confidence alone does not certify exactness: if exact competence is unavailable, there exist exact-labeled reports with arbitrary p_{self} that remain inadmissible under zero certified confidence. (L: IC47.)*

[C:D3.19,3.20,3.22;P3.21,3.23,3.24,3.26,3.27,3.28,3.29,3.30;C3.25;R:AR]

Definition 3.31 (Declared Budget Slice)**.** For a declared regime Γ over state objects and horizon/budget cut $H \in \mathbb{N}$, define the in-scope slice

$$\mathcal{S}_{\Gamma,H} := \{s :\ s \in \Gamma.\text{inScope}\ \wedge\ \text{encLen}_{\Gamma}(s) \le H\}.$$

(L: CC13.)

Proposition 3.32 (Bounded-Slice Irrelevance of a Meta-Coordinate)**.** *Fix coordinate i_∞ and declared slice $\mathcal{S}_{\Gamma,H}$ from Definition 3.31. If optimizer sets are invariant to i_∞ on that slice, i.e.,*

$$\forall s, s' \in \mathcal{S}_{\Gamma,H},\ \Big(\forall j \ne i_\infty,\ s_j = s'_j\Big) \Rightarrow Opt(s) = Opt(s'),$$

then i_∞ is irrelevant on $\mathcal{S}_{\Gamma,H}$, and hence not relevant for the declared task slice. (L: CC32, CC33.)

[C:D3.31;P3.32;R:AR]

Definition 3.33 (Goal Class)**.** A *goal class* is a non-empty set of admissible evaluators over a fixed action/state structure:

$$\mathcal{G} = (\mathcal{U}_{\mathcal{G}}, A, S), \qquad \varnothing \ne \mathcal{U}_{\mathcal{G}} \subseteq \{U : A \times S \to \mathbb{Q}\}.$$

The solver need not know which $U \in \mathcal{U}_{\mathcal{G}}$ is active; it only knows class-membership constraints. *(L: IV1.)*

Definition 3.34 (Interior Pareto Dominance)**.** For a goal class $\mathcal{G}$ and score model $\sigma : S \times \{1, \ldots, n\} \to \mathbb{Q}$, let $J_{\mathcal{G}}$ be the class-tautological coordinate set. State s *interior-Pareto-dominates* s' on $J_{\mathcal{G}}$ when:

$$\forall j \in J_{\mathcal{G}},\ \sigma(s', j) \leq \sigma(s, j) \quad \text{and} \quad \exists j \in J_{\mathcal{G}},\ \sigma(s', j) < \sigma(s, j).$$

(L: IV2, IV3.)

Proposition 3.35 (Interior Dominance Implies Universal Non-Rejection)**.** *If a state s interior-Pareto-dominates s' on a checked coordinate set J, and every admissible evaluator in the goal class is monotone on J, then no admissible evaluator strictly prefers s' over s on that checked slice:*

$$\forall U \in \mathcal{U}_{\mathcal{G}},\ \forall a \in A,\ U(a, s') \leq U(a, s).$$

(L: IV5.)

Proposition 3.36 (Interior Verification Tractability)**.** *If membership in the checked coordinate set is decidable and interior dominance is decidable, then interior verification yields a polynomial-time checker with exact acceptance criterion:*

$$\exists\ \text{verify} : S \times S \to \{0, 1\}, \quad \text{verify}(s, s') = 1 \iff \textit{interior-dominance}(s, s').$$

(L: IV7, IV4.)

Proposition 3.37 (Interior Dominance Is One-Sided)**.** *Interior dominance is one-sided and does not imply full coordinate sufficiency: a counterexample pair outside the checked slice can still violate global optimizer equivalence. (L: IV6, IV8.)*

Corollary 3.38 (Singleton Interior Certificate)**.** *For a singleton checked coordinate i, strict improvement on i with class-monotonicity on $\{i\}$ is already a valid interior certificate of non-rejection. (L: IV9.)*

[C:D3.33,3.34;P3.35,3.36,3.37;C3.38;R:AR]

Definition 3.39 (RLFF Objective (Evidence-Gated Report Reward))**.** Given report type $r \in \{\mathsf{ABSTAIN}, \mathsf{EXACT}, \mathsf{EPSILON}(\varepsilon)\}$ and declared contract, RLFF assigns base reward by report type when evidence exists and applies an explicit inadmissibility penalty otherwise:

$$\text{Reward}(r) = \begin{cases} \text{Base}(r), & \text{if evidence for } r \text{ exists,} \\ -\text{Penalty}, & \text{otherwise.} \end{cases}$$

(L: IC4, IC28, IC29.)

Proposition 3.40 (RLFF Maximizer Is Admissible)**.** *If* ABSTAIN *reward strictly exceeds the inadmissibility branch, then any global RLFF maximizer must be evidence-backed and therefore admissible under the typed-claim discipline. (L: IC31, IC32.)*

Corollary 3.41 (RLFF Abstain Preference Without Certificates)**.** *If* EXACT *and* EPSILON(ε) *have no evidence objects while* ABSTAIN *beats the inadmissibility branch, then RLFF strictly prefers* ABSTAIN *to both report types. (L: IC30.)*

[C:D3.39;P3.40;C3.41;R:AR]

Definition 3.42 (Certainty Inflation)**.** For a typed report r, certainty inflation is the state of emitting r without a matching evidence object. *(L: IC1, IC3.)*

Proposition 3.43 (Certainty Inflation Equals Report Inadmissibility)**.** *For every typed report r:*

$$\mathit{CertaintyInflation}(r) \iff \neg\ \mathit{ClaimAdmissible}(r).$$

For $r = \mathsf{EXACT}$ this is equivalently failure of exact competence. *(L: IC8, IC22.)*

Corollary 3.44 (Hardness-Blocked Regimes Induce Exact Certainty Inflation)**.** *Under the declared hardness/non-collapse premises that block universal exact admissibility, every exact report is certainty-inflated.* *(L: CC19.)*

[C:D3.42;P3.43;C3.44;R:CR]

Definition 3.45 (Raw-vs-Certified Bit Accounting)**.** Fix a declared contract and report type r. A report-bit model declares:

- raw report bits $B_{\text{raw}}(r)$ (payload bits in the asserted report token),
- certificate bits $B_{\text{cert}}(r)$ (bits allocated to the matching evidence class).

Evidence-gated overhead and certified total are then:

$$B_{\text{over}}(r) = \begin{cases} B_{\text{cert}}(r), & \text{if evidence for } r \text{ exists,} \\ 0, & \text{otherwise,} \end{cases} \qquad B_{\text{total}}(r) = B_{\text{raw}}(r) + B_{\text{over}}(r).$$

(L: IC5, IC9, IC12.)

Proposition 3.46 (Raw/Certified Bit Split)**.** *For every typed report r:*

$$B_{\text{total}}(r) = B_{\text{raw}}(r) + B_{\text{over}}(r),$$

with

$$\neg\exists \text{ evidence for } r \Rightarrow B_{\text{over}}(r) = 0, \qquad \exists \text{ evidence for } r \Rightarrow B_{\text{over}}(r) = B_{\text{cert}}(r).$$

Hence $B_{\text{raw}}(r) \leq B_{\text{total}}(r)$ always, and strict inequality holds when evidence exists with positive certificate-bit allocation. *(L: CC11, IC11, IC10, IC16, IC15, IC13, IC14.)*

Theorem 3.47 (Exact Report: Certified-Bit Gap iff Admissibility)**.** *Under a report-bit model with positive exact certificate-bit allocation:*

$$\mathit{ClaimAdmissible}(\mathsf{EXACT}) \iff B_{\text{raw}}(\mathsf{EXACT}) < B_{\text{total}}(\mathsf{EXACT}).$$

Equivalently:

$$B_{\text{total}}(\mathsf{EXACT}) = B_{\text{raw}}(\mathsf{EXACT}) \iff \mathit{ExactCertaintyInflation}.$$

(L: CC18, CC20, CC17.)

Corollary 3.48 (Hardness-Blocked Exact Reports Are Raw-Only)**.** *If exact reporting is inadmissible on the declared contract, then exact accounting collapses to raw-only:*

$$B_{\text{total}}(\mathsf{EXACT}) = B_{\text{raw}}(\mathsf{EXACT}).$$

(L: CC21, IC23.)

[C:D3.45;T3.47;P3.46;C3.48;R:AR]

Corollary 3.49 (No Uncertified Exact-Exhaustion Claim)**.** *If exact competence does not hold on the declared regime/objective, then an* EXACT *report is inadmissible.* *(L: CC41.)*

[C:P3.16;C3.49;R:AR]

Proposition 3.50 (Declared-Contract Separation: Selection vs. Validity)**.** *For solver-side reporting, there are two formally distinct layers:*

- ***Declared-contract selection layer:*** *choosing objective family* $(\mathcal{R}_\varepsilon)_{\varepsilon\geq 0}$*, regime* Γ*, and assumption profile;*
- ***Validity layer:*** *once declared, admissibility of* EXACT/EPSILON(ε)/ABSTAIN *reports is fixed by certificate-typed rules.*

Thus the framework treats contract selection as an external declaration choice, and then enforces mechanically checked admissibility within that declared contract. *(L: CC75, CC22, CC39.)*

Proof. The validity layer is Proposition 3.16: report admissibility is exactly tied to the matching competence certificate type. For exact physical claims outside carve-outs, Proposition 3.54 and Theorem 3.55 require explicit assumptions, and Corollary 3.56 blocks uncertified exact reports under declared non-collapse assumptions. Therefore, once the contract is declared, admissibility is mechanically determined by the typed rules and attached assumption profile. ■

[C:T3.55;P3.16,3.54;C3.56;R:DC]

Definition 3.51 (Exact-Claim Excuse Carve-Outs)**.** For exact physical claims in this framework, we declare four explicit carve-outs:

1. trivial scope (empty in-scope family),
2. tractable class (exact competence available in the active regime),
3. stronger oracle/regime shift (the claim is made under strictly stronger access assumptions),
4. unbounded resources (budget bounds intentionally removed).

Definition 3.52 (Explicit Hardness-Assumption Profile)**.** An explicit hardness-assumption profile for declared class/regime $(\mathcal{R}, \Gamma)$ consists of:

- a non-collapse hypothesis,
- a collapse implication from universal exact competence on $(\mathcal{R}, \Gamma)$.

Definition 3.53 (Well-Typed Exact Physical Claim Policy)**.** An exact physical claim is *well-typed* only if either:

- at least one carve-out from Definition 3.51 is explicitly declared, or
- an explicit hardness-assumption profile (Definition 3.52) is explicitly declared.

Proposition 3.54 (Outside Carve-Outs, Explicit Assumptions Are Required)**.** *If an exact physical claim is well-typed (Definition 3.53) and no carve-out from Definition 3.51 applies, then an explicit hardness-assumption profile must be present.* *(L: CC22.)*

Theorem 3.55 (Claim-Integrity Meta-Theorem)**.** *For declared class/regime objects, every admissible exact-claim policy outside the carve-outs requires explicit assumptions:*

$\neg$ *Excused (Definition 3.51)* $\wedge$ *Well-Typed Exact Claim (Definition 3.53)* $\Rightarrow$ *Has Explicit Hardness Profile (Defini*

(L: CC22.)

Proof. Immediate from Proposition 3.54; this is the universally quantified theorem-level restatement over the typed claim/regime objects. ■

Corollary 3.56 (Outside Carve-Outs, Declared Hardness Blocks Exact Reports)**.** *If no carve-out applies and the declared hardness-assumption profile holds, then* EXACT *reports are inadmissible for every solver on the declared class/regime.* *(L: CC39.)*

[C:T3.55;P3.54;C3.56;R:CR]

Proposition 3.57 (Integrity–Competence Separation)**.** *Integrity and competence are distinct predicates: integrity constrains asserted outputs, while competence adds non-abstaining coverage under resource bounds.* *(L: IC18, IC25.)*

Proof. Take the always-abstain solver $Q_\perp(x) = \mathsf{ABSTAIN}$ with any polynomial-time checker V. Definition 3.8 holds vacuously, so $(Q_\perp, V)$ is integrity-preserving, but it fails Definition 3.9 whenever $\mathcal{X}_\Gamma \neq \emptyset$ because coverage fails. Hence integrity does not imply competence. The converse is immediate because competence includes integrity as a conjunct. ■

Definition 3.58 (Attempted Competence Failure)**.** Fix an exact objective under regime Γ. A solver state is an *attempted competence failure* if:

- integrity holds (Definition 3.8),
- exact competence was actually attempted for the active scope/objective,
- competence on Γ fails for that exact objective (Definition 3.9).

Proposition 3.59 (Attempted-Competence Rationality Matrix)**.** *Let $I, A, C \in \{0, 1\}$ denote integrity, attempted exact competence, and competence available in the active regime. Policy verdict for persistent over-specification is:*

- *if $I = 0$: inadmissible,*
- *if $I = 1$ and $(A, C) = (1, 0)$: conditionally rational,*
- *if $I = 1$ and $(A, C) \in \{(0, 0), (0, 1), (1, 1)\}$: irrational for the same verified-cost objective.*

Hence, in the integrity-preserving plane ($I = 1$), exactly one cell is rational and three are irrational. *(L: IC27, IC7, IC6)*

This separation plus Proposition 3.59 is load-bearing for the regime-conditional trilemma used later: if exact competence is blocked by hardness in a declared regime after an attempted exact procedure, integrity forces one of three responses: abstain, weaken guarantees, or change regime assumptions.*[C:P3.59;R:AR]*

Mechanized status. This separation is machine-checked in `DecisionQuotient/IntegrityCompetence.lean` via: `IC18` and `IC25`; the attempted-competence matrix is mechanized via `IC27`, `IC7`, and `IC6`.

Proposition 3.60 (Integrity-Resource Bound)**.** *Let Γ be a declared regime whose in-scope family includes all instances of SUFFICIENCY-CHECK and whose declared resource bound is polynomial in the encoding length. If $\mathsf{P} \neq \mathsf{coNP}$, then no certifying solver is simultaneously integrity-preserving and competent on Γ for exact sufficiency. Equivalently, for every integrity-preserving solver, at least one of the competence conjuncts must fail on Γ: either full non-abstaining coverage fails or the declared budget bound fails.* *(L: `IC26`, `IC24`, `CC30`.)*

Proof. By Definition 3.9, competence on Γ requires integrity, full in-scope coverage, and budget compliance. Under the coNP-hardness transfer for SUFFICIENCY-CHECK (Theorem 4.6), universal competence on this scope would induce a polynomial-time collapse to $\mathsf{P} = \mathsf{coNP}$. Therefore, under $\mathsf{P} \neq \mathsf{coNP}$, the full competence predicate cannot hold. Since integrity alone is satisfiable (Proposition 3.57, via always-abstain), the obstruction is exactly competence coverage/budget under the declared regime. *[C:T4.6;P3.57,3.60;R:AR]* ■

Proposition 3.61 (Declared-Physics No-Universal-Exact-Certifier Schema)**.** *Fix any declared physical/task class representable by relation $\mathcal{R}$ and regime Γ in the certifying-solver model. If*

$$\big(\exists Q,\ \mathrm{CompetentOn}(\mathcal{R}, \Gamma, Q)\big) \ \Rightarrow\ (\mathsf{P} = \mathsf{coNP}),$$

then under $\mathsf{P} \neq \mathsf{coNP}$ there is no universally exact-competent solver for that declared class. *(L: `CC15`.)*

Corollary 3.62 (No Universal Exact-Claim Admissibility in Hardness-Blocked Classes)**.** *Under the same premises as Proposition 3.61, and under the typed-claim discipline of Definition 3.15, an* EXACT *report is inadmissible for every solver on that declared class. Hence admissible reporting must use* ABSTAIN *or explicitly weakened guarantees.* *(L: `CC38`.)*

[C:P3.61;C3.62;R:CR]

4 Computational Complexity of Decision-Relevant Uncertainty

This section establishes the computational complexity of information sufficiency, formalized as coordinate sufficiency, for an arbitrary decision problem $\mathcal{D}$ with factored state space. Because the problems below are parameterized by $\mathcal{D}$, and the (A, S, U) tuple captures any problem with choices under structured information (Remark 1.1), the hardness results are universal: they apply to every problem with coordinate structure, not to a specific problem domain. We prove three main results:

1. **SUFFICIENCY-CHECK** is coNP-complete
2. **MINIMUM-SUFFICIENT-SET** is coNP-complete (the Σ_2^P structure collapses)
3. **ANCHOR-SUFFICIENCY** (fixed coordinates) is Σ_2^P-complete

Under the model contract of Section 2.5 and the succinct encoding of Section 2.4, these results place exact relevance identification beyond NP-completeness: in the worst case, finding (or certifying) minimal decision-relevant sets is computationally intractable.

Reading convention for this section. Each hardness theorem below is paired with a recovery boundary for the same decision question: when structural-access assumptions in Theorem 6.1 hold, polynomial-time exact procedures are recovered; when they fail in [S], the stated hardness applies. *(L: CC60, CC36, CC4, CC71.)*

4.1 Problem Definitions

Definition 4.1 (Decision Problem Encoding)**.** A *decision problem instance* is a tuple $(A, X_1, \ldots, X_n, U)$ where:

- A is a finite set of alternatives
- $X_1, \ldots, X_n$ are the coordinate domains, with state space $S = X_1 \times \cdots \times X_n$
- $U : A \times S \to \mathbb{Q}$ is the utility function (in the succinct encoding, U is given as a Boolean circuit)

Succinct/circuit encoding. When U is given succinctly as a Boolean or arithmetic circuit, the instance size refers to the circuit description rather than the (possibly exponentially large) truth table. Evaluating $U(a, s)$ on a single pair (a, s) can be done in time polynomial in the circuit size, but quantification over all states (for example, "for all s") ranges over an exponentially large domain and is the source of the hardness results below.

Definition 4.2 (Optimizer Map)**.** For state $s \in S$, define:

$$\mathrm{Opt}(s) := \arg\max_{a \in A} U(a, s)$$

Definition 4.3 (Sufficient Coordinate Set)**.** A coordinate set $I \subseteq \{1, \ldots, n\}$ is *sufficient* if:

$$\forall s, s' \in S : \quad s_I = s'_I \implies \mathrm{Opt}(s) = \mathrm{Opt}(s')$$

where s_I denotes the projection of s onto coordinates in I.

Problem 4.4 (SUFFICIENCY-CHECK)**. Input:** Decision problem $(A, X_1, \ldots, X_n, U)$ and coordinate set $I \subseteq \{1, \ldots, n\}$
Question: Is I sufficient?

Problem 4.5 (MINIMUM-SUFFICIENT-SET)**. Input:** Decision problem $(A, X_1, \ldots, X_n, U)$ and integer k
Question: Does there exist a sufficient set I with $|I| \leq k$?

4.2 Hardness of SUFFICIENCY-CHECK

Classical coNP-completeness methodology follows standard NP/coNP reduction frameworks and polynomial-time many-one reducibility [47, 5]. We instantiate that framework for SUFFICIENCY-CHECK with a mechanized TAUTOLOGY reduction.

Theorem 4.6 (coNP-completeness of SUFFICIENCY-CHECK)**.** *SUFFICIENCY-CHECK is coNP-complete.* *(L: CC61, CC60, L18.)*

Source	Target	Key property preserved
TAUTOLOGY	SUFFICIENCY-CHECK	Tautology iff $\emptyset$ sufficient
$\exists\forall$-SAT	ANCHOR-SUFFICIENCY	Witness anchors iff formula true

Proof. extbfMembership in coNP: The complementary problem INSUFFICIENCY is in NP. A nondeterministic verifier can check a witness pair (s, s') as follows:

1. Verify $s_I = s'_I$ by projecting and comparing coordinates; this takes time polynomial in the input size.
2. Evaluate U on every alternative for both s and s'. When U is given as a circuit each evaluation runs in time polynomial in the circuit size; using these values the verifier checks whether $\mathrm{Opt}(s) \neq \mathrm{Opt}(s')$.

Hence INSUFFICIENCY is in NP and SUFFICIENCY-CHECK is in coNP.

coNP-hardness: We reduce from TAUTOLOGY.

Given Boolean formula $\varphi(x_1, \ldots, x_n)$, construct a decision problem with:

- Alternatives: $A = \{\text{accept}, \text{reject}\}$
- State space: $S = \{\text{reference}\} \cup \{0,1\}^n$ (equivalently, encode this as a product space with one extra coordinate $r \in \{0,1\}$ indicating whether the state is the reference state)
- Utility:

$$\begin{aligned} U(\text{accept}, \text{reference}) &= 1 \\ U(\text{reject}, \text{reference}) &= 0 \\ U(\text{accept}, a) &= \varphi(a) \\ U(\text{reject}, a) &= 0 \quad \text{for assignments } a \in \{0,1\}^n \end{aligned}$$

- Query set: $I = \emptyset$

Claim: $I = \emptyset$ is sufficient $\iff$ φ is a tautology.

($\Rightarrow$) Suppose I is sufficient. Then $\mathrm{Opt}(s)$ is constant over all states. Since $U(\text{accept}, a) = \varphi(a)$ and $U(\text{reject}, a) = 0$:

- $\mathrm{Opt}(a) = \text{accept}$ when $\varphi(a) = 1$
- $\mathrm{Opt}(a) = \{\text{accept}, \text{reject}\}$ when $\varphi(a) = 0$

For Opt to be constant, $\varphi(a)$ must be true for all assignments a; hence φ is a tautology.

($\Leftarrow$) If φ is a tautology, then $U(\text{accept}, a) = 1 > 0 = U(\text{reject}, a)$ for all assignments a. Thus $\mathrm{Opt}(s) = \{\text{accept}\}$ for all states s, making $I = \emptyset$ sufficient. ■

Immediate recovery boundary (SUFFICIENCY-CHECK). Theorem 4.6 is the [S] general-case hardness statement. For the same sufficiency relation, Theorem 6.1 gives polynomial-time recovery under explicit-state, separable, and tree-structured regimes. *(L: cc60, cc71, cc69, cc70, cc72.)*

Mechanized strengthening (all coordinates relevant). The reduction above establishes coNP-hardness using a single witness set $I = \emptyset$. For the ETH-based lower bound in Theorem 5.1, we additionally need worst-case instances where the minimal sufficient set has *linear* size.

We formalized a strengthened reduction in Lean 4: given a Boolean formula φ over n variables, construct a decision problem with n coordinates such that if φ is not a tautology then *every* coordinate is decision-relevant (so $k^* = n$). Intuitively, the construction places a copy of the base gadget at each coordinate and makes the global "accept" condition hold only when every coordinate's local test succeeds; a single falsifying assignment at one coordinate therefore changes the global optimal set, witnessing that coordinate's relevance. This strengthening is mechanized in Lean; see Appendix A.

4.3 Complexity of MINIMUM-SUFFICIENT-SET

Theorem 4.7 (MINIMUM-SUFFICIENT-SET is coNP-complete)**.** *MINIMUM-SUFFICIENT-SET is coNP-complete.* *(L: CC34, CC36.)*

Proof. **Structural observation:** The $\exists\forall$ quantifier pattern suggests Σ_2^P:

$$\exists I\,(|I| \le k)\ \forall s, s' \in S: \quad s_I = s'_I \implies \mathrm{Opt}(s) = \mathrm{Opt}(s')$$

However, this collapses because sufficiency has a simple characterization.

Key lemma: In the Boolean-coordinate collapse model, a coordinate set I is sufficient if and only if I contains all relevant coordinates (proven formally as `sufficient_iff_relevant_subset` / `sufficient_iff_relevantFinset_subset` in Lean):

$$\mathrm{sufficient}(I) \iff \mathrm{Relevant} \subseteq I$$

where $\mathrm{Relevant} = \{i : \exists s, s'.\ s \text{ differs from } s' \text{ only at } i \text{ and } \mathrm{Opt}(s) \neq \mathrm{Opt}(s')\}$. This is the same relevance object as Definition 2.14; Proposition 2.15 gives the minimal-set equivalence in the product-space semantics used by the collapse.

Consequence: The minimum sufficient set is exactly the relevant-coordinate set. Thus MINIMUM-SUFFICIENT-SET asks: "Is the number of relevant coordinates at most k?"

coNP membership: A witness that the answer is NO is a set of $k+1$ coordinates, each proven relevant (by exhibiting s, s' pairs). Verification is polynomial.

coNP-hardness: The $k = 0$ case asks whether no coordinates are relevant, i.e., whether $\emptyset$ is sufficient. This is exactly SUFFICIENCY-CHECK, which is coNP-complete by Theorem 4.6. ■

Immediate recovery boundary (MINIMUM-SUFFICIENT-SET). Theorem 4.7 pairs with Theorem 4.14: exact minimization complexity is governed by relevance-cardinality once the collapse is applied. Recovery then depends on computing relevance efficiently, with structured-access assumptions from Theorem 6.1 giving the corresponding route for the underlying sufficiency computations. *(L: CC36, CC34, CC35, S2P2, CC71.)*

4.4 Anchor Sufficiency (Fixed Coordinates)

We also formalize a strengthened variant that fixes the coordinate set and asks whether there exists an *assignment* to those coordinates that makes the optimal action constant on the induced subcube.

Problem 4.8 (ANCHOR-SUFFICIENCY)**. Input:** Decision problem $(A, X_1, \ldots, X_n, U)$ and fixed coordinate set $I \subseteq \{1, \ldots, n\}$
Question: Does there exist an assignment α to I such that $\mathrm{Opt}(s)$ is constant for all states s with $s_I = \alpha$?

The second-level hardness template used here follows canonical Σ_2^P polynomial-hierarchy characterizations [47, 5], instantiated to the anchor formulation below.

Theorem 4.9 (ANCHOR-SUFFICIENCY is Σ_2^P-complete)**.** *ANCHOR-SUFFICIENCY is Σ_2^P-complete (already for Boolean coordinate spaces).* *(L: CC5, CC4, L2.)*

Proof. **Membership in Σ_2^P:** The problem has the form

$$\exists\alpha\ \forall s \in S:\ (s_I = \alpha) \implies \mathrm{Opt}(s) = \mathrm{Opt}(s_\alpha),$$

which is an $\exists\forall$ pattern.

Σ_2^P-hardness: Reduce from $\exists\forall$-SAT. Given $\exists x\, \forall y\, \varphi(x, y)$ with $x \in \{0,1\}^k$ and $y \in \{0,1\}^m$, if $m = 0$ we first pad with a dummy universal variable (satisfiability is preserved), construct a decision problem with:

- Actions $A = \{\text{YES}, \text{NO}\}$
- State space $S = \{0,1\}^{k+m}$ representing (x, y)
- Utility
$$U(\text{YES}, (x, y)) = \begin{cases} 2 & \text{if } \varphi(x, y) = 1 \\ 0 & \text{otherwise} \end{cases} \qquad U(\text{NO}, (x, y)) = \begin{cases} 1 & \text{if } y = 0^m \\ 0 & \text{otherwise} \end{cases}$$
- Fixed coordinate set $I =$ the x-coordinates.

If $\exists x^\star\, \forall y\, \varphi(x^\star, y) = 1$, then for any y we have $U(\text{YES}) = 2$ and $U(\text{NO}) \leq 1$, so $\text{Opt}(x^\star, y) = \{\text{YES}\}$ is constant. Conversely, fix x and suppose $\exists y_f$ with $\varphi(x, y_f) = 0$.

- If $\varphi(x, 0^m) = 1$, then $\text{Opt}(x, 0^m) = \{\text{YES}\}$. The falsifying assignment must satisfy $y_f \neq 0^m$, where $U(\text{YES}) = U(\text{NO}) = 0$, so $\text{Opt}(x, y_f) = \{\text{YES}, \text{NO}\}$.
- If $\varphi(x, 0^m) = 0$, then $\text{Opt}(x, 0^m) = \{\text{NO}\}$. After padding we have $m > 0$, so choose any $y' \neq 0^m$: either $\varphi(x, y') = 1$ (then $\text{Opt}(x, y') = \{\text{YES}\}$) or $\varphi(x, y') = 0$ (then $\text{Opt}(x, y') = \{\text{YES}, \text{NO}\}$). In both subcases the optimal set differs from $\{\text{NO}\}$.

Hence the subcube for this x is not constant. Thus an anchor assignment exists iff the $\exists\forall$-SAT instance is true. ■

Immediate boundary statement (ANCHOR-SUFFICIENCY). Theorem 4.9 remains a second-level hardness statement in the anchored formulation; unlike MINIMUM-SUFFICIENT-SET, no general collapse to relevance counting is established here, so the corresponding tractability status remains open in this model. *(L: cc5, cc4, L2.)*

4.5 Tractable Subcases

Despite the general hardness, several natural subcases admit efficient algorithms:

Proposition 4.10 (Small State Space)**.** *When $|S|$ is polynomial in the input size (i.e., explicitly enumerable), MINIMUM-SUFFICIENT-SET is solvable in polynomial time.*

Proof. Compute $\text{Opt}(s)$ for all $s \in S$. The minimum sufficient set is exactly the set of coordinates that "matter" for the resulting function, computable by standard techniques. ■

Proposition 4.11 (Linear Utility)**.** *When $U(a, s) = w_a \cdot s$ for weight vectors $w_a \in \mathbb{Q}^n$, MINIMUM-SUFFICIENT-SET reduces to identifying coordinates where weight vectors differ.*

4.6 Implications

Corollary 4.12 (Computational Bottleneck for Exact Minimization)**.** *Under the succinct encoding, exact minimization of sufficient coordinate sets is coNP-hard via the $k = 0$ case, and fixed-anchor minimization is Σ_2^P-complete.* *(L: CC36, CC4)*

Proof. The $k = 0$ case of MINIMUM-SUFFICIENT-SET is SUFFICIENCY-CHECK (Theorem 4.6), giving coNP-hardness for exact minimization. The fixed-anchor variant is exactly Theorem 4.9. ■

The modeling budget for deciding what to model is therefore a computationally constrained resource under this encoding.*[C:Thm. 4.6, Thm. 4.9;R:[S]]*

4.7 The succinct–minimal gap and physical agents

In complexity theory, a *succinct* representation of a combinatorial object is a Boolean circuit of size poly(n) that computes a function whose explicit (truth-table) form has size $2^{\Theta(n)}$. The term is standard but informal: it denotes a size class (polynomial in input variables), not a structural property. In particular, "succinct" does not mean *minimal*. The minimal circuit for a function is the smallest circuit that computes it, a rigorous, unique object. A succinct circuit may be far larger than the minimal one while still qualifying as "succinct" by the poly-size criterion.

The gap. Let C be a succinct circuit computing a utility function U, and let C^* be the minimal circuit for the same function. The *representation gap* $\varepsilon = |C| - |C^*|$ measures how far the agent's encoding is from the tightest possible encoding. This gap has no standard formal definition in the literature; it is the unnamed distance between an informal size class and a rigorous structural minimum.

Proposition 4.13 (Coordinate Representation Gap as a Formal Object)**.** *In the Boolean-coordinate model, let $R_{\text{rel}} = \texttt{relevantFinset}(dp)$ and define*

$$\varepsilon(I) \;:=\; |I \setminus R_{\text{rel}}| + |R_{\text{rel}} \setminus I|.$$

Then:

$$\varepsilon(I) = 0 \iff I = R_{\text{rel}} \iff I \text{ is minimal sufficient.}$$

(L: S2P3, S2P4, S2P5.)

The results of this paper give ε formal complexity consequences. By Theorem 4.7, determining the minimal sufficient coordinate set is coNP-complete under the succinct encoding. By Theorem 5.1, under ETH there exist succinct instances requiring $2^{\Omega(n)}$ time to verify sufficiency. In the formal Boolean-coordinate collapse model, size-bounded $\varepsilon = 0$ search is exactly the relevance-cardinality predicate (*Lean:* `S2P.min_representationGap_zero_iff_relevant_card`). Together: for an agent encoded as a succinct circuit, the worst-case cost of determining whether its own coordinate attention set is minimal is exponential in the number of input coordinates (under ETH). The gap ε is not merely unnamed; it is *computationally irreducible*: no polynomial-time algorithm solves all succinct instances (assuming P $\neq$ coNP), and the hard instance family requires $2^{\Omega(n)}$ time.

Physical agents as circuits. Any physical agent that selects actions based on sensory state can be modeled, in the static snapshot, as a circuit: sensory inputs are coordinates $X_1, \ldots, X_n$; the agent's internal computation maps states to actions; and the quality of that mapping is measured by a utility function U.*[C:Prop. 2.46, Prop. 2.39;R:[bridge-admissible snapshot]]* The explicit-state encoding (truth table) of this mapping has size exponential in the number of sensory dimensions. No physical agent can store it.*[C:Thm. 5.1;R:[E] vs [S+ETH]]* Every physical agent is therefore a succinct encoding, a circuit whose size is bounded by physical resources (neurons, synapses, transistors, parameters) rather than by the size of the state space it navigates.*[C:Thm. 5.1;R:[S]]*

This is not a metaphor. A biological nervous system is a feedforward or recurrent circuit that takes sensory state as input and produces motor commands as output.*[C:Prop. 2.46;R:[represented adjacent class]]* A trained neural network is a parameterized circuit.*[C:Prop. 2.46;R:[represented adjacent class]]* In both cases, the agent *is* a succinct encoding of a utility-to-action mapping, and the question "is this agent attending to the right inputs?" is exactly SUFFICIENCY-CHECK under the succinct encoding. *[C:Thm. 7.2, Thm. 4.6;R:[S]]* The compression that makes a circuit physically realizable, with polynomial size rather than exponential size, is the same compression that makes self-verification exponentially costly in the worst case: the state space the circuit compressed away is precisely the domain that must be exhaustively checked to certify sufficiency.*[C:Thm. 5.1, Prop. 5.3;R:[S+ETH],[Q_fin]]*

The simplicity tax as ε. The simplicity tax (Definition 9.2) measures $|\mathrm{Gap}(M, P)| = |R(P) \setminus A(M)|$: the number of decision-relevant coordinates that the agent's model does not handle natively. In the ε decomposition above, this is exactly the unattended-relevant component $|R_{\mathrm{rel}} \setminus I|$ (*Lean:* `S2P.representationGap_missing_eq_gapCard`, `S2P.representationGap_eq_waste_plus_missing`). For a physical agent modeled as a circuit, this is the coordinate-level manifestation of ε: the agent attends to a superset of what is necessary, or fails to attend to coordinates that are relevant, and faces worst-case cost exponential in n to verify which case holds (Theorem 4.6). Theorem 9.3 then says the total relevance budget is conserved: unhandled coordinates are not eliminated, only redistributed as external per-site cost.

Consequences for learned circuits. Large language models and deep networks are succinct circuits with poly(n) parameters.*[C:Thm. 5.1;R:[S]]* Mechanistic interpretability asks which internal components (attention heads, neurons, layers) are necessary for a given capability; this is SUFFICIENCY-CHECK applied to the circuit's internal coordinate structure. Pruning research asks for the minimal subcircuit that preserves performance; this is MINIMUM-SUFFICIENT-SET. The results of this paper imply that these questions are coNP-hard in the worst case under succinct encoding.*[C:Thm. 4.6, Thm. 4.7;R:[S]]* Empirical progress on interpretability and pruning therefore depends on exploiting the structural properties of specific trained circuits (analogous to the tractable subcases of Theorem 6.1), not on solving the general problem.*[C:Thm. 4.6, Thm. 6.1;R:[S] vs [tractable regimes]]*

Scope and caveats. The formal results above apply to the static, deterministic model fixed by C1–C4 (Section 2.5). Biological neural systems add noise, temporal dynamics, and synaptic plasticity; trained networks add stochastic optimization and distribution shift. These factors may alter empirical tractability relative to the static worst-case bounds. The bridge conditions of Proposition 2.38 delineate when static sufficiency results transfer to sequential settings; beyond those conditions, the governing complexity objects change (Propositions 2.41–2.43). The claims in this subsection are therefore regime-typed consequences of the formal results, not universal assertions about all possible agent architectures.*[C:Prop. 2.38, Thm. 2.44, Prop. 2.46;R:[represented adjacent class]]*

4.8 Quantifier Collapse for MINIMUM-SUFFICIENT-SET

Theorem 4.14 (Collapse of the Apparent $\exists\forall$ Structure)**.** *The apparent second-level predicate*

$$\exists I\,(|I| \le k)\ \forall s, s' \in S:\ s_I = s'_I \implies Opt(s) = Opt(s')$$

is equivalent to the coNP predicate $|Relevant| \le k$*, where*

$$Relevant = \{i : \exists s, s'.\ s \text{ differs from } s' \text{ only at } i \text{ and } Opt(s) \ne Opt(s')\}.$$

Consequently, MINIMUM-SUFFICIENT-SET is governed by coNP certificates rather than a genuine Σ_2^P *alternation.* *(L: CC34, CC35, S2P2.)*

Proof. By the formal equivalence `sufficient_iff_relevant_subset` (finite-set form `sufficient_iff_relevantFinset_subset`), a coordinate set I is sufficient iff Relevant $\subseteq I$. Therefore:

$$\exists I\ (|I| \le k \wedge \text{sufficient}(I)) \iff \exists I\ (|I| \le k \wedge \text{Relevant} \subseteq I) \iff |\text{Relevant}| \le k.$$

So the existential-over-universal presentation collapses to counting the relevant coordinates.

A NO certificate for $|\text{Relevant}| \le k$ is a list of $k+1$ distinct relevant coordinates, each witnessed by two states that differ only on that coordinate and yield different optimal sets; this is polynomially verifiable. Hence the resulting predicate is in coNP, matching Theorem 4.7.

This also clarifies why ANCHOR-SUFFICIENCY remains Σ_2^P-complete: once an anchor assignment is existentially chosen, the universal quantifier over the residual subcube does not collapse to a coordinate-counting predicate. ■

5 Encoding-Regime Separation

The hardness results of Section 4 apply to worst-case instances under the succinct encoding. This section states an encoding-regime separation: an explicit-state upper bound versus a succinct-encoding worst-case lower bound, and an intermediate query-access obstruction family. The complexity class is a property of the encoding measured against a fixed decision question (Remark 2.21), not a property of the question itself: the same sufficiency question admits polynomial-time algorithms under one encoding and worst-case exponential cost under another.

Model note. Theorem 5.1 has Part 1 in [E] (time polynomial in $|S|$) and Part 2 in [S+ETH] (time exponential in n). Proposition 5.3 is [Q_fin] (finite-state Opt-oracle core), while Propositions 5.6 and 5.10 are [Q_bool] interface refinements (value-entry and state-batch). Relative to Definition 2.22, Propositions 5.7 and 5.14 are explicit simulation-transfer instances. The [E] vs [S+ETH] split in Theorem 5.1 is a same-question encoding/cost-model separation, not a bidirectional simulation claim between those regimes. These regimes are defined in Section 2.4 and are not directly comparable as functions of one numeric input-length parameter.

Theorem 5.1 (Explicit–Succinct Regime Separation)**.** *Let* $\mathcal{D} = (A, X_1, \ldots, X_n, U)$ *be a decision problem with* $|S| = N$ *states. Let* k^* *be the size of the minimal sufficient set.*

1. ***[E] Explicit-state upper bound:*** *Under the explicit-state encoding, SUFFICIENCY-CHECK is solvable in time polynomial in* N *(e.g.* $O(N^2|A|)$*).*

2. ***[S+ETH] Succinct lower bound (worst case):*** *Assuming ETH, there exists a family of succinctly encoded instances with n coordinates and minimal sufficient set size $k^* = n$ such that SUFFICIENCY-CHECK requires time $2^{\Omega(n)}$.*

(L: CC16, CC23, CC24.)

Proof. **Part 1 (Explicit-state upper bound):** Under the explicit-state encoding, SUFFICIENCY-CHECK is decidable in time polynomial in N by direct enumeration: compute $\mathrm{Opt}(s)$ for all $s \in S$ and then check all pairs (s, s') with $s_I = s'_I$.

Equivalently, for any fixed I, the projection map $s \mapsto s_I$ has image size $|S_I| \leq |S| = N$, so any algorithm that iterates over projection classes (or over all state pairs) runs in polynomial time in N. Thus, in particular, when $k^* = O(\log N)$, SUFFICIENCY-CHECK is solvable in polynomial time under the explicit-state encoding.

Remark (bounded coordinate domains). In the general model $S = \prod_i X_i$, for a fixed I one always has $|S_I| \leq \prod_{i \in I} |X_i|$ (and $|S_I| \leq N$). If the coordinate domains are uniformly bounded, $|X_i| \leq d$ for all i, then $|S_I| \leq d^{|I|}$.

Part 2 (Succinct ETH lower bound, worst case): A strengthened version of the TAUTOLOGY reduction used in Theorem 4.6 produces a family of instances in which the minimal sufficient set has size $k^* = n$: given a Boolean formula φ over n variables, we construct a decision problem with n coordinates such that if φ is not a tautology then *every* coordinate is decision-relevant (thus $k^* = n$). This strengthening is mechanized in Lean (see Appendix A). Under the Exponential Time Hypothesis (ETH) [28, 5], TAUTOLOGY requires time $2^{\Omega(n)}$ in the succinct encoding, so SUFFICIENCY-CHECK inherits a $2^{\Omega(n)}$ worst-case lower bound via this reduction. ■

Corollary 5.2 (Regime Separation (by Encoding))**.** *There is a clean separation between explicit-state tractability and succinct worst-case hardness (with respect to the encodings in Section 2.4):*

- *Under the explicit-state encoding, SUFFICIENCY-CHECK is polynomial in $N = |S|$.*
- *Under ETH, there exist succinctly encoded instances with $k^* = \Omega(n)$ (indeed $k^* = n$) for which SUFFICIENCY-CHECK requires $2^{\Omega(n)}$ time.*

For Boolean coordinate spaces ($N = 2^n$), the explicit-state bound is polynomial in 2^n (exponential in n), while under ETH the succinct lower bound yields $2^{\Omega(n)}$ time for the hard family in which $k^* = \Omega(n)$.

Proposition 5.3 (Finite-State Query Lower-Bound Core via Empty-Set Subproblem)**.** *In the finite-state query regime [Q_fin], let S be any finite state type with $|S| \geq 2$ and let $Q \subset S$ with $|Q| < |S|$. Then there exist two decision problems $\mathcal{D}_{\text{yes}}, \mathcal{D}_{\text{no}}$ over the same state space such that:*

- *their oracle views on all states in Q are identical,*
- *$\emptyset$ is sufficient for $\mathcal{D}_{\text{yes}}$,*
- *$\emptyset$ is not sufficient for $\mathcal{D}_{\text{no}}$.*

Consequently, no deterministic query procedure using fewer than $|S|$ state queries can solve SUFFICIENCY-CHECK on all such instances; the worst-case query complexity is $\Omega(|S|)$. *(L: CC50, L7, L13.)*

Proof. This is exactly the finite-state indistinguishability theorem CC4. For any $|Q| < |S|$, there is an unqueried hidden state s_0 producing oracle-indistinguishable yes/no instances with opposite truth values on the $I = \emptyset$ subproblem. Since SUFFICIENCY-CHECK contains that subproblem, fewer than $|S|$ queries cannot be correct on both instances, yielding the $\Omega(|S|)$ worst-case lower bound. ■

Corollary 5.4 (Information Barrier in Query Regimes)**.** *For the fixed sufficiency question, finite query interfaces induce an information barrier:*

Q_fin (Opt-oracle): with $|Q| < |S|$, there exist yes/no instances indistinguishable on all queried states but with opposite truth values on the $I = \emptyset$ subproblem.

Q_bool value-entry and state-batch interfaces preserve the same obstruction at cardinality $< 2^n$.

Hence the barrier is representational-access based: without querying enough of the hidden state support, exact certification is blocked by indistinguishability rather than by search-strategy choice. *(L: CC27, CC29, CC28.)*

Corollary 5.5 (Boolean-Coordinate Instantiation)**.** *In the Boolean-coordinate state space $S = \{0,1\}^n$, Proposition 5.3 yields the familiar $\Omega(2^n)$ worst-case query lower bound for Opt-oracle access.* *(L: CC49; core: L6.)*

Proposition 5.6 (Value-Entry Query Lower Bound)**.** *In the mechanized Boolean value-entry query submodel [Q_bool], for any deterministic procedure using fewer than 2^n value-entry queries $(a, s) \mapsto U(a, s)$, there exist two queried-value-indistinguishable instances with opposite truth values for SUFFICIENCY-CHECK on the $I = \emptyset$ subproblem. Therefore worst-case value-entry query complexity is also $\Omega(2^n)$.* *(L: L9, L19.)*

Proof. The theorem L2 constructs, for any query set of cardinality $< 2^n$, an unqueried hidden state s_0 and a yes/no instance pair that agree on all queried values but disagree on $\emptyset$-sufficiency. The auxiliary bound L6 ensures that fewer than 2^n value-entry queries cannot cover all states, so the indistinguishability argument applies in the worst case. ■

Proposition 5.7 (Subproblem-to-Full Transfer Rule)**.** *If every full-problem solver induces a solver for a fixed subproblem, then any lower bound for that subproblem lifts to the full problem.* *(L: CC58, CC59, CC52.)*

This is the restriction-map instance of Definition 2.22: a solver for the full regime induces one for the restricted subproblem regime, so lower bounds transfer.

Proposition 5.8 (Randomized Robustness (Seedwise))**.** *In [Q_bool], for any query set with cardinality $< 2^n$, the indistinguishable yes/no pair from Proposition 5.3 forces one decoding error* per random seed *for any seed-indexed decoder from oracle transcripts. Consequently, finite-support randomization does not remove the obstruction: averaging preserves a constant error floor on the hard pair.* *(L: L11, L12, L5.)*

Proposition 5.9 (Randomized Robustness (Weighted Form))**.** *For any finite-support seed weighting μ, the same hard pair satisfies a weighted identity: the weighted sum of yes-error and no-error equals total seed weight. Hence randomization cannot collapse both errors simultaneously.* *(L: L23, L24.)*

Proposition 5.10 (State-Batch Query Lower Bound)**.** *In [Q_bool], the same $\Omega(2^n)$ lower bound holds for a state-batch oracle that returns the full Boolean-action utility tuple at each queried state.* *(L: L8, L14.)*

Proposition 5.11 (Finite-State Empty-Subproblem Generalization)**.** *The empty-subproblem indistinguishability lower-bound core extends from Boolean-vector state spaces to any finite state type with at least two states.* *(L: L7, L13.)*

Proposition 5.12 (Adversary-Family Tightness by Full Scan)**.** *For the const/spike adversary family used in the query lower bounds, querying all states distinguishes the pair; thus the lower-bound family is tight up to constant factors under full-state scan.* *(L: L10.)*

Proposition 5.13 (Weighted Query-Cost Transfer)**.** *Let $w(q)$ be per-query cost and $w_{\min}$ a lower bound on all queried costs. Any cardinality lower bound $|Q| \geq L$ lifts to weighted cost:*

$$\sum_{q \in Q} w(q) \;\geq\; w_{\min} \cdot L.$$

(L: L21, L22.)

Proposition 5.14 (Oracle-Lattice Transfer: Batch ⇒ Entry)**.** *In [Q_bool], agreement on state-batch oracle views over touched states implies agreement on all value-entry views for the corresponding entry-query set.* *(L: L20, CC43.)*

This is the oracle-transducer instance of Definition 2.22: batch transcripts induce entry transcripts while preserving indistinguishability for the target sufficiency question.

Proposition 5.15 (Oracle-Lattice Strictness: Opt vs Value Entries)**.** *'Opt'-oracle views are strictly coarser than value-entry views: there exist instances with identical 'Opt' views but distinguishable value entries.* *(L: L3, L4.)*

Remark 5.16 (Instantiation of Definitions 3.1 and 3.2). Theorem 5.1 and Propositions 5.3–5.6 keep the structural problem fixed (same sufficiency relation) and separate representational hardness by access regime: explicit-state access exposes the boundary $s \mapsto \mathrm{Opt}(s)$, finite-state query access already yields $\Omega(|S|)$ lower bounds at the Opt-oracle level (Boolean instantiation: $\Omega(2^n)$), value-entry/state-batch interfaces preserve the obstruction in [Q_bool], and succinct access can hide structure enough to force ETH-level worst-case cost on a hard family.

This regime-typed landscape identifies tractability under [E], a finite-state query lower-bound core under [Q_fin] with Boolean interface refinements under [Q_bool], and worst-case intractability under [S+ETH] for the same underlying decision question.

The structural reason for this separation is enumerable access. Under the explicit-state encoding, the mapping $s \mapsto \mathrm{Opt}(s)$ is directly inspectable and sufficiency verification reduces to scanning state pairs, at cost polynomial in $|S|$. Under succinct encoding, the circuit compresses an exponential state space into polynomial size; universal verification over that compressed domain, namely "for all s, s' with $s_I = s'_I$, does $\mathrm{Opt}(s) = \mathrm{Opt}(s')$?", carries worst-case cost exponential in n, because the domain the circuit compressed away cannot be reconstructed without undoing the compression.

5.1 Budgeted Physical Crossover

The [E] vs [S] split can be typed against an explicit physical budget. Let $E(n)$ and $S(n)$ denote explicit and succinct representation sizes for the same decision question at scale parameter n, and let B be a hard representation budget. These crossover claims are interpreted through the physical-to-core encoding discipline of Section 2.7: physical assumptions are transferred explicitly into core assumptions, and encoded physical counterexamples map to core failures on the encoded slice. *[C:P2.32;C2.33;R:AR]*

Proposition 5.17 (Budgeted Crossover Witness)**.** *If $E(n) > B$ and $S(n) \leq B$ for some n, then explicit representation is infeasible at (B, n) while succinct representation remains feasible at (B, n).* *(L: CC45, PBC1.)*

Proof. This is exactly the definitional split encoded in PBC1: explicit infeasibility is $B < E(n)$ and succinct feasibility is $S(n) \leq B$. The theorem CC2 returns both conjuncts. ■

Proposition 5.18 (Above-Cap Crossover Existence)**.** *Assume there is a global succinct-size cap C and explicit size is unbounded:*

$$\forall n,\ S(n) \leq C, \qquad \forall B,\ \exists n,\ B < E(n).$$

Then for every budget $B \geq C$, there exists a crossover witness at budget B. *(L: CC44, PBC2.)*

Proof. Fix $B \geq C$. By unboundedness of E, choose n with $B < E(n)$. By the cap assumption, $S(n) \leq C \leq B$, so (B, n) is a crossover witness. The mechanized closure theorem is CC1. ■

Proposition 5.19 (Crossover Does Not Imply Exact-Certification Competence)**.** *Fix any collapse schema "exact certification competence ⇒ complexity collapse" for the target exact-certification predicate. Under the corresponding non-collapse assumption, exact-certification competence is impossible even when a budgeted crossover witness exists.* *(L: CC46.)*

Proof. The mechanized theorem CC3 bundles the crossover split with the same logical closure used in Proposition 3.60: representational feasibility and certification hardness are distinct predicates. ■

Proposition 5.20 (Crossover Policy Closure Under Integrity)**.** *In the certifying-solver model (Definitions 3.8–3.9), assume:*

- *a budgeted crossover witness at (B, n),*
- *the same collapse implication as Proposition 3.60,*
- *a solver that is integrity-preserving for the declared relation.*

Then either full in-scope non-abstaining coverage fails, or declared budget compliance fails. Equivalently, in uncertified hardness-blocked regions, integrity is compatible with ABSTAIN/UNKNOWN *but not with unconditional exact-competence claims.* *(L: CC47.)*

5.2 Thermodynamic Lift in Declared Model

The crossover and hardness statements are complexity-theoretic. The thermodynamic lift follows the same assumption discipline as the rest of the paper: claims are exactly those derivable from the declared conversion model and declared bit-operation lower-bound premises.

Proposition 5.21 (Bit-to-Energy/Carbon Lift)**.** *Fix a declared thermodynamic conversion model with constants (λ, ρ) where λ lower-bounds energy per irreversible bit operation and ρ lower-bounds carbon per joule. If a bit-operation lower bound $b_{\mathrm{LB}} \leq b_{\mathrm{used}}$ holds, then:*

$$E_{\mathrm{LB}} = \lambda b_{\mathrm{LB}} \leq \lambda b_{\mathrm{used}}, \qquad C_{\mathrm{LB}} = \rho E_{\mathrm{LB}} \leq \rho(\lambda b_{\mathrm{used}}).$$

(L: CC65, CC66.)

Proposition 5.22 (Hardness–Thermodynamics Bundle)**.** *Under the same non-collapse/collapse schema used by Proposition 3.60, exact-certification competence is impossible; combined with a declared bit-operation lower bound, this yields simultaneous lower bounds on energy and carbon in the declared conversion model.* *(L: CC67.)*

Proposition 5.23 (Mandatory Physical Cost)**.** *Within the declared linear thermodynamic model, if*

$$\lambda > 0, \quad \rho > 0, \quad b_{\mathrm{LB}} > 0,$$

then both lower bounds are strictly positive:

$$E_{\mathrm{LB}} = \lambda b_{\mathrm{LB}} > 0, \qquad C_{\mathrm{LB}} = \rho E_{\mathrm{LB}} > 0.$$

(L: CC68.)

Proposition 5.24 (Conserved Additive Accounting)**.** *Within the same declared linear model, lower-bound accounting is additive over composed bit-operation lower bounds:*

$$E_{\mathrm{LB}}(b_1 + b_2) = E_{\mathrm{LB}}(b_1) + E_{\mathrm{LB}}(b_2), \qquad C_{\mathrm{LB}}(b_1 + b_2) = C_{\mathrm{LB}}(b_1) + C_{\mathrm{LB}}(b_2).$$

(L: CC64.)

6 Tractable Special Cases: When You Can Solve It

We distinguish the encodings of Section 2.4. The tractability results below state the model assumption explicitly. Structural insight: hardness dissolves exactly when the full decision boundary $s \mapsto \mathrm{Opt}(s)$ is recoverable in polynomial time from the input representation; the three cases below instantiate this single principle. Concretely, each tractable regime corresponds to a specific structural insight (explicit boundary exposure, additive separability, or tree factorization) that removes the hardness witness; this supports reading the general-case hardness as missing structural access in the current representation rather than as an intrinsic semantic barrier.

Theorem 6.1 (Tractable Subcases)**.** *SUFFICIENCY-CHECK is polynomial-time solvable in the following cases:*

1. ***Explicit-state encoding:*** *The input contains the full utility table over $A \times S$. SUFFICIENCY-CHECK runs in $O(|S|^2|A|)$ time; if $|A|$ is constant, $O(|S|^2)$.*
2. ***Separable utility (any encoding):*** $U(a, s) = f(a) + g(s)$.
3. ***Tree-structured utility with explicit local factors (succinct structured encoding):*** *There exists a rooted tree on coordinates and local functions u_i such that*

$$U(a, s) = \sum_i u_i\big(a, s_i, s_{\mathrm{parent}(i)}\big),$$

with the root term depending only on (a, s_{root}) and all u_i given explicitly as part of the input.

(L: CC69, CC70, CC72, CC71.)

6.1 Explicit-State Encoding

Proof of Part 1. Given the full table of $U(a, s)$, compute $\mathrm{Opt}(s)$ for all $s \in S$ in $O(|S||A|)$ time. For SUFFICIENCY-CHECK on a given I, verify that for all pairs (s, s') with $s_I = s'_I$, we have $\mathrm{Opt}(s) = \mathrm{Opt}(s')$. This takes $O(|S|^2|A|)$ time by direct enumeration and is polynomial in the explicit input length. If $|A|$ is constant, the runtime is $O(|S|^2)$. ■

6.2 Separable Utility

Proof of Part 2. If $U(a, s) = f(a) + g(s)$, then:

$$\mathrm{Opt}(s) = \arg\max_{a \in A}[f(a) + g(s)] = \arg\max_{a \in A} f(a)$$

The optimal action is independent of the state. Thus $I = \emptyset$ is always sufficient. ■

6.3 Tree-Structured Utility

Proof of Part 3. Assume the tree decomposition and explicit local tables as stated. For each node i and each value of its parent coordinate, compute the set of actions that are optimal for some assignment of the subtree rooted at i. This is a bottom-up dynamic program that combines local tables with child summaries; each table lookup is explicit in the input. A coordinate is relevant if and only if varying its value changes the resulting optimal action set. The total runtime is polynomial in n, $|A|$, and the size of the local tables. ■

6.4 Practical Implications

Condition	Examples
Explicit-state encoding	Small or fully enumerated state spaces
Separable utility	Additive costs, linear models
Tree-structured utility	Hierarchies, causal trees

Proposition 6.2 (Heuristic Reusability)**.** *Let $\mathcal{C}$ be a structure class for which SUFFICIENCY-CHECK is polynomial (Theorem 6.1). If membership in $\mathcal{C}$ is decidable in polynomial time, then the combined detect-then-check procedure is a sound, polynomial-time heuristic applicable to all future instances in $\mathcal{C}$. For each subcase of Theorem 6.1, structure detection is polynomial (under the declared representation assumptions).* (L: `CC53`, `CC7`, `CC55`, `CC74`.)

Remark 6.3 (Heuristics and Integrity). Proposition 6.2 removes the complexity-theoretic obstruction to integrity-preserving action on detectable tractable instances: integrity no longer *forces* abstention. Whether the agent acts still requires competence (Definition 3.9): budget and coverage must also be satisfied. An agent that detects structure class $\mathcal{C}$, applies the corresponding polynomial checker, and abstains when detection fails maintains integrity; it never claims sufficiency without verification. Mistaking the heuristic for the general solution, claiming polynomial-time competence on a coNP-hard problem, violates integrity (Proposition 3.60).

7 Regime-Conditional Corollaries

This section derives regime-typed engineering corollaries from the core complexity theorems. Theorem 7.2 maps configuration simplification to SUFFICIENCY-CHECK; Theorems 4.6, 4.7, and 5.1 then yield exact minimization consequences under [S] and [S+ETH].

Regime tags used below follow Section 2.5: [S], [S+ETH], [E], [S_bool]. Any prescription that requires exact minimization is constrained by these theorem-level bounds.*[C:T4.6,4.7,5.1;R:S,S+ETH]* Theorem 7.4 implies that persistent failure to isolate a minimal sufficient set is a boundary-characterization signal in the current model, not a universal irreducibility claim.*[C:T7.4;R:S]*

Conditional rationality criterion. For the objective "minimize verified total cost while preserving integrity," persistent over-specification is admissible only in the *attempted competence failure* cell (Definition 3.58): after an attempted exact procedure, if exact irrelevance cannot be certified efficiently, integrity retains uncertified coordinates rather than excluding them.*[C:P3.59;R:AR]* When exact competence is available in the active regime (e.g., Theorem 6.1 and the exact-identifiability criterion), proven-irrelevant coordinates are removable; persistent over-specification is irrational for that same objective.*[C:T6.1;P3.59;R:E,STR]* Proposition 3.59 makes this explicit: in the integrity-preserving matrix, one cell is rational and three are irrational, so irrationality is the default verdict. *[C:P3.59;R:AR] (L: IC18, IC25, IC7, CC71, S2P1.)*

Remark 7.1 (Regime Contract for Engineering Corollaries). All claims in this section are formal corollaries under the declared model assumptions.

- Competence claims are indexed by the regime tuple of Definition 3.9; prescriptions are meaningful only relative to feasible resources under that regime (bounded-rationality feasibility discipline) [71, 5].*[C:P3.57;R:AR]*
- Integrity (Definition 3.8) forbids overclaiming beyond certifiable outputs; ABSTAIN/UNKNOWN is first-class when certification is unavailable.*[C:P3.57;R:AR]*
- Therefore, hardness results imply a regime-conditional trilemma: abstain, weaken guarantees (heuristics/approximation), or change encoding/structural assumptions to recover competence.*[C:T4.6,5.1;P3.59;R:S,S+ETH]*

7.1 Configuration Simplification is SUFFICIENCY-CHECK

Because SUFFICIENCY-CHECK is a meta-problem parameterized by an arbitrary decision problem $\mathcal{D}$, real engineering problems with factored configuration spaces are instances of the hardness landscape established above.

Theorem 7.2 (C1–C4: Configuration Simplification Reduces to SUFFICIENCY-CHECK)**.** *Given a software system with configuration parameters $P = \{p_1, \ldots, p_n\}$ and observed behaviors $B = \{b_1, \ldots, b_m\}$, the problem of determining whether parameter subset $I \subseteq P$ preserves all behaviors is equivalent to SUFFICIENCY-CHECK. (L: CR1)*

Proof. Construct decision problem $\mathcal{D} = (A, X_1, \ldots, X_n, U)$ where:

- Actions $A = B \cup \{\perp\}$, where $\perp$ is a sentinel "no-observed-behavior" action
- Coordinates X_i = domain of parameter p_i
- State space $S = X_1 \times \cdots \times X_n$
- For $b \in B$, utility $U(b, s) = 1$ if behavior b occurs under configuration s, else 0
- Sentinel utility $U(\perp, s) = 1$ iff no behavior in B occurs under configuration s, else 0

Then

$$\mathrm{Opt}(s) = \begin{cases} \{b \in B : b \text{ occurs under configuration } s\}, & \text{if this set is nonempty,} \\ \{\bot\}, & \text{otherwise.} \end{cases}$$

So the optimizer map exactly encodes observed-behavior equivalence classes, including the empty-behavior case.

Coordinate set I is sufficient iff:

$$s_I = s'_I \implies \mathrm{Opt}(s) = \mathrm{Opt}(s')$$

This holds iff configurations agreeing on parameters in I exhibit identical behaviors.

Therefore, "does parameter subset I preserve all behaviors?" is exactly SUFFICIENCY-CHECK for the constructed decision problem. ■

Remark 7.3 (Reduction Scope). The reduction above requires only:

1. a finite behavior set,
2. parameters with finite domains, and
3. an evaluable behavior map from configurations to achieved behaviors.

These are exactly the model-contract premises C1–C3 instantiated for configuration systems.

Theorem 7.4 (S: Over-Modeling as Boundary-Signal Corollary)**.** *By contraposition of Theorem 7.2, if no coordinate set can be certified as exactly relevance-identifying (Definition 2.16) for the modeled system, then the decision boundary is not completely characterized by the current parameterization.* *(L: cc40, cc6)*

Proof. Assume the decision boundary were completely characterized by the current parameterization. Then, via Theorem 7.2, the corresponding sufficiency instance admits exact relevance membership, hence a coordinate set that satisfies Definition 2.16. Contraposition gives the claim: persistent failure of exact relevance identification signals incomplete characterization of decision relevance in the model. ■

7.2 Cost Asymmetry Under ETH

We now prove a cost asymmetry result under the stated cost model and complexity constraints.[1]

Theorem 7.5 (S+ETH: Cost-Asymmetry Consequence)**.** *Consider an engineer specifying a system configuration with n parameters. Let:*

- $C_{over}(k)$ = *cost of maintaining k extra parameters beyond minimal*
- $C_{find}(n)$ = *cost of finding minimal sufficient parameter set*
- C_{under} = *expected cost of production failures from underspecification*

Assume ETH in the succinct encoding model of Section 2.4. Then:

[1] Naive subset enumeration still gives an intuitive baseline of $O(2^n)$ checks, but that is an algorithmic upper bound; the theorem below uses ETH for the lower-bound argument.

1. *Exact identification of minimal sufficient sets has worst-case finding cost $C_{find}(n) = 2^{\Omega(n)}$. (Under ETH, SUFFICIENCY-CHECK has a $2^{\Omega(n)}$ lower bound in the succinct model, and exact minimization subsumes this decision task.)*

2. *Maintenance cost is linear: $C_{over}(k) = O(k)$.*

3. *Under ETH, exponential finding cost dominates linear maintenance cost for sufficiently large n.*

Therefore, there exists n_0 such that for all $n > n_0$, the finding-vs-maintenance asymmetry satisfies:

$$C_{over}(k) < C_{find}(n) + C_{under}$$

Within [S+ETH], persistent over-specification is unresolved boundary characterization, not proof that all included parameters are intrinsically necessary. Conversely, when exact competence is available in the active regime, certifiably irrelevant coordinates are removable; persistence is irrational for the same cost-minimization objective.[C:T7.5;P3.59;R:S+ETH,TA] (L: CC12, HD13.)

Proof. Under ETH, the TAUTOLOGY reduction used in Theorem 4.6 yields a $2^{\Omega(n)}$ worst-case lower bound for SUFFICIENCY-CHECK in the succinct encoding. Any exact algorithm that outputs a minimum sufficient set can decide whether the optimum size is 0 by checking whether the returned set is empty; this is exactly the SUFFICIENCY-CHECK query for $I = \emptyset$. Hence exact minimal-set finding inherits the same exponential worst-case lower bound.

Maintaining k extra parameters incurs:

- Documentation cost: $O(k)$ entries
- Testing cost: $O(k)$ test cases
- Migration cost: $O(k)$ parameters to update

Total maintenance cost is $C_{\text{over}}(k) = O(k)$.

The eventual dominance step is mechanized in HD8: for fixed linear-overhead parameter k and additive constant C_{under} there is n_0 such that $k < 2^n + C_{\text{under}}$ for all $n \geq n_0$. Therefore:

$$C_{\text{over}}(k) \ll C_{\text{find}}(n)$$

For any fixed nonnegative C_{under}, the asymptotic dominance inequality remains and only shifts the finite threshold n_0. ■

Corollary 7.6 (Impossibility of Automated Configuration Minimization)**.** *Assuming $\mathsf{P} \neq \mathsf{coNP}$, there exists no polynomial-time algorithm that:*

1. *Takes an arbitrary configuration file with n parameters*

2. *Identifies the minimal sufficient parameter subset*

3. *Guarantees correctness (no false negatives)*

(L: CC37)

Proof. Such an algorithm would solve MINIMUM-SUFFICIENT-SET in polynomial time, contradicting Theorem 4.7 (assuming $\mathsf{P} \neq \mathsf{coNP}$). ■

Remark 7.7. Corollary 7.6 is a formal boundary statement: an always-correct polynomial-time minimizer for arbitrary succinct inputs would collapse P and coNP.

Remark 7.8 (FPT Scope Caveat). The practical force of worst-case hardness depends on instance structure, especially k^*. If SUFFICIENCY-CHECK is FPT in parameter k^*, then small-k^* families can remain tractable even under succinct encodings. The strengthened mechanized gadget (`all_coords_relevant_of_not_tautology`) still proves existence of hard families with $k^* = n$; what is typical in deployed systems is an empirical question outside this formal model.

7.3 Regime-Conditional Operational Corollaries

Theorems 7.4 and 7.5 yield the following conditional operational consequences:

1. Conservative retention under unresolved relevance. If irrelevance cannot be certified efficiently under the active regime, the admissible policy is to retain a conservative superset of parameters.*[C:T7.4;R:S]*

2. Heuristic selection as weakened-guarantee mode. Under [S+ETH], exact global minimization can be exponentially costly in the worst case (Theorem 7.5); methods such as AIC/BIC/cross-validation instantiate the "weaken guarantees" branch of Definition 3.9 [1, 56, 66, 25, 50].*[C:T7.5;R:S+ETH]*

3. Full-parameter inclusion as an $O(n)$ upper-bound strategy. Under [S+ETH], if exact minimization is unresolved, including all n parameters is an $O(n)$ maintenance upper-bound policy that avoids false irrelevance claims.*[C:T7.5;R:S+ETH]*

4. Irrationality outside attempted-competence-failure conditions. If the active regime admits exact competence (tractable structural-access conditions or exact relevance identifiability), or if exact competence was never actually attempted, continued over-specification is irrational; hardness no longer justifies it for the stated objective.*[C:T6.1;P3.59;R:TA] (L: CC71, IC6, S2P1.)*

These corollaries are direct consequences of the hardness/tractability landscape: over-specification is an attempted-competence-failure policy, not a default optimum. Outside that cell, the admissible exits are to shift to tractable regimes from Theorem 6.1 or adopt explicit approximation commitments.*[C:T6.1;P3.59;R:RG]*

8 Dominance Theorems for Hardness Placement

Regime for this section: the mechanized Boolean-coordinate model [S_bool] plus the architecture cost model defined below. The centralization-vs-distribution framing follows established software-architecture criteria about modular decomposition and long-run maintenance tradeoffs [49, 10, 6, 60].

8.1 Over-Specification as Diagnostic Signal

Corollary 8.1 (Persistent Over-Specification)**.** *In the mechanized Boolean-coordinate model, if a coordinate is relevant and omitted from a candidate set I, then I is not sufficient.* *(L: S2P6)*

Proof. This is the contrapositive of S2P3. ■

Corollary 8.2 (Exact Relevance Identifiability Criterion)**.** *In the mechanized Boolean-coordinate model, for any candidate set I:*

$$I \text{ is exactly relevance-identifying} \iff \big(I \text{ is sufficient and } I \subseteq R_{\text{rel}}\big),$$

where R_{rel} is the full relevant-coordinate set. *(L: S2P1)*

Proof. This is exactly S2P1, with $R_{\text{rel}} = \texttt{relevantFinset}$. ■

Remark 8.3 (Over-Specification Is Regime-Conditional, Not Default). In this paper's formal typing, persistent over-specification is admissible only under attempted competence failure: after an attempted exact procedure, exact relevance competence remains unavailable in the active regime, so integrity retains uncertified coordinates (Section 3; Section 7).*[C:P3.59;R:AR]* Once exact competence is available in the active regime (Corollaries 8.7–8.9 together with Corollary 8.2), certifiably irrelevant coordinates are removable; persistent over-specification is irrational for the same objective (verified total-cost minimization).*[C:T6.1;P3.59;R:TA] (L: IC18, IC25, IC7, L15, L16, L17, S2P1.)*

8.2 Architectural Decision Quotient

Definition 8.4 (Architectural Decision Quotient)**.** For a software system with configuration space S and behavior space B:

$$\mathrm{ADQ}(I) = \frac{|\{b \in B : b \text{ achievable with some } s \text{ where } s_I \text{ fixed}\}|}{|B|}$$

Proposition 8.5 (ADQ Ordering)**.** *For coordinate sets I, J in the same system, if $\mathrm{ADQ}(I) < \mathrm{ADQ}(J)$, then fixing I leaves a strictly smaller achievable-behavior set than fixing J. (L: CC2)*

Proof. The denominator $|B|$ is shared. Thus $\mathrm{ADQ}(I) < \mathrm{ADQ}(J)$ is equivalent to a strict inequality between the corresponding achievable-behavior set cardinalities. ■

8.3 Corollaries for Practice

Corollary 8.6 (Cardinality Criterion for Exact Minimization)**.** *In the mechanized Boolean-coordinate model, existence of a sufficient set of size at most k is equivalent to the relevance set having cardinality at most k. (L: S2P2)*

Proof. By S2P2, sufficiency of size $\leq k$ is equivalent to a relevance-cardinality bound $\leq k$ in the Boolean-coordinate model. ■

Corollary 8.7 (Bounded-Regime Tractability)**.** *When the bounded-action or explicit-state conditions of Theorem 6.1 hold, minimal modeling can be solved in polynomial time in the stated input size. (L: L15)*

Proof. This is the bounded-regime branch of Theorem 6.1, mechanized as L3. ■

Corollary 8.8 (Separable-Utility Tractability)**.** *When utility is separable with explicit factors, sufficiency checking is polynomial in the explicit-state regime. (L: L16)*

Proof. This is the separable-utility branch of Theorem 6.1, mechanized as L4. ■

Corollary 8.9 (Tree-Structured Tractability)**.** *When utility factors form a tree structure with explicit local factors, sufficiency checking is polynomial in the explicit-state regime. (L: L17)*

Proof. This is the tree-factor branch of Theorem 6.1, mechanized as L5. ■

Corollary 8.10 (Mechanized Hard Family)**.** *There is a machine-checked family of reduction instances where, for non-tautological source formulas, every coordinate is relevant ($k^* = n$), exhibiting worst-case boundary complexity. (L: L1)*

Proof. The strengthened reduction proves that non-tautological source formulas induce instances where every coordinate is relevant; this is mechanized as L1. ■

8.4 Hardness Distribution: Right Place vs Wrong Place

Definition 8.11 (Hardness Distribution)**.** Let P be a problem family under the succinct encoding of Section 2.4. In this section, baseline hardness $H(P; n)$ denotes worst-case computational step complexity on instances with n coordinates (equivalently, as a function of succinct input length L) in the fixed encoding regime. A *solution architecture* S partitions this baseline hardness into:

- $H_{\text{central}}(S)$: hardness paid once, at design time or in a shared component
- $H_{\text{distributed}}(S)$: hardness paid per use site

For n use sites, total realized hardness is:

$$H_{\text{total}}(S) = H_{\text{central}}(S) + n \cdot H_{\text{distributed}}(S)$$

Proposition 8.12 (Baseline Lower-Bound Principle)**.** *For any problem family P measured by $H(P; n)$ above, any solution architecture S and any number of use sites $n \geq 1$, if $H_{total}(S)$ is measured in the same worst-case step units over the same input family, then:*

$$H_{total}(S) = H_{central}(S) + n \cdot H_{distributed}(S) \geq H(P; n).$$

For SUFFICIENCY-CHECK, Theorem 5.1 provides the baseline on the hard succinct family: $H(\textit{SUFFICIENCY-CHECK}; n) = 2^{\Omega(n)}$ under ETH. (L: HD20)

Proof. By definition, $H(P; n)$ is a worst-case lower bound for correct solutions in this encoding regime and cost metric. Any such solution architecture decomposes total realized work as $H_{\text{central}} + n \cdot H_{\text{distributed}}$, so that total cannot fall below the baseline. ■

Definition 8.13 (Hardness Efficiency)**.** The *hardness efficiency* of solution S with n use sites is:

$$\eta(S, n) = \frac{H_{\text{central}}(S)}{H_{\text{central}}(S) + n \cdot H_{\text{distributed}}(S)}$$

(L: HD10)

Proposition 8.14 (Efficiency Equivalence)**.** *For fixed n and positive total hardness, larger $\eta(S, n)$ is equivalent to a larger central share of realized hardness.* (L: HD10)

Proof. From Definition 8.13, $\eta(S, n)$ is exactly the fraction of total realized hardness paid centrally. ■

Definition 8.15 (Right vs Wrong Hardness Placement)**.** For a solution architecture S in this linear model:

$$\text{right hardness} \iff H_{\text{distributed}}(S) = 0, \qquad \text{wrong hardness} \iff H_{\text{distributed}}(S) > 0.$$

(L: HD11, HD12)

Theorem 8.16 (Centralization Dominance)**.** *Let $S_{\text{right}}, S_{\text{wrong}}$ be architectures over the same problem family with*

$$H_{\text{distributed}}(S_{\text{right}}) = 0, \quad H_{\text{central}}(S_{\text{right}}) > 0, \quad H_{\text{distributed}}(S_{\text{wrong}}) > 0,$$

and let $n > \max\big(1, H_{\text{central}}(S_{\text{right}})\big)$. Then:

1. *Lower total realized hardness:*

$$H_{total}(S_{\text{right}}) < H_{total}(S_{\text{wrong}})$$

2. *Fewer error sites: errors in centralized components affect 1 location; errors in distributed components affect n locations*

3. *Quantified leverage: moving one unit of work from distributed to central saves exactly $n - 1$ units of total realized hardness*

(L: HD1, HD2)

Proof. (1) and (2) are exactly the bundled theorem HD1. (3) is exactly HD2. ■

Corollary 8.17 (Right-Place vs Wrong-Place Hardness)**.** *In the linear model, a right-hardness architecture strictly dominates a wrong-hardness architecture once use-site count exceeds central one-time hardness. Formally, for architectures $S_{\text{right}}, S_{\text{wrong}}$ over the same problem family, if S_{right} has right hardness, S_{wrong} has wrong hardness, and $n > H_{\text{central}}(S_{\text{right}})$, then*

$$H_{\text{central}}(S_{\text{right}}) + n\, H_{\text{distributed}}(S_{\text{right}}) < H_{\text{central}}(S_{\text{wrong}}) + n\, H_{\text{distributed}}(S_{\text{wrong}}).$$

(L: HD16)

Proof. This is the mechanized theorem HD10. ■

Proposition 8.18 (Dominance Modes)**.** *This section uses two linear-model dominance modes plus generalized nonlinear dominance and boundary modes:*

1. ***Strict threshold dominance:*** *Corollary 8.17 gives strict inequality once $n > H_{\text{central}}(S_{\text{right}})$.*

2. ***Global weak dominance:*** *under the decomposition identity used in HD3, centralized hardness placement is never worse for all $n \geq 1$.*

3. ***Generalized nonlinear dominance:*** *under bounded-vs-growing site-cost assumptions (Theorem 8.22), right placement strictly dominates beyond a finite threshold without assuming linear per-site cost.*

4. ***Generalized boundary mode:*** *without those growth-separation assumptions, strict dominance is not guaranteed (Proposition 8.24).*

Proof. Part (1) is Corollary 8.17. Part (2) is exactly HD3. Part (3) is Theorem 8.22. Part (4) is Proposition 8.24. ■

Illustrative Instantiation (Type Systems). Consider a capability C (e.g., provenance tracking) with one-time central cost H_{central} and per-site manual cost $H_{\text{distributed}}$:

Approach	H_{central}	$H_{\text{distributed}}$
Native type system support	High (learning cost)	Low (type checker enforces)
Manual implementation	Low (no new concepts)	High (reimplement per site)

The table is schematic; the formal statement is Corollary 8.19. The type-theoretic intuition behind this instantiation is consistent with classic accounts of abstraction and static interface guarantees [12, 53, 51, 36, 62, 22].

Corollary 8.19 (Type-System Threshold)**.** *For the formal native-vs-manual architecture instance, native support has lower total realized cost for all*

$$n > H_{\text{baseline}}(P),$$

where $H_{\text{baseline}}(P)$ *corresponds to the Lean identifier* `intrinsicDOF(P)` *in module* `HardnessDistribution`*.* *(L: HD14)*

Proof. Immediate from HD9. ■

8.5 Extension: Non-Additive Site-Cost Models

Definition 8.20 (Generalized Site Accumulation)**.** Let $C_S : \mathbb{N} \to \mathbb{N}$ be a per-site accumulation function for architecture S. Define generalized total realized hardness by

$$H^{\text{gen}}_{\text{total}}(S, n) = H_{\text{central}}(S) + C_S(n).$$

Definition 8.21 (Eventual Saturation)**.** A cost function $f : \mathbb{N} \to \mathbb{N}$ is *eventually saturating* if there exists N such that for all $n \geq N$, $f(n) = f(N)$.

Theorem 8.22 (Generalized Dominance by Growth Separation)**.** *Let*

$$H^{\text{gen}}_{total}(S, n) = H_{central}(S) + C_S(n).$$

For two architectures $S_{\text{right}}, S_{\text{wrong}}$*, suppose there exists* $B \in \mathbb{N}$ *such that:*

1. $C_{S_{\text{right}}}(m) \leq B$ *for all* m *(bounded right-side per-site accumulation),*
2. $m \leq C_{S_{\text{wrong}}}(m)$ *for all* m *(identity-lower-bounded wrong-side growth).*

Then for every

$$n > H_{central}(S_{\text{right}}) + B,$$

one has

$$H^{\text{gen}}_{total}(S_{\text{right}}, n) < H^{\text{gen}}_{total}(S_{\text{wrong}}, n).$$

(L: HD8)

Proof. This is exactly the mechanized theorem HD6. ■

Corollary 8.23 (Eventual Generalized Dominance)**.** *If condition (1) above holds and there exists* N *such that condition (2) holds for all* $m \geq N$*, then there exists* N_0 *such that for all* $n \geq N_0$*,*

$$H^{\text{gen}}_{total}(S_{\text{right}}, n) < H^{\text{gen}}_{total}(S_{\text{wrong}}, n).$$

(L: HD9)

Proof. Immediate from HD7. ■

Proposition 8.24 (Generalized Assumption Boundary)**.** *In the generalized model, strict right-vs-wrong dominance is not unconditional. There are explicit counterexamples:*

1. *If wrong-side growth lower bounds are dropped, right-side strict dominance can fail for all* n*.*
2. *If right-side boundedness is dropped, strict dominance can fail for all* n *even when wrong-side growth is linear.*

(L: HD7, HD6)

Proof. This is exactly the pair of mechanized boundary theorems listed above. ■

Theorem 8.25 (Linear Model: Saturation iff Zero Distributed Hardness)**.** *In the linear model of this section,*

$$H_{total}(S, n) = H_{central}(S) + n \cdot H_{distributed}(S),$$

the function $n \mapsto H_{total}(S, n)$ is eventually saturating if and only if $H_{distributed}(S) = 0$. *(L: HD19)*

Proof. This is exactly the mechanized equivalence theorem above. ■

Theorem 8.26 (Generalized Model: Saturation is Possible)**.** *There exists a generalized site-cost model with eventual saturation. In particular, for*

$$C_K(n) = \begin{cases} n, & n \leq K \\ K, & n > K, \end{cases}$$

both C_K and $n \mapsto H_{central} + C_K(n)$ are eventually saturating. *(L: HD17, HD5)*

Proof. This is the explicit construction mechanized in Lean. ■

Corollary 8.27 (Positive Linear Slope Cannot Represent Saturation)**.** *No positive-slope linear per-site model can represent the saturating family above for all n.* *(L: HD15)*

Proof. This follows from the mechanized theorem that any linear representation of the saturating family must have zero slope. ■

Mechanized strengthening reference. The strengthened all-coordinates-relevant reduction is presented in Section 4 ("Mechanized strengthening") and formalized in `Reduction_AllCoords.lean` via `all_coords_relevant_of_not_tautology`.

The next section develops the major practical consequence of this framework: the Simplicity Tax Theorem.

9 Conservation Law for Decision-Relevant Coordinates

The load-bearing fact in this section is not the set identity itself; it is the difficulty of shrinking the required set $R(P)$ in the first place. By Theorem 4.6 (and Theorem 4.7 for minimization), exact relevance identification is intractable in the worst case under succinct encoding. The identities below therefore quantify how unresolved relevance is redistributed between central and per-site work.*[C:T4.6,4.7;R:S]*

Definition 9.1. Let $R(P)$ be the required dimensions (those affecting Opt) and $A(M)$ the dimensions model M handles natively. The *expressive gap* is $\mathrm{Gap}(M, P) = R(P) \setminus A(M)$.

Definition 9.2 (Simplicity Tax)**.** The *simplicity tax* is the size of the expressive gap:

$$\mathrm{SimplicityTax}(M, P) := |\mathrm{Gap}(M, P)|.$$

Theorem 9.3 (Redistribution Identity)**.** $|Gap(M, P)| + |R(P) \cap A(M)| = |R(P)|$. *The total cannot be reduced, only redistributed between "handled natively" and "handled externally."* *(L: HD4)*

Proof. In the finite-coordinate model this is the exact set-cardinality identity

$$|R \setminus A| + |R \cap A| = |R|,$$

formalized as HD5. ■

Remark 9.4 (Why this is nontrivial in context). The algebraic identity in Theorem 9.3 is elementary. Its force comes from upstream hardness: reducing $|R(P)|$ by exact relevance minimization is worst-case intractable under the succinct encoding, so redistribution is often the only tractable lever available.

Theorem 9.5 (Linear Growth)**.** *For n decision sites:*

$$\mathit{TotalExternalWork} = n \times \mathit{SimplicityTax}(M, P).$$

(L: HD21)

Proof. This is by definition of per-site externalization and is mechanized as HD12. ■

Theorem 9.6 (Amortization)**.** *Let $H_{central}$ be the one-time cost of using a complete model. There exists*

$$n^* = \left\lfloor \frac{H_{central}}{\mathit{SimplicityTax}(M, P)} \right\rfloor$$

such that for $n > n^*$, the complete model has lower total cost. *(L: HD3)*

Proof. For positive per-site tax, the threshold inequality

$$n > \left\lfloor \frac{H_{\text{central}}}{\text{SimplicityTax}} \right\rfloor \implies H_{\text{central}} < n \cdot \text{SimplicityTax}$$

is mechanized as HD4. ■

Corollary 9.7 (Gap Externalization)**.** *If $\mathit{Gap}(M, P) \neq \emptyset$, then external handling cost scales linearly with the number of decision sites.* *(L: HD21, HD18)*

Proof. The exact linear form is HD12. When the gap is nonempty (positive tax), monotone growth with n is HD11. ■

Corollary 9.8 (Exact Minimization Criterion)**.** *For mechanized Boolean-coordinate instances, "there exists a sufficient set of size at most k" is equivalent to "the relevant-coordinate set has cardinality at most k."* *(L: S2P2)*

Proof. This is S2P2. ■

Appendix A provides theorem statements and module paths for the corresponding Lean formalization.

10 Related Work

10.1 Formalized Complexity and Mechanized Proof Infrastructure

Formalizing complexity-theoretic arguments in proof assistants remains comparatively sparse. Lean and Mathlib provide the current host environment for this artifact [16, 17, 67]. Closest mechanized precedents include verified computability/complexity developments in Coq and Isabelle [21, 32, 45, 44, 26], and certifying toolchain work that treats proofs as portable machine-checkable evidence [34, 43].

Recent LLM-evaluation work also highlights inference nondeterminism from numerical precision and hardware configuration, reinforcing the value of proof-level claims whose validity does not depend on stochastic inference trajectories [72].

10.2 Computational Decision Theory

The complexity of decision-making has been studied extensively. Foundational treatments of computational complexity and strategic decision settings establish the baseline used here [47, 5]. Our work extends this to the meta-question of identifying relevant information. Decision-theoretic and information-selection framing in this paper also sits in the tradition of statistical sufficiency and signaling/information economics [20, 65, 42, 40, 29].

Large-sample Bayesian-network structure learning and causal identification are already known to be hard in adjacent formulations [13, 61, 30]. Our object differs: we classify coordinate sufficiency for optimal-action invariance, with mechanized reductions and regime typing.

Relation to prior hardness results. Closest adjacent results are feature-selection/model-selection hardness results for predictive subset selection [9, 3]. Relative to those works, this paper changes two formal objects: (i) the reductions are machine-checked (TAUTOLOGY and $\exists\forall$-SAT mappings with explicit polynomial bounds), and (ii) the output is a regime-typed hardness/tractability map for decision relevance under explicit encoding assumptions. The target predicate here is decision relevance, not predictive compression.

10.3 Succinct Representations and Regime Separation

Representation-sensitive complexity is established in planning and decision-process theory: classical and compactly represented MDP/planning problems exhibit sharp complexity shifts under different input models [48, 41, 37]. The explicit-vs-succinct separation in this paper is the same representation-sensitive phenomenon for the coordinate-sufficiency predicate.

The distinction in this paper is the object and scope of the classification: we classify *decision relevance* (sufficiency, anchor sufficiency, and minimum sufficient sets) for a fixed decision relation, with theorem-level regime typing and mechanized reduction chains.

10.4 Oracle and Query-Access Lower Bounds

Query-access lower bounds are a standard source of computational hardness transfer [18, 11]. Our `[Q_fin]` and `[Q_bool]` results are in this tradition, but specialized to the same sufficiency predicate used throughout the paper: they establish access-obstruction lower bounds while keeping the underlying decision relation fixed across regimes.

Complementary companion work studies zero-error identification under constrained observation and proves rate-query lower bounds plus matroid structure of minimal distinguishing query sets [63].

Our object here is different: sufficiency of coordinates for optimal-action invariance in a decision problem, with coNP/Σ_2^P classification and regime-typed transfer theorems.

10.5 Energy Landscapes and Chemical Reaction Networks

The geometric interpretation of decision complexity connects to the physics of energy landscapes. Wales established a widely used picture in which molecular configurations form high-dimensional landscapes with basins and transition saddles [68, 69, 70]. Protein-folding theory uses the same landscape language via funnel structure toward native states [46]. In this paper's formal model, the corresponding core object is coordinate-projection invariance of the optimizer map: sufficiency asks whether fixing coordinates preserves Opt, while insufficiency is witnessed by agreeing projections with different optimal sets (Definition 2.4, Proposition 2.6).*[C:D2.4;P2.6;R:DM](L: DP7,DP8.)*

Section 2.7 makes this transfer rule theorem-level: encoded physical claims are derived from core claims through an explicit assumption-transfer map, and an encoded physical counterexample refutes the corresponding core rule on that encoded slice.*[C:P2.32;C2.33;R:AR](L: CT6,CT5,CT8,CT7.)*

Under that mapping, the explicit-vs-succinct split is the mechanized statement of access asymmetry: [E] admits polynomial verification on explicit state access, while [S+ETH] admits worst-case exponential lower bounds on succinct inputs (Theorem 5.1); finite query access already exhibits indistinguishability barriers (Proposition 5.3). The complexity separation is mechanized. *[C:T5.1;P5.3;R:E,S+ETH,Q_fin](L: CC16,CC23,CC24,CC50.)*

The geometric core is mechanized directly: product-space slice cardinality, hypercube node-count identity, and node-vs-edge separation are proved in Lean at the decision abstraction level, with edge witnesses tied exactly to non-sufficiency.*(L: DQ3, L25, L68, DP21.)*

The circuit-to-decision instantiation is also mechanized in a separate bridge layer: geometry and dynamics are represented as typed components of a circuit object; decision circuits add an explicit interpretation layer; molecule-level constructors instantiate both views.*(L: DQ519, DQ518, DQ517, DQ155, DQ154, DQ159, DQ160.)*

Chemical reaction networks provide a concrete physical class where this encoding applies directly. Prior work establishes hardness for output optimization, reconfiguration, and reconstruction in CRN settings [4, 2, 19]. Those domain-specific CRN complexity results are external literature claims, not mechanized in this Lean artifact. The mechanized statement here is the transfer principle: once a CRN decision task is encoded as a C1–C4 decision problem, it inherits the same sufficiency hardness/tractability regime map (Theorems 7.2, 4.6, 6.1).*[C:T7.2,4.6,6.1;R:E,S,TA](L: CR1,CC61,CC69,CC70,CC71,CC72.)*

The physical bridge is not a single-domain add-on: Theorem 2.35 composes law-objective semantics, reduction equivalence, and hardness lower-bound accounting in one mechanized theorem, so physical instantiation and complexity claims share one proof chain.*[C:T2.35;R:AR](L: CT4.)*

For matter-only dynamics, the law-induced objective specialization is mechanized for arbitrary universes (arbitrary state/action types and transition-feasibility relations): the utility schema is induced from transition laws, and under a strict allowed-vs-blocked gap with nonempty feasible set, the optimizer equals the feasible-action set; with logical determinism at the action layer, the optimizer is singleton.*[C:P2.34;R:AR](L: DQ8, DQ5, DQ7, DQ6.)*

The interface-time semantics are also explicit and mechanized: timed states are $\mathbb{N}$-indexed, one decision event is exactly one tick, run length equals elapsed interface time, and this unit-time law is invariant across substrate tags under interface-consistency assumptions.*[C:P2.27,2.28,2.29,2.30;R:AR](L: DQ548,DQ537,DQ539,DQ534,DQ541,DQ549.)*

Regime-side interface machinery is also formalized as typed access objects with an explicit simulation preorder and certificate-upgrade statements. The access-channel and five-way composition

theorems are encoded as assumption-explicit composition laws rather than informal metaphors.*(L: PC10, PC12, PC15, PC17, PC19, PC20, PC21.)*

10.6 Thermodynamic Costs and Landauer's Principle

The thermodynamic lift in Section 5 converts declared bit-operation lower bounds into declared energy/carbon lower bounds via linear conversion constants. In this artifact, those conversion consequences are mechanized conditionally through Propositions 5.21, 5.22, 5.23, and 5.24. Landauer's principle and reversible-computation context [33, 7] define the physical assumption set; the constants are declared premises, not Lean-derived constants.*[C:P5.21,5.22,5.23,5.24;R:AR](L: CC65,CC66,CC67,CC68,CC64.)*

Coupling this to integrity yields a mechanized policy condition: when exact competence is hardness-blocked in the declared regime, integrity requires abstention or explicitly weakened guarantees (Propositions 3.60, 3.59). A stronger behavioral claim such as "abstention is globally utility-optimal for every objective" is *not* currently mechanized and remains objective-dependent. *[C:P3.60,3.59;R:AR](L: IC26,IC24,CC30,IC27,IC7,IC6.)*

10.7 Feature Selection

In machine learning, feature selection asks which input features are relevant for prediction. This is known to be NP-hard in general [9, 25]. Our results show the decision-theoretic analog is coNP-complete for both checking and minimization. *[C:T4.6,4.7;R:S](L: CC61,CC36.)*

10.8 Value of Information

The value of information (VOI) framework [27, 52] quantifies the maximum rational payment for information. Our work addresses a different question: not the *value* of information, but the *complexity* of identifying which information has value.

10.9 Model Selection

Statistical model selection (AIC/BIC/cross-validation) provides practical heuristics for choosing among models [1, 56, 66]. Our results formalize the regime-level reason heuristic selection appears: without added structural assumptions, exact optimal model selection inherits worst-case intractability, so heuristic methods implement explicit weakened-guarantee policies for unresolved structure. *[C:T4.6,5.1,6.1;R:E,S,S+ETH](L: CC61,CC16,CC69,CC70,CC71.)*

10.10 Information Compression vs Certification Length

The paper now makes an explicit two-part information accounting object (Definition 3.45): raw report bits and evidence-backed certification bits. This sharpens the information-theoretic reading of typed claims: compression of report payload is not equivalent to compression of certifiable truth conditions [57, 14, 64].

From a zero-error viewpoint, this distinction is structural rather than stylistic: compressed representations can remain short while exact decodability depends on confusability structure and admissible decoding conditions [58, 31, 38, 15]. The same separation appears here between short reports and evidence-backed exact claims.

The rate-distortion and MDL lines make the same split in different language: there is a difference between achievable compression rate, computational procedure for obtaining/optimizing that rate, and certifiable model adequacy [59, 8, 54, 24, 35].

Formally, Theorem 3.47 and Corollary 3.48 characterize the exact-report boundary: admissible exact reports require a strict certified-bit gap above raw payload, while hardness-blocked exact reports collapse to raw-only accounting. This directly links representational succinctness to certification cost under the declared contract, rather than treating "short reports" as automatically high-information claims. *[C:T3.47;P3.46;C3.48;R:AR](L: CC18,CC11,CC21.)*

10.11 Certifying Outputs and Proof-Carrying Claims

Our integrity layer matches the certifying-algorithms pattern: algorithms emit candidate outputs together with certificates that can be checked quickly, separating *producing* claims from *verifying* claims [39, 43]. In this paper, Definition 3.8 is exactly that soundness discipline.

At the systems level, this is the same architecture as proof-carrying code: a producer ships evidence and a consumer runs a small checker before accepting the claim [43, 39]. Our competence definition adds the regime-specific coverage/resource requirement that certifying soundness alone does not provide.

The feasibility qualifier in Definition 3.9 yields the bounded-rationality constraint used in this paper: admissible policy is constrained by computational feasibility under the declared resource model (Remark 2.24) [71, 5]. *[C:P3.57;R:AR] (L: IC18.)*

Cryptographic-verifiability perspective (scope). The role of cryptography in this paper is structural verifiability, not secrecy: the relevant analogy is certificate-carrying claims with lightweight verification, not confidential encoding [23, 43]. Concretely, the typed-report discipline is modeled as a game-based reporting interface: a prover emits (r, π) and a verifier accepts exactly when π certifies the declared report type under the active contract. In that model, Evidence–Admissibility Equivalence gives completeness for admissible report types, while Typed Claim Admissibility plus Exact Claims Require Exact Evidence give soundness against exact overclaiming. Certified-confidence constraints then act as evidence-gated admissibility constraints on top of that verifier interface, so raw payload bits and certified bits represent distinct resources rather than stylistic notation variants. *[C:P3.18,3.16,3.21,3.23;R:AR](L: IC2,CC75,IC35,IC38.)*

Three-axis integration. In the cited literature, these pillars are treated separately: representation-sensitive hardness, query-access lower bounds, and certifying soundness disciplines. This paper composes all three for the same decision-relevance object in one regime-typed and machine-checked framework. *[C:T5.1;P5.3,3.16;R:S+ETH,Qf,AR](L: CC16,CC50,CC75.)*

11 Conclusion

Methodology and Disclosure

Role of LLMs in this work. This paper was developed through human-AI collaboration. The author provided the core intuitions: the connection between decision-relevance and computational complexity, the conjecture that SUFFICIENCY-CHECK is coNP-complete, and the insight that the Σ_2^{P} structure collapses for MINIMUM-SUFFICIENT-SET. Large language models (Claude, GPT-4) served as implementation partners for proof drafting, Lean formalization, and LaTeX generation.

The Lean 4 proofs were iteratively refined: the author specified the target statements, the LLM proposed proof strategies, and the Lean compiler served as the arbiter of correctness. The complexity-theoretic reductions required careful human oversight to ensure the polynomial bounds were correctly established.

What the author contributed: The problem formulations (SUFFICIENCY-CHECK, MINIMUM-SUFFICIENT-SET, ANCHOR-SUFFICIENCY), the hardness conjectures, the tractability conditions, and the connection to over-modeling in engineering practice.

What LLMs contributed: LaTeX drafting, Lean tactic exploration, reduction construction assistance, and prose refinement.

The proofs are machine-checked; their validity is independent of generation method. This methodology is disclosed for academic transparency.

Summary of Results

This paper establishes the computational complexity of coordinate sufficiency problems:

- **SUFFICIENCY-CHECK** is coNP-complete (Theorem 4.6)
- **MINIMUM-SUFFICIENT-SET** is coNP-complete (Theorem 4.7)
- **ANCHOR-SUFFICIENCY** is Σ_2^{P}-complete (Theorem 4.9)
- An encoding-regime separation contrasts explicit-state polynomial-time (polynomial in $|S|$) with a succinct worst-case ETH lower bound witnessed by a hard family with $k^* = n$ (Theorem 5.1)
- Full intermediate query-access lower bounds are formalized as a finite-state Opt-oracle core ($\Omega(|S|)$, Boolean instantiation $\Omega(2^n)$) plus Boolean-interface refinements for value-entry and state-batch access, with explicit subproblem-to-full transfer, weighted randomized robustness, and oracle-lattice transfer/strictness closures (Propositions 5.3–5.15)
- Tractable subcases exist for explicit-state encoding, separable utility, and tree-structured utility with explicit local factors (Theorem 6.1)
- Discrete-time interface semantics are formalized: time is $\mathbb{N}$-valued, decision events are exactly one-tick transitions, run length equals elapsed time, and substrate-consistent realizations preserve the same one-unit event law (Propositions 2.27–2.30)
- Physical-system transport is formalized: encoded physical claims are derived from core rules through explicit assumption transfer, physical counterexamples map to encoded core failures, and the bundled bridge theorem composes law semantics, reduction equivalence, and required-work lower bounds (Proposition 2.32, Corollary 2.33, Theorem 2.35)

[C:T4.6,4.7,4.9,5.1,6.1,2.35;P5.3,5.6,5.10,5.7,5.14,5.15,2.27,2.28,2.29,2.30,2.32;C2.33;R:E,S,S+ETH,Qf,Qb,AR] (L: CC61,CC36,CC5,CC16,CC69,CC50,L9,L8,L11,L21,L20,DQ548,DQ537,DQ539,DQ541,CT6,CT8,CT4.)

These results place the problem of identifying decision-relevant coordinates at the first and second levels of the polynomial hierarchy.

Beyond classification, the paper contributes a formal claim-typing framework (Section 3): structural complexity is a property of the fixed decision question, while representational hardness is regime-conditional access cost. This is why encoding-regime changes can move practical hardness without changing the underlying semantics. *[C:P2.20;T3.4;R:AR] (L: CC62,CC76.)*

The reduction constructions and key equivalence theorems are machine-checked in Lean 4 (see Appendix A for proof listings). The formalization verifies that the TAUTOLOGY reduction correctly maps tautologies to sufficient coordinate sets; complexity classifications (coNP-completeness,

Σ_2^{P}-completeness) follow by composition with standard complexity-theoretic results (TAUTOLOGY is coNP-complete, $\exists\forall$-SAT is Σ_2^{P}-complete). The strengthened gadget showing that non-tautologies yield instances with *all coordinates relevant* is also formalized. *[C:T4.6,4.7,4.9,5.1;R:S,S+ETH] (L: CC61,CC36,CC5,CC16,CC24.)*

Complexity Characterization

The results provide precise complexity characterizations within the formal model:

1. **Exact bounds.** SUFFICIENCY-CHECK is coNP-complete, both coNP-hard and in coNP.
2. **Constructive reductions.** The reductions from TAUTOLOGY and $\exists\forall$-SAT are explicit and machine-checked.
3. **Encoding-regime separation.** Under [E], SUFFICIENCY-CHECK is polynomial in $|S|$. Under [S+ETH], there exist succinct worst-case instances (with $k^* = n$) requiring $2^{\Omega(n)}$ time. Under [Q_fin], the Opt-oracle core has $\Omega(|S|)$ worst-case query complexity (Boolean instantiation $\Omega(2^n)$), and under [Q_bool] the value-entry/state-batch refinements preserve the obstruction with weighted-cost transfer closures.

[C:T4.6,5.1;P5.3,5.6,5.10,5.7;R:E,S,S+ETH,Qf,Qb] (L: CC61,CC16,CC50,L9,L8,L11.)

The Complexity Redistribution Corollary

Section 9 develops a quantitative consequence: when a problem requires k dimensions and a model handles only $j < k$ natively, the remaining $k - j$ dimensions must be handled externally at each decision site. For n sites, total external work is $(k - j) \times n$.*[C:T9.5;R:LC] (L: HD21.)*

The set identity is elementary; its operational content comes from composition with the hardness results on exact relevance minimization. This redistribution corollary is formalized in Lean 4 (`HardnessDistribution.lean`), proving:

- Redistribution identity: complexity burden cannot be eliminated by omission, only moved between native handling and external handling
- Dominance: complete models have lower total work than incomplete models
- Amortization: there exists a threshold n^* beyond which higher-dimensional models have lower total cost

[C:T9.3,8.16,9.6;R:LC] (L: HD4,HD1,HD3,HD21.)

Open Questions

The landscape above is complete for the static sufficiency class under C1–C4 and the declared regimes; the items below are adjacent-class extensions or secondary refinements. Several questions remain for future work:

- **Fixed-parameter tractability (primary):** Is SUFFICIENCY-CHECK FPT when parameterized by the minimal sufficient-set size k^*, or is it W[2]-hard under this parameterization?

- **Sequential/stochastic bridge extension:** Characterize the exact frontier where adjacent sequential objectives reduce to the static class via Proposition 2.38, and where genuinely new complexity objects (e.g., horizon/sample/regret complexity) must replace the present coNP/Σ_2^P analysis.
- **Average-case complexity:** What is the complexity under natural distributions on decision problems?
- **Typed-confidence calibration:** For signaled reports $(r, g, p_{\text{self}}, p_{\text{cert}})$, characterize proper scoring and calibration objectives that preserve signal consistency and typed admissibility.
- **Learning cost formalization:** Can central cost H_{central} be formalized as the rank of a concept matroid, making the amortization threshold precisely computable?

Practical Corollaries

The practical corollaries are regime-indexed and theorem-indexed:

- **[E] and structured regimes:** polynomial-time exact procedures exist (Theorem 6.1).
- **[Q_fin]/[Q_bool] query-access lower bounds:** worst-case Opt-oracle complexity is $\Omega(|S|)$ in the finite-state core (Boolean instantiation $\Omega(2^n)$), and value-entry/state-batch interfaces satisfy the same obstruction in the mechanized Boolean refinement (Propositions 5.3–5.15), with randomized weighted robustness and oracle-lattice closures.
- **[S+ETH] hard families:** exact minimization inherits exponential worst-case cost (Theorem 5.1 together with Theorem 4.6).
- **[S_bool] mechanized criterion:** minimization reduces to relevance-cardinality constraints (Corollary 8.6).
- **Redistribution consequences:** omitted native coverage externalizes work with explicit growth/amortization laws (Theorems 9.3–9.6).
- **Physical-system instantiation:** once a physical class is encoded into the declared decision interface, the same regime-typed hardness/tractability map and integrity constraints apply to that class (Section 2.7).

Hence the design choice is typed: enforce a tractable regime, or adopt weakened guarantees with explicit verification boundaries.*[C:T6.1,5.1;R:RG](L: CC71,CC16.)* Equivalently, under the attempted-competence matrix, over-specification is rational only in attempted-competence-failure cells; once exact competence is available in the active regime (or no attempted exact competence was made), persistent over-specification is irrational for the same verified-cost objective.*[C:P3.59;R:AR](L: IC27.)* By Proposition 3.59, the integrity-preserving matrix contains 3 irrational cells and 1 rational cell. *[C:P3.59;R:AR](L: IC27.)*

Epistemic Alignment Implications for Agent Training

Within this paper's formalism, epistemic alignment is a claim-discipline property: outputs are asserted only when certifiable, and otherwise emitted as abstain reports (optionally carrying tentative guesses and self-reflected confidence under zero certified confidence). Formally, this is the integrity/competence split of Section 3, not a full value-alignment claim.*[C:D3.8,3.9;P3.57;R:TR](L: IC18.)*

Truth-Objective Abstention Requirement. If a solver-agent declares a truth-preserving objective for claims about relation $\mathcal{R}$ in regime Γ, uncertified outputs are admissible only as ABSTAIN/UNKNOWN rather than asserted answers. Under the signal discipline, this includes abstain-with-guess/self-confidence reports while keeping certification evidence-gated. *[C:D3.19,3.20;P3.23,3.29;C3.25;R:AR](L: IC46,IC38,IC41.)* Equivalently, training policies that suppress abstention in uncertified regions violate the typed claim discipline under the declared task model. *[C:D3.8;P3.57,3.60,3.59;R:AR](L: IC18,CC30,IC27.)*

This yields a training doctrine for solver-agents under declared tasks/regimes:

1. Treat ABSTAIN/UNKNOWN as first-class admissible outputs, not errors.*[C:D3.8;R:TR]*
2. For signaled reports, maintain two channels: self-reflected confidence may be emitted, but certified confidence is positive only with evidence.*[C:D3.20;P3.23,3.30;R:AR](L: IC38,IC47.)*
3. Treat runtime disclosure as scalar-witnessed: always emit exact step-count, and emit completion percentage only when a declared bound exists.*[C:P3.26,3.27,3.28;R:AR](L: IC42,IC35,IC44.)*
4. Optimize competent coverage subject to integrity constraints, rather than optimizing answer-rate alone.*[C:D3.9;P3.57;R:AR](L: IC18.)*
5. In hardness-blocked regimes, abstention or explicit guarantee-weakening is structurally required unless regime assumptions are changed.*[C:T5.1;P3.60,3.59;R:S,S+ETH](L: CC16,CC30,IC27.)*

Operationally, a reporting schema consistent with this framework tracks at least five quantities: integrity violations, competent non-abstaining coverage on the declared scope, abstention quality under regime shift, the calibration gap between self-reflected and certified confidence channels, and scalar runtime witness distributions (with bound-qualified fraction semantics when bounds are declared). This is consistent with bounded-rationality feasibility discipline under declared resource models [71, 5].

A Lean 4 Proof Listings

The complete Lean 4 formalization is available in the companion artifact (Zenodo DOI listed on the title page). The mechanization consists of 12218 lines across 54 files, with 564 theorem/lemma statements.

Handle IDs. Inline theorem metadata now cites compact IDs (for example, `HD6`, `CC12`, `IC4`) instead of full theorem constants. The full ID-to-handle mapping is listed in Section A.1.

A.1 Lean Handle ID Map

ID	Full Lean handle
AR1	AccessRegime.le
AR2	AccessRegime.answer_space
AR3	AccessRegime.le_refl
AR4	AccessRegime.le_trans
AR5	AccessRegime.query_space
AU1	AdditiveUtility.isRelevant
AU2	AdditiveUtility.relevantSet
AU3	AdditiveUtility.toProblem
BC1	BooleanCircuit.circuitSize

ID	Full Lean handle (continued)
C1	Complexity.BitStr
C2	Complexity.Decides
C3	Complexity.KarpReduces
C4	Complexity.Lang
C5	Complexity.ManyOneReducesPoly
C6	Complexity.P
C7	Complexity.PolyTime
C8	Complexity.bitEnc
C9	Counted.bind
C10	Counted.bind_steps
C11	Counted.pure
C12	Counted.pure_steps
C13	Counted.result
C14	Counted.steps
C15	Counted.steps_eq_fst
C16	Counted.tick
C17	Counted.tick_bind_steps
C31	Clause3.eval
CC1	DecisionQuotient.ClaimClosure.RegimeSimulation
CC2	DecisionQuotient.ClaimClosure.adq_ordering
CC3	DecisionQuotient.ClaimClosure.agent_transfer_licensed_iff_snapshot
CC4	DecisionQuotient.ClaimClosure.anchor_sigma2p_complete_conditional
CC5	DecisionQuotient.ClaimClosure.anchor_sigma2p_reduction_core
CC6	DecisionQuotient.ClaimClosure.boundaryCharacterized_iff_exists_sufficient_subset
CC7	DecisionQuotient.ClaimClosure.bounded_actions_detectable
CC8	DecisionQuotient.ClaimClosure.bridge_boundary_represented_family
CC9	DecisionQuotient.ClaimClosure.bridge_failure_witness_non_one_step
CC10	DecisionQuotient.ClaimClosure.bridge_transfer_iff_one_step_class
CC11	DecisionQuotient.ClaimClosure.certified_total_bits_split_core
CC12	DecisionQuotient.ClaimClosure.cost_asymmetry_eth_conditional
CC13	DecisionQuotient.ClaimClosure.declaredBudgetSlice
CC14	DecisionQuotient.ClaimClosure.declaredRegimeFamily_complete
CC15	DecisionQuotient.ClaimClosure.declared_physics_no_universal_exact_certifier_core
CC16	DecisionQuotient.ClaimClosure.dichotomy_conditional
CC17	DecisionQuotient.ClaimClosure.epsilon_admissible_iff_raw_lt_certified_total_core
CC18	DecisionQuotient.ClaimClosure.exact_admissible_iff_raw_lt_certified_total_core
CC19	DecisionQuotient.ClaimClosure.exact_certainty_inflation_under_hardness_core
CC20	DecisionQuotient.ClaimClosure.exact_raw_eq_certified_iff_certainty_inflation_core
CC21	DecisionQuotient.ClaimClosure.exact_raw_only_of_no_exact_admissible_core
CC22	DecisionQuotient.ClaimClosure.explicit_assumptions_required_of_not_excused_core
CC23	DecisionQuotient.ClaimClosure.explicit_state_upper_core
CC24	DecisionQuotient.ClaimClosure.hard_family_all_coords_core
CC25	DecisionQuotient.ClaimClosure.horizonTwoWitness_immediate_empty_sufficient
CC26	DecisionQuotient.ClaimClosure.horizon_gt_one_bridge_can_fail_on_sufficiency
CC27	DecisionQuotient.ClaimClosure.information_barrier_opt_oracle_core
CC28	DecisionQuotient.ClaimClosure.information_barrier_state_batch_core
CC29	DecisionQuotient.ClaimClosure.information_barrier_value_entry_core
CC30	DecisionQuotient.ClaimClosure.integrity_resource_bound_for_sufficiency
CC31	DecisionQuotient.ClaimClosure.integrity_universal_applicability_core
CC32	DecisionQuotient.ClaimClosure.meta_coordinate_irrelevant_of_invariance_on_declared_slice
CC33	DecisionQuotient.ClaimClosure.meta_coordinate_not_relevant_on_declared_slice
CC34	DecisionQuotient.ClaimClosure.minsuff_collapse_core
CC36	DecisionQuotient.ClaimClosure.minsuff_conp_complete_conditional
CC37	DecisionQuotient.ClaimClosure.no_auto_minimize_of_p_neq_conp
CC38	DecisionQuotient.ClaimClosure.no_exact_claim_admissible_under_hardness_core

ID	Full Lean handle (continued)
CC39	DecisionQuotient.ClaimClosure.no_exact_claim_under_declared_assumptions_unless_excused_core
CC40	DecisionQuotient.ClaimClosure.no_exact_identifier_implies_not_boundary_characterized
CC41	DecisionQuotient.ClaimClosure.no_uncertified_exact_claim_core
CC42	DecisionQuotient.ClaimClosure.one_step_bridge
CC43	DecisionQuotient.ClaimClosure.oracle_lattice_transfer_as_regime_simulation
CC44	DecisionQuotient.ClaimClosure.physical_crossover_above_cap_core
CC45	DecisionQuotient.ClaimClosure.physical_crossover_core
CC46	DecisionQuotient.ClaimClosure.physical_crossover_hardness_core
CC47	DecisionQuotient.ClaimClosure.physical_crossover_policy_core
CC48	DecisionQuotient.ClaimClosure.process_bridge_failure_witness
CC49	DecisionQuotient.ClaimClosure.query_obstruction_boolean_corollary
CC50	DecisionQuotient.ClaimClosure.query_obstruction_finite_state_core
CC51	DecisionQuotient.ClaimClosure.regime_core_claim_proved
CC52	DecisionQuotient.ClaimClosure.regime_simulation_transfers_hardness
CC53	DecisionQuotient.ClaimClosure.reusable_heuristic_of_detectable
CC54	DecisionQuotient.ClaimClosure.selectorSufficient_not_implies_setSufficient
CC55	DecisionQuotient.ClaimClosure.separable_detectable
CC56	DecisionQuotient.ClaimClosure.snapshot_vs_process_typed_boundary
CC57	DecisionQuotient.ClaimClosure.standard_assumption_ledger_unpack
CC58	DecisionQuotient.ClaimClosure.stochastic_objective_bridge_can_fail_on_sufficiency
CC59	DecisionQuotient.ClaimClosure.subproblem_hardness_lifts_to_full
CC60	DecisionQuotient.ClaimClosure.subproblem_transfer_as_regime_simulation
CC61	DecisionQuotient.ClaimClosure.sufficiency_conp_complete_conditional
CC62	DecisionQuotient.ClaimClosure.sufficiency_conp_reduction_core
CC63	DecisionQuotient.ClaimClosure.sufficiency_iff_dq_ratio
CC64	DecisionQuotient.ClaimClosure.sufficiency_iff_projectedOptCover_eq_opt
CC65	DecisionQuotient.ClaimClosure.thermo_conservation_additive_core
CC66	DecisionQuotient.ClaimClosure.thermo_energy_carbon_lift_core
CC67	DecisionQuotient.ClaimClosure.thermo_eventual_lift_core
CC68	DecisionQuotient.ClaimClosure.thermo_hardness_bundle_core
CC69	DecisionQuotient.ClaimClosure.thermo_mandatory_cost_core
CC70	DecisionQuotient.ClaimClosure.tractable_bounded_core
CC71	DecisionQuotient.ClaimClosure.tractable_separable_core
CC72	DecisionQuotient.ClaimClosure.tractable_subcases_conditional
CC73	DecisionQuotient.ClaimClosure.tractable_tree_core
CC74	DecisionQuotient.ClaimClosure.transition_coupled_bridge_can_fail_on_sufficiency
CC75	DecisionQuotient.ClaimClosure.tree_structure_detectable
CC76	DecisionQuotient.ClaimClosure.typed_claim_admissibility_core
CC77	DecisionQuotient.ClaimClosure.typed_static_class_completeness
CDP1	ComputableDecisionProblem.checkSufficiency
CDP2	ComputableDecisionProblem.checkSufficiency_iff_abstract
CDP3	ComputableDecisionProblem.computeOpt
CDP4	ComputableDecisionProblem.computeOpt_eq_abstract
CDP5	ComputableDecisionProblem.decisionEquivDec
CDP6	ComputableDecisionProblem.decisionEquivDec_iff
CDP7	ComputableDecisionProblem.decisionEquivDec_iff_abstract
CDP8	ComputableDecisionProblem.insufficiencyWitnesses
CDP9	ComputableDecisionProblem.isOptimalDec
CDP10	ComputableDecisionProblem.isOptimalDec_iff
CDP11	ComputableDecisionProblem.isOptimalDec_iff_abstract
CDP12	ComputableDecisionProblem.mem_computeOpt_iff
CDP13	ComputableDecisionProblem.toAbstract
CDP14	ComputableDecisionProblem.verifyWitness
CR1	DecisionQuotient.ConfigReduction.config_sufficiency_iff_behavior_preserving

ID	Full Lean handle (continued)
CT1	CT.law_gap_physical_claim_holds
CT2	CT.no_law_gap_counterexample
CT3	CT.no_physical_counterexample_of_core_theorem
CT4	CT.physical_bridge_bundle
CT5	CT.physical_claim_lifts_from_core
CT6	CT.physical_claim_lifts_from_core_conditional
CT7	CT.physical_counterexample_invalidates_core_rule
CT8	CT.physical_counterexample_yields_core_counterexample
DP1	DecisionQuotient.DecisionProblem.minimalSufficient_iff_relevant
DP2	DecisionQuotient.DecisionProblem.relevantSet_is_minimal
DP3	DecisionQuotient.DecisionProblem.sufficient_implies_selectorSufficient
DP4	DecisionQuotient.ClaimClosure.DecisionProblem.epsOpt_zero_eq_opt
DP5	DecisionQuotient.ClaimClosure.DecisionProblem.sufficient_iff_zeroEpsilonSufficient
DP6	DecisionProblem.anchorSufficient
DP7	DecisionProblem.classMonotoneOn
DP8	DecisionProblem.constant_opt_all_sufficient
DP9	DecisionProblem.DecisionEquiv
DP10	DecisionProblem.DecisionQuotientType
DP11	DecisionProblem.Opt
DP12	DecisionProblem.OptQuotient
DP13	DecisionProblem.SelectedAction
DP14	DecisionProblem.decisionEquiv_refl
DP15	DecisionProblem.decisionEquiv_symm
DP16	DecisionProblem.decisionEquiv_trans
DP17	DecisionProblem.decisionSetoid
DP18	DecisionProblem.dominant_all_sufficient
DP19	DecisionProblem.dominant_unique
DP20	DecisionProblem.edgeOnComplement
DP21	DecisionProblem.edgeOnComplement_iff_not_sufficient
DP22	DecisionProblem.emptySet_not_sufficient_iff_exists_opt_difference
DP23	DecisionProblem.emptySet_sufficient_iff_constant
DP24	DecisionProblem.epsOpt
DP25	DecisionProblem.epsOpt_zero_eq_opt
DP26	DecisionProblem.erase_of_not_mem
DP27	DecisionProblem.factors_implies_respects
DP28	DecisionProblem.hasConstantOpt
DP29	DecisionProblem.hasConstantOpt'
DP30	DecisionProblem.hasDominant
DP31	DecisionProblem.irrelevant_iff_not_relevant
DP32	DecisionProblem.isEpsilonSufficient
DP33	DecisionProblem.isIrrelevant
DP34	DecisionProblem.isMinimalSufficient
DP35	DecisionProblem.isMinimalSufficient'
DP36	DecisionProblem.isOptimal
DP37	DecisionProblem.isRelevant
DP38	DecisionProblem.isSelectorSufficient
DP39	DecisionProblem.isSufficient
DP40	DecisionProblem.isSufficientAt
DP41	DecisionProblem.isSufficientOnSet
DP42	DecisionProblem.minimalSufficient_all_relevant'
DP43	DecisionProblem.minimalSufficient_eq_relevant'
DP44	DecisionProblem.minimalSufficient_iff_relevant
DP45	DecisionProblem.monotoneIn
DP46	DecisionProblem.nonnegativelyMonotoneCoord
DP47	DecisionProblem.not_sufficient_iff_exists_counterexample

ID	Full Lean handle (continued)
DP48	DecisionProblem.numOptClasses
DP49	DecisionProblem.numOptClasses_le
DP50	DecisionProblem.numOptClasses_pos
DP51	DecisionProblem.opt_eq_optQuotient_comp
DP52	DecisionProblem.opt_factors_through_quotient
DP53	DecisionProblem.preservesOpt
DP54	DecisionProblem.preservesOptStrong
DP55	DecisionProblem.quotientEntropy
DP56	DecisionProblem.quotientEntropy_nonneg
DP57	DecisionProblem.quotientEntropy_zero_of_constant
DP58	DecisionProblem.quotientMap
DP59	DecisionProblem.quotientMap_preservesOpt
DP60	DecisionProblem.quotientSize
DP61	DecisionProblem.quotient_is_coarsest
DP62	DecisionProblem.quotient_represents_opt_equiv
DP63	DecisionProblem.relevantSet
DP64	DecisionProblem.relevantSet_is_minimal
DP65	DecisionProblem.relevant_necessary
DP66	DecisionProblem.sufficientSets
DP67	DecisionProblem.sufficientSets_principal'
DP68	DecisionProblem.sufficientSets_upward
DP69	DecisionProblem.sufficientSets_upward_closed
DP70	DecisionProblem.sufficient_contains_all_relevant
DP71	DecisionProblem.sufficient_contains_relevant
DP72	DecisionProblem.sufficient_erase_irrelevant'
DP73	DecisionProblem.sufficient_iff_zeroEpsilonSufficient
DP74	DecisionProblem.sufficient_implies_selectorSufficient
DP75	DecisionProblem.sufficient_superset
DP76	DecisionProblem.univ_sufficient_of_injective
DQ3	DecisionQuotient.card_anchoredSlice_eq_pow_sub
DQ5	DecisionQuotient.opt_eq_feasible_of_gap
DQ6	DecisionQuotient.opt_eq_of_allowed_iff
DQ7	DecisionQuotient.opt_singleton_of_logicallyDeterministic
DQ8	DecisionQuotient.universe_sets_objective_schema
DQ9	DecisionQuotient.AdditiveUtility
DQ10	DecisionQuotient.AssignX
DQ11	DecisionQuotient.AssignY
DQ12	DecisionQuotient.Assignment
DQ13	DecisionQuotient.BinaryState
DQ14	DecisionQuotient.BooleanCircuit
DQ15	DecisionQuotient.CircuitDecisionProblem
DQ16	DecisionQuotient.ClaimClosure.agreeOn_refl
DQ17	DecisionQuotient.ClaimClosure.agreeOn_symm
DQ18	DecisionQuotient.ClaimClosure.agreeOn_trans
DQ19	DecisionQuotient.ClaimClosure.irrelevantOn_implies_not_relevantOn
DQ20	DecisionQuotient.ClaimClosure.isIrrelevantOn
DQ21	DecisionQuotient.ClaimClosure.isRelevantOn
DQ22	DecisionQuotient.Clause3
DQ23	DecisionQuotient.ComputableDecisionProblem
DQ24	DecisionQuotient.ConfigReduction.ConfigAction
DQ25	DecisionQuotient.ConfigReduction.behaviorPreserving
DQ26	DecisionQuotient.ConfigReduction.configDecisionProblem
DQ27	DecisionQuotient.ConfigReduction.configUtility
DQ28	DecisionQuotient.ConfigReduction.configUtility_le_one
DQ29	DecisionQuotient.ConfigReduction.no_occurs_iff_of_behaviorPreserving

ID	Full Lean handle (continued)
DQ30	DecisionQuotient.ConfigReduction.none_mem_Opt_iff_no_occurs
DQ31	DecisionQuotient.ConfigReduction.some_mem_Opt_iff_occurs
DQ32	DecisionQuotient.ConfigReduction.some_mem_Opt_of_occurs
DQ33	DecisionQuotient.ConfigReduction.some_not_mem_Opt_of_not_occurs
DQ34	DecisionQuotient.CoordinateSpace
DQ35	DecisionQuotient.Counted
DQ36	DecisionQuotient.DQAction
DQ37	DecisionQuotient.DQInstance
DQ38	DecisionQuotient.DecisionProblem
DQ39	DecisionQuotient.EncodedDecisionProblem
DQ40	DecisionQuotient.ExistsForallFormula
DQ41	DecisionQuotient.ExistsForallFormula.embedVar
DQ42	DecisionQuotient.ExistsForallFormula.eval_padUniversal
DQ43	DecisionQuotient.ExistsForallFormula.padUniversal
DQ44	DecisionQuotient.ExistsForallFormula.restrictY
DQ45	DecisionQuotient.ExistsForallFormula.satisfiable
DQ46	DecisionQuotient.ExistsForallFormula.satisfiable_iff_padUniversal
DQ47	DecisionQuotient.ExistsForallFormula.satisfiedBy
DQ48	DecisionQuotient.ExistsForallFormula.sumAssignment
DQ49	DecisionQuotient.ExistsForallFormula.sumAssignment_embed
DQ50	DecisionQuotient.FiniteDecisionProblem
DQ51	DecisionQuotient.FiniteDecisionProblem.decisionQuotient
DQ52	DecisionQuotient.FiniteDecisionProblem.isSufficient
DQ53	DecisionQuotient.FiniteDecisionProblem.isSufficient_superset
DQ54	DecisionQuotient.FiniteDecisionProblem.mem_optimalActions_iff
DQ55	DecisionQuotient.FiniteDecisionProblem.optimalActions
DQ56	DecisionQuotient.FiniteDecisionProblem.optimalActions_subset_actions
DQ57	DecisionQuotient.FiniteDecisionProblem.toDecisionProblem
DQ58	DecisionQuotient.HardnessDistribution.IsEventuallyConstant
DQ59	DecisionQuotient.HardnessDistribution.SolutionArchitecture
DQ60	DecisionQuotient.HardnessDistribution.SpecificationProblem
DQ61	DecisionQuotient.HardnessDistribution.amortizationThreshold
DQ62	DecisionQuotient.HardnessDistribution.amortization_threshold_native_manual
DQ63	DecisionQuotient.HardnessDistribution.centralization_step_reduces_total
DQ64	DecisionQuotient.HardnessDistribution.centralized_constant
DQ65	DecisionQuotient.HardnessDistribution.centralized_minimal_errors
DQ66	DecisionQuotient.HardnessDistribution.distributed_errors_grow
DQ67	DecisionQuotient.HardnessDistribution.distributed_multiplies
DQ68	DecisionQuotient.HardnessDistribution.dof_conservation
DQ69	DecisionQuotient.HardnessDistribution.errorSites
DQ70	DecisionQuotient.HardnessDistribution.gapCard
DQ71	DecisionQuotient.HardnessDistribution.generalizedTotalDOF
DQ72	DecisionQuotient.HardnessDistribution.generalized_right_dominates_wrong_pointwise
DQ73	DecisionQuotient.HardnessDistribution.hardnessEfficiency
DQ74	DecisionQuotient.HardnessDistribution.less_distributed_less_total
DQ75	DecisionQuotient.HardnessDistribution.leverageRatio
DQ76	DecisionQuotient.HardnessDistribution.linear_lt_exponential_eventually
DQ77	DecisionQuotient.HardnessDistribution.linear_represents_saturating_only_zero_slope
DQ78	DecisionQuotient.HardnessDistribution.manualApproach
DQ79	DecisionQuotient.HardnessDistribution.manual_errors_grow
DQ80	DecisionQuotient.HardnessDistribution.manual_is_wrong
DQ81	DecisionQuotient.HardnessDistribution.nativeTypeSystem
DQ82	DecisionQuotient.HardnessDistribution.native_is_right
DQ83	DecisionQuotient.HardnessDistribution.native_minimal_errors
DQ84	DecisionQuotient.HardnessDistribution.right_fewer_error_sites

ID	Full Lean handle (continued)
DQ85	DecisionQuotient.HardnessDistribution.saturatingSiteCost
DQ86	DecisionQuotient.HardnessDistribution.simplicityTax
DQ87	DecisionQuotient.HardnessDistribution.simplicityTax_conservation
DQ88	DecisionQuotient.HardnessDistribution.totalDOF
DQ89	DecisionQuotient.HardnessDistribution.totalDOF_eventually_constant_of_zero_distributed
DQ90	DecisionQuotient.HardnessDistribution.totalExternalWork
DQ91	DecisionQuotient.HardnessDistribution.totalExternalWork_eq_n_mul_gapCard
DQ92	DecisionQuotient.HardnessDistribution.zero_distributed_of_totalDOF_eventually_constant
DQ93	DecisionQuotient.InP
DQ94	DecisionQuotient.InsufficiencyWitness
DQ95	DecisionQuotient.IntegrityCompetence.CertifyingSolver
DQ96	DecisionQuotient.IntegrityCompetence.ClaimAdmissible
DQ97	DecisionQuotient.IntegrityCompetence.CompetentOn
DQ98	DecisionQuotient.IntegrityCompetence.EpsilonCompetentOn
DQ99	DecisionQuotient.IntegrityCompetence.EpsilonRelation
DQ100	DecisionQuotient.IntegrityCompetence.Regime
DQ101	DecisionQuotient.IntegrityCompetence.SolverIntegrity
DQ102	DecisionQuotient.IntegrityCompetence.admissibleIrrationalCount
DQ103	DecisionQuotient.IntegrityCompetence.admissibleRationalCount
DQ104	DecisionQuotient.IntegrityCompetence.admissible_epsilon_implies_integrity
DQ105	DecisionQuotient.IntegrityCompetence.admissible_exact_implies_integrity
DQ106	DecisionQuotient.IntegrityCompetence.alwaysAbstain
DQ107	DecisionQuotient.IntegrityCompetence.alwaysAbstain_integrity
DQ108	DecisionQuotient.IntegrityCompetence.certifiedTotalBits_eq_raw_plus_overhead
DQ109	DecisionQuotient.IntegrityCompetence.claim_admissible_abstain
DQ110	DecisionQuotient.IntegrityCompetence.claim_admissible_epsilon_iff
DQ111	DecisionQuotient.IntegrityCompetence.claim_admissible_exact_iff
DQ112	DecisionQuotient.IntegrityCompetence.competent_has_coverage
DQ113	DecisionQuotient.IntegrityCompetence.epsilon_admissible_iff_raw_lt_certifiedTotal
DQ114	DecisionQuotient.IntegrityCompetence.exact_admissible_iff_raw_lt_certifiedTotal
DQ115	DecisionQuotient.IntegrityCompetence.exact_raw_eq_certifiedTotal_iff_ exactCertaintyInflation
DQ116	DecisionQuotient.IntegrityCompetence.inducedRelation
DQ117	DecisionQuotient.IntegrityCompetence.no_uncertified_epsilon_claim
DQ118	DecisionQuotient.IntegrityCompetence.no_uncertified_exact_claim
DQ119	DecisionQuotient.IntegrityCompetence.overModelVerdict
DQ120	DecisionQuotient.IntegrityCompetence.overModelVerdict_inadmissible_iff
DQ121	DecisionQuotient.IntegrityCompetence.overModelVerdict_irrational_iff
DQ122	DecisionQuotient.IntegrityCompetence.program_framed_as_solver
DQ123	DecisionQuotient.IntegrityCompetence.report_admissible_implies_raw_lt_certifiedTotal
DQ124	DecisionQuotient.IntegrityCompetence.report_not_admissible_implies_raw_eq_certifiedTotal
DQ125	DecisionQuotient.IntegrityCompetence.report_raw_eq_certifiedTotal_iff_certaintyInflation
DQ126	DecisionQuotient.IntegrityCompetence.rlffReward_abstain
DQ127	DecisionQuotient.IntegrityCompetence.rlffReward_of_evidence
DQ128	DecisionQuotient.IntegrityCompetence.rlffReward_of_no_evidence
DQ129	DecisionQuotient.IntegrityCompetence.solverIntegrity_substrate_parametric
DQ130	DecisionQuotient.IntegrityCompetence.solverOfPartialMap
DQ131	DecisionQuotient.IntegrityCompetence.solverOfPartialMap_integrity
DQ132	DecisionQuotient.IntegrityCompetence.steps_run_scalar_always_defined
DQ133	DecisionQuotient.IntegrityCompetence.steps_run_scalar_falsifiable
DQ134	DecisionQuotient.IntegrityCompetence.zero_epsilon_competence_iff_exact
DQ135	DecisionQuotient.InteriorVerification.CoordScore
DQ136	DecisionQuotient.InteriorVerification.not_sufficientOnSet_of_counterexample
DQ137	DecisionQuotient.InteriorVerification.singleton_coordinate_interior_certificate
DQ138	DecisionQuotient.IsPolynomialTime

ID	Full Lean handle (continued)
DQ139	DecisionQuotient.Literal
DQ140	DecisionQuotient.MinSufficientSetInstance
DQ141	DecisionQuotient.Outside
DQ142	DecisionQuotient.PhysicalBudgetCrossover.EncodingSizeModel
DQ143	DecisionQuotient.PhysicalBudgetCrossover.ExplicitInfeasible
DQ144	DecisionQuotient.PhysicalBudgetCrossover.ExplicitUnbounded
DQ145	DecisionQuotient.PhysicalBudgetCrossover.HasCrossover
DQ146	DecisionQuotient.PhysicalBudgetCrossover.SuccinctBoundedBy
DQ147	DecisionQuotient.PhysicalBudgetCrossover.SuccinctFeasible
DQ148	DecisionQuotient.PhysicalBudgetCrossover.crossover_for_all_budgets_above_cap
DQ149	DecisionQuotient.PhysicalBudgetCrossover.crossover_hardness_bundle
DQ150	DecisionQuotient.PhysicalBudgetCrossover.crossover_integrity_policy
DQ151	DecisionQuotient.PhysicalBudgetCrossover.has_crossover_of_bounded_succinct_unbounded_explicit
DQ152	DecisionQuotient.PhysicalBudgetCrossover.has_crossover_of_witness
DQ153	DecisionQuotient.Physics.Instantiation.Configuration
DQ154	DecisionQuotient.Physics.Instantiation.DecisionCircuit
DQ155	DecisionQuotient.Physics.Instantiation.DecisionInterpretation
DQ156	DecisionQuotient.Physics.Instantiation.EnergyLandscape
DQ157	DecisionQuotient.Physics.Instantiation.LandauerBound
DQ158	DecisionQuotient.Physics.Instantiation.Molecule
DQ159	DecisionQuotient.Physics.Instantiation.MoleculeAsCircuit
DQ160	DecisionQuotient.Physics.Instantiation.MoleculeAsDecisionCircuit
DQ162	DecisionQuotient.Physics.Instantiation.Reaction
DQ163	DecisionQuotient.Physics.Instantiation.k_Boltzmann
DQ164	DecisionQuotient.PolyReduction
DQ165	DecisionQuotient.PolyTimeApprox
DQ166	DecisionQuotient.PolytimeElicitationMechanism
DQ167	DecisionQuotient.Prior
DQ168	DecisionQuotient.ProductSpace
DQ169	DecisionQuotient.QBFState
DQ170	DecisionQuotient.QueryAlgorithm
DQ171	DecisionQuotient.QueryTranscript
DQ172	DecisionQuotient.SAT3Instance
DQ173	DecisionQuotient.SAT3Instance.eval
DQ174	DecisionQuotient.SAT3Instance.eval_eq_true_iff
DQ175	DecisionQuotient.SAT3Instance.satisfiable
DQ176	DecisionQuotient.SAT3Instance.satisfiedBy
DQ177	DecisionQuotient.SeparableUtility
DQ178	DecisionQuotient.SetComparisonInstance
DQ179	DecisionQuotient.SharpSATInstance
DQ180	DecisionQuotient.Sigma2PHardness.GadgetState
DQ181	DecisionQuotient.Sigma2PHardness.I_eq_I_of_encodeX
DQ182	DecisionQuotient.Sigma2PHardness.I_of_encodeX_subset
DQ183	DecisionQuotient.Sigma2PHardness.I_of_x
DQ184	DecisionQuotient.Sigma2PHardness.I_of_x_card
DQ185	DecisionQuotient.Sigma2PHardness.bit
DQ186	DecisionQuotient.Sigma2PHardness.bit_le_one
DQ187	DecisionQuotient.Sigma2PHardness.bit_lt_two
DQ188	DecisionQuotient.Sigma2PHardness.cCoord
DQ189	DecisionQuotient.Sigma2PHardness.cCoord_inj
DQ190	DecisionQuotient.Sigma2PHardness.cCoord_inj_bits
DQ191	DecisionQuotient.Sigma2PHardness.cCoord_injective
DQ192	DecisionQuotient.Sigma2PHardness.cCoord_val
DQ193	DecisionQuotient.Sigma2PHardness.cCoord_val_lt_of_lt

ID	Full Lean handle (continued)
DQ194	DecisionQuotient.Sigma2PHardness.disjoint_I_of_x00_x11
DQ195	DecisionQuotient.Sigma2PHardness.encodeX
DQ196	DecisionQuotient.Sigma2PHardness.encodeX_I_of_x
DQ197	DecisionQuotient.Sigma2PHardness.exactlyIdentifiesRelevant
DQ198	DecisionQuotient.Sigma2PHardness.exactlyIdentifiesRelevant_iff_mem_relevant
DQ199	DecisionQuotient.Sigma2PHardness.exactlyIdentifiesRelevant_implies_representationGap_zero
DQ200	DecisionQuotient.Sigma2PHardness.goodEq
DQ201	DecisionQuotient.Sigma2PHardness.goodEq_x00
DQ202	DecisionQuotient.Sigma2PHardness.goodEq_x11
DQ203	DecisionQuotient.Sigma2PHardness.mem_I_of_x_iff
DQ204	DecisionQuotient.Sigma2PHardness.mem_relevantFinset_iff
DQ205	DecisionQuotient.Sigma2PHardness.not_goodEq_x01
DQ206	DecisionQuotient.Sigma2PHardness.not_sufficient_of_missing_relevantFinset
DQ207	DecisionQuotient.Sigma2PHardness.relevantFinset
DQ208	DecisionQuotient.Sigma2PHardness.representationGap
DQ209	DecisionQuotient.Sigma2PHardness.representationGap_eq_waste_plus_missing
DQ210	DecisionQuotient.Sigma2PHardness.representationGap_missing_eq_gapCard
DQ211	DecisionQuotient.Sigma2PHardness.representationGap_zero_iff_exactlyIdentifiesRelevant
DQ212	DecisionQuotient.Sigma2PHardness.representationGap_zero_iff_minimalSufficient
DQ213	DecisionQuotient.Sigma2PHardness.representationGap_zero_iff_sufficient_and_subset_relevantFinset
DQ214	DecisionQuotient.Sigma2PHardness.representationGap_zero_implies_sufficient
DQ215	DecisionQuotient.Sigma2PHardness.sufficient_iff_relevantFinset_subset
DQ216	DecisionQuotient.Sigma2PHardness.sufficient_iff_relevant_subset
DQ217	DecisionQuotient.Sigma2PHardness.sufficient_of_contains_relevant
DQ218	DecisionQuotient.Sigma2PHardness.two_mul_add_one_lt_two_mul
DQ219	DecisionQuotient.Sigma2PHardness.univ_sufficient_bool
DQ220	DecisionQuotient.Sigma2PHardness.validI
DQ221	DecisionQuotient.Sigma2PHardness.validI_I_of_x
DQ222	DecisionQuotient.Sigma2PHardness.validI_iff_exists_x
DQ223	DecisionQuotient.Sigma2PHardness.vector1_obstruction
DQ224	DecisionQuotient.Sigma2PHardness.x00
DQ225	DecisionQuotient.Sigma2PHardness.x01
DQ226	DecisionQuotient.Sigma2PHardness.x11
DQ227	DecisionQuotient.Sigma2PHardness.yCoord
DQ228	DecisionQuotient.Sigma2PHardness.yCoord_val
DQ229	DecisionQuotient.Signal
DQ230	DecisionQuotient.StateAbstraction
DQ231	DecisionQuotient.StateBatchQuery
DQ232	DecisionQuotient.StructuredUtility
DQ233	DecisionQuotient.SufficiencyCheckInstance
DQ234	DecisionQuotient.SufficiencyInstance
DQ235	DecisionQuotient.Summary.bounded_actions_tractable
DQ236	DecisionQuotient.Summary.complexity_dichotomy
DQ237	DecisionQuotient.Summary.conp_completeness
DQ238	DecisionQuotient.Summary.eth_lower_bound
DQ239	DecisionQuotient.Summary.min_sufficient_inapproximability
DQ240	DecisionQuotient.Summary.parameterized_results
DQ241	DecisionQuotient.Summary.separable_utility_tractable
DQ242	DecisionQuotient.Summary.sharp_p_hardness
DQ243	DecisionQuotient.Summary.tractability_tightness
DQ244	DecisionQuotient.Summary.tree_structure_tractable
DQ245	DecisionQuotient.ThermodynamicLift.ThermoModel
DQ246	DecisionQuotient.ThermodynamicLift.carbonLowerBound
DQ247	DecisionQuotient.ThermodynamicLift.carbon_lower_additive

ID	Full Lean handle (continued)
DQ248	DecisionQuotient.ThermodynamicLift.carbon_lower_from_bits_lower
DQ249	DecisionQuotient.ThermodynamicLift.carbon_lower_mandatory
DQ250	DecisionQuotient.ThermodynamicLift.energyLowerBound
DQ251	DecisionQuotient.ThermodynamicLift.energy_lower_additive
DQ252	DecisionQuotient.ThermodynamicLift.energy_lower_from_bits_lower
DQ253	DecisionQuotient.ThermodynamicLift.energy_lower_mandatory
DQ254	DecisionQuotient.ThermodynamicLift.eventual_thermo_lift
DQ255	DecisionQuotient.ThermodynamicLift.hardness_thermo_bundle_conditional
DQ256	DecisionQuotient.ThermodynamicLift.mandatory_conserved_bundle_conditional
DQ257	DecisionQuotient.ThermodynamicLift.mandatory_cost_bundle
DQ258	DecisionQuotient.TreeStructured
DQ259	DecisionQuotient.UniverseDynamics
DQ260	DecisionQuotient.ValueQueryState
DQ261	DecisionQuotient.actions_card
DQ262	DecisionQuotient.adversarialOpt
DQ263	DecisionQuotient.adversarialOpt_eq_false_of_ne
DQ264	DecisionQuotient.adversarialOpt_eq_true_of_eq
DQ265	DecisionQuotient.agreeOn
DQ266	DecisionQuotient.agreeOn_flipAt
DQ267	DecisionQuotient.agreeOn_iff_subset_agreementSet
DQ268	DecisionQuotient.agreeOn_xCoords_iff
DQ269	DecisionQuotient.agreementSet
DQ270	DecisionQuotient.all_coords_relevant_of_not_tautology
DQ271	DecisionQuotient.anchor_sufficiency_sigma2p
DQ272	DecisionQuotient.anchor_sufficiency_sigma2p_nonempty
DQ273	DecisionQuotient.anchoredSlice
DQ274	DecisionQuotient.anchoredSliceEquivOutside
DQ275	DecisionQuotient.anchoredSlice_mul_fixed_eq_full
DQ276	DecisionQuotient.approxWithin
DQ277	DecisionQuotient.bestExpectedUtility
DQ278	DecisionQuotient.bool_minimalSufficient_unique
DQ279	DecisionQuotient.bool_sufficient_erase_irrelevant
DQ280	DecisionQuotient.bounded_actions_complexity
DQ281	DecisionQuotient.bounded_actions_polynomial_time
DQ282	DecisionQuotient.buildDQProblem
DQ283	DecisionQuotient.card_anchoredSlice
DQ284	DecisionQuotient.card_anchoredSlice_eq_uniform
DQ285	DecisionQuotient.card_binary_state
DQ286	DecisionQuotient.card_function_space
DQ287	DecisionQuotient.card_outside_eq_sub
DQ288	DecisionQuotient.certificate_lower_bound_for_I
DQ289	DecisionQuotient.certificate_lower_bound_for_I_card
DQ290	DecisionQuotient.certificate_lower_bound_for_I_card_summary
DQ291	DecisionQuotient.certificate_lower_bound_for_I_empty
DQ292	DecisionQuotient.certificate_lower_bound_for_I_empty_summary
DQ293	DecisionQuotient.certificate_lower_bound_for_I_summary
DQ294	DecisionQuotient.certificate_lower_bound_poly
DQ295	DecisionQuotient.certificate_lower_bound_poly_ge
DQ296	DecisionQuotient.certificate_lower_bound_poly_ge_summary
DQ297	DecisionQuotient.certificate_lower_bound_poly_summary
DQ298	DecisionQuotient.compatibleStates
DQ299	DecisionQuotient.complexity_summary
DQ300	DecisionQuotient.constTrueFinite_empty_sufficient
DQ301	DecisionQuotient.constTrueProblem
DQ302	DecisionQuotient.constTrueProblemFinite

ID	Full Lean handle (continued)
DQ303	DecisionQuotient.constTrueProblemFinite_opt
DQ304	DecisionQuotient.constTrueProblem_opt
DQ305	DecisionQuotient.constTrue_empty_sufficient
DQ306	DecisionQuotient.const_vs_scaled_opt_view_equal
DQ307	DecisionQuotient.const_vs_scaled_value_entry_diff_at_true
DQ308	DecisionQuotient.constant_opt_no_relevant
DQ309	DecisionQuotient.constant_opt_quotientSize_one
DQ310	DecisionQuotient.countSatisfyingAssignments
DQ311	DecisionQuotient.countedCheckPairs
DQ312	DecisionQuotient.countedCheckPairs_steps_bound
DQ313	DecisionQuotient.countedCompare
DQ314	DecisionQuotient.countedOptEqual
DQ315	DecisionQuotient.covers
DQ316	DecisionQuotient.covers_iff_agreeOn
DQ317	DecisionQuotient.cube_lt_pow
DQ318	DecisionQuotient.cubic_step_bound
DQ319	DecisionQuotient.cyclic_dependencies_coNP_hard
DQ320	DecisionQuotient.decisionQuotient_cases
DQ321	DecisionQuotient.decision_quotient_FPTAS
DQ322	DecisionQuotient.decision_quotient_sharp_P_hard
DQ323	DecisionQuotient.decode_error_sum_two_labels
DQ324	DecisionQuotient.dichotomy_by_relevant_size
DQ325	DecisionQuotient.distinguish_requires_target
DQ326	DecisionQuotient.dqExact
DQ327	DecisionQuotient.dqProjection
DQ328	DecisionQuotient.dq_approximation_hard
DQ329	DecisionQuotient.emptySufficiency_query_indistinguishable_pair
DQ330	DecisionQuotient.emptySufficiency_query_indistinguishable_pair_finite
DQ331	DecisionQuotient.emptySufficiency_stateBatch_indistinguishable_pair
DQ332	DecisionQuotient.emptySufficiency_valueEntry_indistinguishable_pair
DQ333	DecisionQuotient.empty_minimal_sufficient_iff_constant
DQ334	DecisionQuotient.endpoints
DQ335	DecisionQuotient.endpoints_card_le
DQ336	DecisionQuotient.eth_implies_sat3_exponential
DQ337	DecisionQuotient.eth_lower_bound_informal
DQ338	DecisionQuotient.eth_lower_bound_sufficiency_check
DQ339	DecisionQuotient.exactDQ
DQ340	DecisionQuotient.exists_coord_not_mem
DQ341	DecisionQuotient.exists_distinct_patterns
DQ342	DecisionQuotient.exists_forall_iff_anchor_sufficient
DQ343	DecisionQuotient.exists_forall_iff_anchor_sufficient_padded
DQ344	DecisionQuotient.exists_not_mem_of_card_lt_univ
DQ345	DecisionQuotient.exists_state_not_in_endpoints
DQ346	DecisionQuotient.expectedUtility
DQ347	DecisionQuotient.exponential_query_complexity
DQ348	DecisionQuotient.feasibleActions
DQ349	DecisionQuotient.flipAt
DQ350	DecisionQuotient.flipAt_ne
DQ351	DecisionQuotient.fptRunningTime
DQ352	DecisionQuotient.fpt_kernel_bound
DQ353	DecisionQuotient.full_query_distinguishes_const_spike_finite
DQ354	DecisionQuotient.greedy_approximation_ratio
DQ355	DecisionQuotient.hard_when_all_relevant
DQ356	DecisionQuotient.indistinguishable_pair_forces_one_error
DQ357	DecisionQuotient.indistinguishable_pair_forces_one_error_per_seed

ID	Full Lean handle (continued)
DQ358	DecisionQuotient.infeasible_not_optimal_of_gap
DQ359	DecisionQuotient.lawDecisionProblem
DQ360	DecisionQuotient.lawUtility
DQ361	DecisionQuotient.lawUtility_eq_of_allowed_iff
DQ362	DecisionQuotient.logicallyDeterministic
DQ363	DecisionQuotient.mem_compatibleStates_iff
DQ364	DecisionQuotient.mem_optFinset_iff
DQ365	DecisionQuotient.mem_optimalActions_iff_actionValue
DQ366	DecisionQuotient.mem_xCoords
DQ367	DecisionQuotient.min_sufficient_W2_hard
DQ368	DecisionQuotient.min_sufficient_inapproximability_informal
DQ369	DecisionQuotient.min_sufficient_set_coNP_hard
DQ370	DecisionQuotient.min_sufficient_set_inapprox_statement
DQ371	DecisionQuotient.mkState
DQ372	DecisionQuotient.mkState_castAdd
DQ373	DecisionQuotient.mkState_natAdd
DQ374	DecisionQuotient.monotone_opt_at_top
DQ375	DecisionQuotient.mss_equiv_relevant_card
DQ376	DecisionQuotient.no_satisfying_of_count_zero
DQ377	DecisionQuotient.not_tautology_iff_exists_opt_difference
DQ378	DecisionQuotient.numPatterns
DQ379	DecisionQuotient.optComparisonCost
DQ380	DecisionQuotient.optFinset
DQ381	DecisionQuotient.optFinset_subset_projectedOptCover
DQ382	DecisionQuotient.opt_both
DQ383	DecisionQuotient.opt_falsifying
DQ384	DecisionQuotient.opt_falsifying_many
DQ385	DecisionQuotient.opt_no_only
DQ386	DecisionQuotient.opt_reference
DQ387	DecisionQuotient.opt_reference_many
DQ388	DecisionQuotient.opt_satisfying
DQ389	DecisionQuotient.opt_tautology_many
DQ390	DecisionQuotient.opt_yes_only
DQ391	DecisionQuotient.optimalActions_eq_of_separable
DQ392	DecisionQuotient.optimalActions_sat
DQ393	DecisionQuotient.optimalActions_unsat
DQ394	DecisionQuotient.oracleView
DQ395	DecisionQuotient.oracleViewFinite
DQ396	DecisionQuotient.oracleViewFinite_eq_of_agree
DQ397	DecisionQuotient.oracleView_eq_of_agree
DQ398	DecisionQuotient.over_modeling_justified
DQ399	DecisionQuotient.pairCheckCost
DQ400	DecisionQuotient.parameterized_complexity_summary
DQ401	DecisionQuotient.patternClass
DQ402	DecisionQuotient.poly_compose_bound
DQ403	DecisionQuotient.poly_inner_bound
DQ404	DecisionQuotient.poly_reduction_implies_many_one_exists
DQ405	DecisionQuotient.poly_reduction_preserves_P
DQ406	DecisionQuotient.polytime_elicitation_exists_structured
DQ407	DecisionQuotient.proj_binary_state
DQ408	DecisionQuotient.projectToCoords
DQ409	DecisionQuotient.projectedOptCover
DQ410	DecisionQuotient.projectedOptCover_eq_opt_of_sufficient
DQ411	DecisionQuotient.qbfEval
DQ412	DecisionQuotient.qbfEval_mkState

ID	Full Lean handle (continued)
DQ413	DecisionQuotient.qbfProblem
DQ414	DecisionQuotient.qbfUtility
DQ415	DecisionQuotient.queryComplexityLowerBound
DQ416	DecisionQuotient.queryDistinguishes
DQ417	DecisionQuotient.query_lower_bound_statement
DQ418	DecisionQuotient.quotientSize_bool_le_pow
DQ419	DecisionQuotient.quotientSize_le_card
DQ420	DecisionQuotient.quotientSize_le_pow_coords
DQ421	DecisionQuotient.quotientSize_one_all_sufficient
DQ422	DecisionQuotient.quotientSize_one_iff_constant
DQ423	DecisionQuotient.quotientSize_pos
DQ424	DecisionQuotient.recoverCount
DQ425	DecisionQuotient.recoverCount_correct
DQ426	DecisionQuotient.reductionProblem
DQ427	DecisionQuotient.reductionProblemMany
DQ428	DecisionQuotient.reductionUtility
DQ429	DecisionQuotient.reductionUtilityMany
DQ430	DecisionQuotient.reduction_not_separable
DQ431	DecisionQuotient.replace_proj_other
DQ432	DecisionQuotient.replace_proj_self
DQ433	DecisionQuotient.sameProjection
DQ434	DecisionQuotient.sameProjection_trans
DQ435	DecisionQuotient.sat3ToCircuit
DQ436	DecisionQuotient.sat3_reduction_size_preserving
DQ437	DecisionQuotient.satSet
DQ438	DecisionQuotient.satSet_empty_of_count_zero
DQ439	DecisionQuotient.scaledTrueProblem
DQ440	DecisionQuotient.scaledTrueProblem_opt
DQ441	DecisionQuotient.separable_isSufficient
DQ442	DecisionQuotient.sharpSAT_encoded_in_DQ
DQ443	DecisionQuotient.sharpSAT_exactDQ
DQ444	DecisionQuotient.sharpSATtoDQ
DQ445	DecisionQuotient.sharpSATtoDQInstance
DQ446	DecisionQuotient.single_action_always_sufficient
DQ447	DecisionQuotient.someEmbedding
DQ448	DecisionQuotient.spikeFinite_empty_not_sufficient
DQ449	DecisionQuotient.spikeProblem
DQ450	DecisionQuotient.spikeProblemFinite
DQ451	DecisionQuotient.spikeProblemFinite_opt_at
DQ452	DecisionQuotient.spikeProblemFinite_opt_off
DQ453	DecisionQuotient.spikeProblem_opt_at
DQ454	DecisionQuotient.spikeProblem_opt_off
DQ455	DecisionQuotient.spike_empty_not_sufficient
DQ456	DecisionQuotient.stateBatchView
DQ457	DecisionQuotient.stateBatchView_eq_if_hidden_untouched
DQ458	DecisionQuotient.structured_isSufficient
DQ459	DecisionQuotient.succ_cube
DQ460	DecisionQuotient.sufficiencyToSetComparison
DQ461	DecisionQuotient.sufficiency_FPT_coords
DQ462	DecisionQuotient.sufficiency_W1_hard_unbounded_actions
DQ463	DecisionQuotient.sufficiency_check_coNP_hard
DQ464	DecisionQuotient.sufficiency_check_polynomial
DQ465	DecisionQuotient.sufficiency_iff_dq_ratio
DQ466	DecisionQuotient.sufficiency_iff_projectedOptCover_eq_opt
DQ467	DecisionQuotient.sufficiency_in_P

ID	Full Lean handle (continued)
DQ468	DecisionQuotient.sufficiency_poly_bounded_actions
DQ469	DecisionQuotient.sufficiency_poly_separable
DQ470	DecisionQuotient.sufficiency_poly_tree_structured
DQ471	DecisionQuotient.sufficient_implies_tautology
DQ472	DecisionQuotient.sufficient_implies_tautology_many
DQ473	DecisionQuotient.sufficient_means_factorizable
DQ474	DecisionQuotient.sufficient_of_projectedOptCover_eq_opt
DQ475	DecisionQuotient.sufficient_preserves_decisions
DQ476	DecisionQuotient.targetProblem
DQ477	DecisionQuotient.targetProblem_opt_off_target
DQ478	DecisionQuotient.targetProblem_opt_on_target
DQ479	DecisionQuotient.targetProblems_agree_outside
DQ480	DecisionQuotient.tautology_iff_opt_constant
DQ481	DecisionQuotient.tautology_iff_sufficient
DQ482	DecisionQuotient.tautology_iff_sufficient_many
DQ483	DecisionQuotient.tautology_implies_sufficient
DQ484	DecisionQuotient.tautology_implies_sufficient_many
DQ485	DecisionQuotient.totalCheckCost
DQ486	DecisionQuotient.totalCheckCost_le_pow
DQ487	DecisionQuotient.touchedStates
DQ488	DecisionQuotient.touchedStates_card_le_query_card
DQ489	DecisionQuotient.tractability_conditions_tight
DQ490	DecisionQuotient.tractable_small_state_space
DQ491	DecisionQuotient.tractable_when_few_relevant
DQ492	DecisionQuotient.two_actions_coNP_hard
DQ493	DecisionQuotient.two_le_one_false
DQ494	DecisionQuotient.two_le_zero_false
DQ495	DecisionQuotient.two_state_sufficiency
DQ496	DecisionQuotient.unique_matching_pattern
DQ497	DecisionQuotient.valueEntryView
DQ498	DecisionQuotient.valueEntryView_eq_if_hidden_untouched
DQ499	DecisionQuotient.valueEntryView_eq_of_stateBatchView_eq_on_touched
DQ500	DecisionQuotient.valueOfInformation
DQ501	DecisionQuotient.valueOfInformation_const
DQ502	DecisionQuotient.vectorB_blocked
DQ503	DecisionQuotient.voi_computation_sharp_P_hard
DQ504	DecisionQuotient.voi_fptas_smooth_utilities
DQ505	DecisionQuotient.weightedQueryCost
DQ506	DecisionQuotient.weightedQueryCost_ge_min_mul_card
DQ507	DecisionQuotient.weightedQueryCost_ge_min_mul_of_card_lb
DQ508	DecisionQuotient.weighted_seed_error_identity
DQ509	DecisionQuotient.weighted_seed_half_floor
DQ510	DecisionQuotient.xCoords
DQ511	DecisionQuotient.xPart
DQ512	DecisionQuotient.xPart_mkState
DQ513	DecisionQuotient.yAllFalse
DQ514	DecisionQuotient.yAllFalse_mkState
DQ515	DecisionQuotient.yPart
DQ516	DecisionQuotient.yPart_mkState
DQ517	DecisionQuotient.Physics.Instantiation.Circuit
DQ518	DecisionQuotient.Physics.Instantiation.Dynamics
DQ519	DecisionQuotient.Physics.Instantiation.Geometry
DQ520	DecisionQuotient.Physics.Instantiation.MoleculeCircuit
DQ521	DecisionQuotient.Physics.Instantiation.MoleculeDynamics
DQ522	DecisionQuotient.Physics.Instantiation.MoleculeGeometry

ID	Full Lean handle (continued)
DQ523	DecisionQuotient.Physics.Instantiation.geometry_plus_dynamics_is_circuit
DQ524	DecisionQuotient.Physics.Instantiation.law_objective_schema
DQ525	DecisionQuotient.Physics.Instantiation.law_opt_eq_feasible_of_gap
DQ526	DecisionQuotient.Physics.Instantiation.law_opt_singleton_of_deterministic
DQ527	DecisionQuotient.Physics.DecisionTime.DecisionEvent
DQ528	DecisionQuotient.Physics.DecisionTime.DecisionProcess
DQ529	DecisionQuotient.Physics.DecisionTime.SubstrateModel
DQ530	DecisionQuotient.Physics.DecisionTime.TimeUnitStep
DQ531	DecisionQuotient.Physics.DecisionTime.TimedState
DQ532	DecisionQuotient.Physics.DecisionTime.decisionTrace
DQ533	DecisionQuotient.Physics.DecisionTime.decisionTrace_length_eq_ticks
DQ534	DecisionQuotient.Physics.DecisionTime.decision_count_equals_elapsed_time
DQ535	DecisionQuotient.Physics.DecisionTime.decision_event_iff_eq_tick
DQ536	DecisionQuotient.Physics.DecisionTime.decision_event_implies_time_unit
DQ537	DecisionQuotient.Physics.DecisionTime.decision_taking_place_is_unit_of_time
DQ538	DecisionQuotient.Physics.DecisionTime.run
DQ539	DecisionQuotient.Physics.DecisionTime.run_elapsed_time_eq_ticks
DQ540	DecisionQuotient.Physics.DecisionTime.run_time_exact
DQ541	DecisionQuotient.Physics.DecisionTime.substrate_step_is_time_unit
DQ542	DecisionQuotient.Physics.DecisionTime.substrate_step_realizes_decision_event
DQ543	DecisionQuotient.Physics.DecisionTime.tick
DQ544	DecisionQuotient.Physics.DecisionTime.tick_decision_witness
DQ545	DecisionQuotient.Physics.DecisionTime.tick_increments_time
DQ546	DecisionQuotient.Physics.DecisionTime.tick_is_decision_event
DQ547	DecisionQuotient.Physics.DecisionTime.time_coordinate_falsifiable
DQ548	DecisionQuotient.Physics.DecisionTime.time_is_discrete
DQ549	DecisionQuotient.Physics.DecisionTime.time_unit_law_substrate_invariant
DQ555	DecisionQuotient.Physics.PhysicalIncompleteness.PhysicallyInstantiated
DQ556	DecisionQuotient.Physics.PhysicalIncompleteness.UniverseModel
DQ557	DecisionQuotient.Physics.PhysicalIncompleteness.no_surjective_instantiation_of_card_gap
DQ558	DecisionQuotient.Physics.PhysicalIncompleteness.physical_incompleteness_of_bounds
DQ559	DecisionQuotient.Physics.PhysicalIncompleteness.physical_incompleteness_of_card_gap
DQ560	DecisionQuotient.HardnessDistribution.hardnessLowerBound
DQ561	DecisionQuotient.HardnessDistribution.hardness_is_irreducible_required_work
DQ562	DecisionQuotient.HardnessDistribution.requiredWork
DQ563	DecisionQuotient.HardnessDistribution.requiredWork_eq_affine_in_sites
DQ564	DecisionQuotient.HardnessDistribution.workGrowthDegree
DQ565	DecisionQuotient.HardnessDistribution.workGrowthDegree_zero_iff_eventually_constant
DQ566	DecisionQuotient.Physics.ClaimTransport.LawGapInstance
DQ567	DecisionQuotient.Physics.ClaimTransport.PhysicalEncoding
DQ568	DecisionQuotient.Physics.ClaimTransport.lawGapEncoding
DQ569	DecisionQuotient.Physics.ClaimTransport.lawGapPhysicalClaim
DQ570	DecisionQuotient.Physics.ClaimTransport.law_gap_physical_claim_holds
DQ571	DecisionQuotient.Physics.ClaimTransport.no_law_gap_counterexample
DQ572	DecisionQuotient.Physics.ClaimTransport.no_physical_counterexample_of_core_theorem
DQ573	DecisionQuotient.Physics.ClaimTransport.physical_bridge_bundle
DQ574	DecisionQuotient.Physics.ClaimTransport.physical_claim_lifts_from_core
DQ575	DecisionQuotient.Physics.ClaimTransport.physical_claim_lifts_from_core_conditional
DQ576	DecisionQuotient.Physics.ClaimTransport.physical_counterexample_invalidates_core_rule
DQ577	DecisionQuotient.Physics.ClaimTransport.physical_counterexample_yields_core_counterexample
DQ578	DecisionQuotient.Physics.Instantiation.asDecisionCircuit
DQ579	DecisionQuotient.Physics.Instantiation.asDecisionCircuit_preserves_circuit
DQ580	DecisionQuotient.Physics.Instantiation.molecule_decision_preserves_dynamics
DQ581	DecisionQuotient.Physics.Instantiation.molecule_decision_preserves_geometry

ID	Full Lean handle (continued)
F1	Formula.eval
F2	Formula.isTautology
GC1	GoalClass.classMonotoneOn
GC2	GoalClass.isNonNegativelyTautologicalCoord
GC3	GoalClass.isTautologicalCoord
GC4	GoalClass.tautologicalSet
HD1	DecisionQuotient.HardnessDistribution.centralization_dominance_bundle
HD2	DecisionQuotient.HardnessDistribution.centralization_step_saves_n_minus_one
HD3	DecisionQuotient.HardnessDistribution.centralized_higher_leverage
HD4	DecisionQuotient.HardnessDistribution.complete_model_dominates_after_threshold
HD5	DecisionQuotient.HardnessDistribution.gap_conservation_card
HD6	DecisionQuotient.HardnessDistribution.generalizedTotal_with_saturation_eventually_constant
HD7	DecisionQuotient.HardnessDistribution.generalized_dominance_can_fail_without_right_boundedness
HD8	DecisionQuotient.HardnessDistribution.generalized_dominance_can_fail_without_wrong_growth
HD9	DecisionQuotient.HardnessDistribution.generalized_right_dominates_wrong_of_bounded_vs_identity_lower
HD10	DecisionQuotient.HardnessDistribution.generalized_right_eventually_dominates_wrong
HD11	DecisionQuotient.HardnessDistribution.hardnessEfficiency_eq_central_share
HD12	DecisionQuotient.HardnessDistribution.isRightHardness
HD13	DecisionQuotient.HardnessDistribution.isWrongHardness
HD14	DecisionQuotient.HardnessDistribution.linear_lt_exponential_plus_constant_eventually
HD15	DecisionQuotient.HardnessDistribution.native_dominates_manual
HD16	DecisionQuotient.HardnessDistribution.no_positive_slope_linear_represents_saturating
HD17	DecisionQuotient.HardnessDistribution.right_dominates_wrong
HD18	DecisionQuotient.HardnessDistribution.saturatingSiteCost_eventually_constant
HD19	DecisionQuotient.HardnessDistribution.simplicityTax_grows
HD20	DecisionQuotient.HardnessDistribution.totalDOF_eventually_constant_iff_zero_distributed
HD21	DecisionQuotient.HardnessDistribution.totalDOF_ge_intrinsic
IC1	DecisionQuotient.IntegrityCompetence.CertaintyInflation
IC2	DecisionQuotient.IntegrityCompetence.CompletionFractionDefined
IC3	DecisionQuotient.IntegrityCompetence.EvidenceForReport
IC4	DecisionQuotient.IntegrityCompetence.ExactCertaintyInflation
IC5	DecisionQuotient.IntegrityCompetence.Percent
IC6	DecisionQuotient.IntegrityCompetence.RLFFWeights
IC7	DecisionQuotient.IntegrityCompetence.ReportSignal
IC8	DecisionQuotient.IntegrityCompetence.ReportBitModel
IC9	DecisionQuotient.IntegrityCompetence.SignalConsistent
IC10	DecisionQuotient.IntegrityCompetence.admissible_irrational_strictly_more_than_rational
IC11	DecisionQuotient.IntegrityCompetence.admissible_matrix_counts
IC12	DecisionQuotient.IntegrityCompetence.abstain_signal_exists_with_guess_self
IC13	DecisionQuotient.IntegrityCompetence.certaintyInflation_iff_not_admissible
IC14	DecisionQuotient.IntegrityCompetence.certificationOverheadBits
IC15	DecisionQuotient.IntegrityCompetence.certificationOverheadBits_of_evidence
IC16	DecisionQuotient.IntegrityCompetence.certificationOverheadBits_of_no_evidence
IC17	DecisionQuotient.IntegrityCompetence.certifiedTotalBits
IC18	DecisionQuotient.IntegrityCompetence.certifiedTotalBits_ge_raw
IC19	DecisionQuotient.IntegrityCompetence.certifiedTotalBits_gt_raw_of_evidence
IC20	DecisionQuotient.IntegrityCompetence.certifiedTotalBits_of_evidence
IC21	DecisionQuotient.IntegrityCompetence.certifiedTotalBits_of_no_evidence
IC22	DecisionQuotient.IntegrityCompetence.claim_admissible_of_evidence
IC23	DecisionQuotient.IntegrityCompetence.competence_implies_integrity
IC24	DecisionQuotient.IntegrityCompetence.completion_fraction_defined_of_declared_bound
IC25	DecisionQuotient.IntegrityCompetence.epsilon_competence_implies_integrity

ID	Full Lean handle (continued)
IC26	DecisionQuotient.IntegrityCompetence.evidence_nonempty_iff_claim_admissible
IC27	DecisionQuotient.IntegrityCompetence.evidence_of_claim_admissible
IC28	DecisionQuotient.IntegrityCompetence.exact_claim_admissible_iff_exact_evidence_nonempty
IC29	DecisionQuotient.IntegrityCompetence.exact_claim_requires_evidence
IC30	DecisionQuotient.IntegrityCompetence.exactCertaintyInflation_iff_no_exact_competence
IC31	DecisionQuotient.IntegrityCompetence.exact_raw_only_of_no_exact_admissible
IC32	DecisionQuotient.IntegrityCompetence.integrity_forces_abstention
IC33	DecisionQuotient.IntegrityCompetence.integrity_not_competent_of_nonempty_scope
IC34	DecisionQuotient.IntegrityCompetence.integrity_resource_bound
IC35	DecisionQuotient.IntegrityCompetence.no_completion_fraction_without_declared_bound
IC36	DecisionQuotient.IntegrityCompetence.overModelVerdict_rational_iff
IC37	DecisionQuotient.IntegrityCompetence.percentZero
IC38	DecisionQuotient.IntegrityCompetence.rlffBaseReward
IC39	DecisionQuotient.IntegrityCompetence.rlffReward
IC40	DecisionQuotient.IntegrityCompetence.rlff_abstain_strictly_prefers_no_certificates
IC41	DecisionQuotient.IntegrityCompetence.rlff_maximizer_has_evidence
IC42	DecisionQuotient.IntegrityCompetence.rlff_maximizer_is_admissible
IC43	DecisionQuotient.IntegrityCompetence.self_reflected_confidence_not_certification
IC44	DecisionQuotient.IntegrityCompetence.signal_certified_positive_implies_admissible
IC45	DecisionQuotient.IntegrityCompetence.signal_consistent_of_claim_admissible
IC46	DecisionQuotient.IntegrityCompetence.signal_no_evidence_forces_zero_certified
IC47	DecisionQuotient.IntegrityCompetence.signal_exact_no_competence_forces_zero_certified
IPT1	IsPolynomialTime.const
IPT2	IsPolynomialTime.of_steps_le
IV1	DecisionQuotient.InteriorVerification.GoalClass
IV2	DecisionQuotient.InteriorVerification.InteriorDominanceVerifiable
IV3	DecisionQuotient.InteriorVerification.TautologicalSetIdentifiable
IV4	DecisionQuotient.InteriorVerification.agreeOnSet
IV5	DecisionQuotient.InteriorVerification.interiorParetoDominates
IV6	DecisionQuotient.InteriorVerification.interior_certificate_implies_non_rejection
IV7	DecisionQuotient.InteriorVerification.interior_dominance_implies_universal_non_rejection
IV8	DecisionQuotient.InteriorVerification.interior_dominance_not_full_sufficiency
IV9	DecisionQuotient.InteriorVerification.interior_verification_tractable_certificate
IW1	InsufficiencyWitness.certifies_not_sufficient
IW2	InsufficiencyWitness.not_abstract_sufficient
L1	ExactPhysicalClaimWellTyped
L2	ExcusedBy
L3	ExplicitHardnessAssumptions
L4	Literal.eval
L5	OneStepSequentialObjective
L6	RegimeSimulation
L7	StandardComplexityAssumptions
L8	StochasticCriterionObjective
L9	StructureDetectable
L10	TransitionCoupledObjective
L11	TwoStepObjective
L12	adq
L13	adq_ordering
L14	agentBridgeClass
L15	agent_transfer_licensed_iff_snapshot
L16	anchor_sigma2p_complete_conditional
L17	anchor_sigma2p_reduction_core
L19	boundaryCharacterized_iff_exists_sufficient_subset
L20	bounded_actions_detectable
L21	bounded_actions_reusable_heuristic

ID	Full Lean handle (continued)
L22	bridgeFailureWitness
L23	bridgeTransferLicensed
L24	bridge_boundary_represented_family
L25	boolHypercube_node_count
L26	boundaryCharacterized
L27	bridge_failure_witness_non_one_step
L28	bridge_transfer_iff_one_step_class
L29	certified_total_bits_split_core
L30	constantBoolDP
L31	constantBoolDP_empty_sufficient
L32	constantBoolDP_opt
L33	cost_asymmetry_eth_conditional
L34	declaredRegimeFamily
L35	declaredRegimeFamily_complete
L36	declared_physics_no_universal_exact_certifier_core
L37	dichotomy_conditional
L38	epsilon_admissible_iff_raw_lt_certified_total_core
L39	exact_admissible_iff_raw_lt_certified_total_core
L40	exact_certainty_inflation_under_hardness_core
L41	exact_raw_eq_certified_iff_certainty_inflation_core
L42	exact_raw_only_of_no_exact_admissible_core
L43	explicit_assumptions_required_of_not_excused_core
L44	explicit_state_upper_core
L45	firstCoordDP
L46	firstCoordDP_empty_not_sufficient
L47	firstCoordDP_opt
L48	hard_family_all_coords_core
L49	horizonTwoWitness
L50	horizonTwoWitness_immediate_empty_sufficient
L51	horizonTwoWitness_not_empty_sufficient_two_step
L52	horizon_gt_one_bridge_can_fail_on_sufficiency
L53	i0
L54	i0_bridge
L55	information_barrier_opt_oracle_core
L56	information_barrier_state_batch_core
L57	information_barrier_value_entry_core
L58	integrity_resource_bound_for_sufficiency
L59	integrity_universal_applicability_core
L60	minsuff_collapse_core
L61	minsuff_collapse_to_conp_conditional
L62	minsuff_conp_complete_conditional
L63	no_auto_minimize_of_p_neq_conp
L64	no_exact_claim_admissible_under_hardness_core
L65	no_exact_claim_under_declared_assumptions_unless_excused_core
L66	no_exact_identifier_implies_not_boundary_characterized
L67	no_uncertified_exact_claim_core
L68	node_count_does_not_determine_edge_geometry
L69	one_step_bridge
L70	oracle_lattice_transfer_as_regime_simulation
L71	physical_crossover_above_cap_core
L72	physical_crossover_core
L73	physical_crossover_hardness_core
L74	physical_crossover_policy_core
L75	preferTrueSelector
L76	process_bridge_failure_witness

ID	Full Lean handle (continued)
L77	query_obstruction_boolean_corollary
L78	query_obstruction_finite_state_core
L79	regimeCoreClaim
L80	regime_core_claim_proved
L81	regime_simulation_transfers_hardness
L82	reusable_heuristic_of_detectable
L83	s0_bridge
L84	s1_bridge
L85	sFalse
L86	sTrue
L87	selectorGapProblem
L88	selectorGap_not_set_sufficient_empty
L89	selectorGap_selectorSufficient_empty
L90	selectorGap_true_mem_opt
L91	selectorSufficient_not_implies_setSufficient
L92	separable_detectable
L93	separable_reusable_heuristic
L94	snapshot_vs_process_typed_boundary
L95	standard_assumption_ledger_unpack
L96	stochasticWitness
L97	stochastic_objective_bridge_can_fail_on_sufficiency
L98	structureDetectable_of_decidable
L99	subproblem_hardness_lifts_to_full
L100	subproblem_transfer_as_regime_simulation
L101	sufficiency_conp_complete_conditional
L102	sufficiency_conp_reduction_core
L103	thermo_conservation_additive_core
L104	thermo_energy_carbon_lift_core
L105	thermo_eventual_lift_core
L106	thermo_hardness_bundle_core
L107	thermo_mandatory_cost_core
L108	tractable_bounded_core
L109	tractable_separable_core
L110	tractable_subcases_conditional
L111	tractable_tree_core
L112	transitionWitness
L113	transition_coupled_bridge_can_fail_on_sufficiency
L114	tree_reusable_heuristic
L115	tree_structure_detectable
L116	typed_claim_admissibility_core
L117	typed_static_class_completeness
L118	universal_solver_framing_core
OSSO1	OneStepSequentialObjective.Opt
OSSO2	OneStepSequentialObjective.isSufficient
OSSO3	OneStepSequentialObjective.toDecisionProblem
PBC1	DecisionQuotient.PhysicalBudgetCrossover.CrossoverAt
PBC2	DecisionQuotient.PhysicalBudgetCrossover.explicit_infeasible_succinct_feasible_of_crossover
PC1	PhysicalComplexity.ComputationalRequirement
PC2	PhysicalComplexity.InstanceSize
PC3	PhysicalComplexity.Landauer_bound
PC4	PhysicalComplexity.PhysicalComputer
PC5	PhysicalComplexity.bit_energy_cost
PC6	PhysicalComplexity.coNP_physically_impossible
PC7	PhysicalComplexity.coNP_requirement
PC8	PhysicalComplexity.k_Boltzmann

ID	Full Lean handle (continued)
PC9	PhysicalComplexity.sufficiency_physically_impossible
PC10	PhysicalComplexity.AccessRegime.AccessRegime
PC11	PhysicalComplexity.AccessRegime.PhysicalDevice
PC12	PhysicalComplexity.AccessRegime.RegimeEval
PC13	PhysicalComplexity.AccessRegime.RegimeProof
PC14	PhysicalComplexity.AccessRegime.RegimeSample
PC15	PhysicalComplexity.AccessRegime.RegimeWithCertificate
PC16	PhysicalComplexity.AccessRegime.certificate_amortizes_hardness
PC17	PhysicalComplexity.AccessRegime.certificate_upgrades_regime
PC18	PhysicalComplexity.AccessRegime.physical_succinct_certification_hard
PC19	PhysicalComplexity.AccessRegime.regime_upgrade_with_certificate
PC20	PhysicalComplexity.AccessRegime.AccessChannelLaw
PC21	PhysicalComplexity.AccessRegime.FiveWayMeet
PC22	PhysicalComplexity.AccessRegime.AuditableWithCertificate
PC23	PhysicalComplexity.AccessRegime.HardUnderEval
PC24	PhysicalComplexity.coNP_not_in_P_physically
PC25	PhysicalComplexity.pow2_eventually_exceeds
PC26	PhysicalComplexity.AccessRegime.RegimeEvalOn
PC27	PhysicalComplexity.AccessRegime.RegimeProofOn
PC28	PhysicalComplexity.AccessRegime.RegimeSampleOn
PC29	PhysicalComplexity.AccessRegime.RegimeWithCertificateOn
PC30	PhysicalComplexity.AccessRegime.certificate_upgrades_regime_on
PC31	PhysicalComplexity.AccessRegime.regime_upgrade_with_certificate_on
PD2	PhysicalDevice.is_succinct
PD3	PhysicalDevice.derived_explicit_size
PD4	PhysicalDevice.is_succinct_of_gap
PR1	PolyReduction.comp_exists
PR2	PolyReduction.id
QA1	QueryAlgorithm.run
QF1	QFormula.eval
QF2	QFormula.eval_map
QF3	QFormula.map
QT1	QueryTranscript.size
S1	Signal.const
S2	Signal.priorOn
S2P1	DecisionQuotient.Sigma2PHardness.exactlyIdentifiesRelevant_iff_sufficient_and_subset_relevantFinset
S2P2	DecisionQuotient.Sigma2PHardness.min_representationGap_zero_iff_relevant_card
S2P3	DecisionQuotient.Sigma2PHardness.min_sufficient_set_iff_relevant_card
S2P6	DecisionQuotient.Sigma2PHardness.representationGap_eq_zero_iff
SAT3I1	SAT3Instance.inputSize
SCO1	StochasticCriterionObjective.OptChance
SCO2	StochasticCriterionObjective.toExpectedDecisionProblem
TCO1	TransitionCoupledObjective.Opt
TCO2	TransitionCoupledObjective.score
TCO3	TransitionCoupledObjective.toImmediateDecisionProblem
TSO1	TwoStepObjective.Opt
TSO2	TwoStepObjective.score
TSO3	TwoStepObjective.toImmediateDecisionProblem

A.2 What Is Machine-Checked

The Lean formalization establishes:

1. **Correctness of the TAUTOLOGY reduction:** The theorem `tautology_iff_sufficient` proves that the mapping from Boolean formulas to decision problems preserves the decision structure (accept iff the formula is a tautology).

2. **Decision problem definitions:** Formal definitions of sufficiency, optimality, and the decision quotient.

3. **Economic theorems:** Simplicity Tax redistribution identities and hardness distribution results.

4. **Query-access lower-bound core:** Formalized Boolean query-model indistinguishability theorem for the full problem via the $I = \emptyset$ subproblem (`emptySufficiency_query_indistinguishable_pair`), plus obstruction-scale identities (`queryComplexityLowerBound`, `exponential_query_complexity`).

Complexity classifications (coNP-completeness, Σ_2^P-completeness) follow by conditional composition with standard results (e.g., TAUTOLOGY coNP-completeness and $\exists\forall$-SAT Σ_2^P-completeness), represented explicitly as hypotheses in the conditional transfer theorems listed below. The Lean proofs verify the reduction constructions and the transfer closures under those hypotheses. The assumptions themselves are unpacked by an explicit ledger projection theorem (CC5) so dependency tracking is machine-auditable.

A.3 Assumption Ledger (Auto)

Source files.

- `DecisionQuotient/ClaimClosure.lean`

Assumption bundles and fields.

- (No assumption bundles parsed.)

Conditional closure handles.

- `DQ.anchor_sigma2p_complete_conditional`
- `DQ.cost_asymmetry_eth_conditional`
- `DQ.dichotomy_conditional`
- `DQ.minsuff_collapse_to_conp_conditional`
- `DQ.minsuff_conp_complete_conditional`
- `DQ.sufficiency_conp_complete_conditional`
- `DQ.tractable_subcases_conditional`

A.4 Polynomial-Time Reduction Definition

We use Mathlib's Turing machine framework to define polynomial-time computability:

```
/-- Polynomial-time computable function using Turing machines -/
def PolyTime {α β : Type} (ea : FinEncoding α) (eb : FinEncoding β)
    (f : α → β) : Prop :=
  Nonempty (Turing.TM2ComputableInPolyTime ea eb f)

/-- Polynomial-time many-one (Karp) reduction -/
def ManyOneReducesPoly {α β : Type} (ea : FinEncoding α) (eb : FinEncoding β)
    (A : Set α) (B : Set β) : Prop :=
  ∃ f : α → β, PolyTime ea eb f ∧ ∀ x, x ∈ A ↔ f x ∈ B
```

This uses the standard definition: a reduction is polynomial-time computable via Turing machines and preserves membership.

A.5 The Main Reduction Theorem

Theorem A.1 (TAUTOLOGY Reduction Correctness, Lean)**.** *The reduction from TAUTOLOGY to SUFFICIENCY-CHECK is correct:*

```
theorem tautology_iff_sufficient (φ : Formula n) :
    φ.isTautology ↔ (reductionProblem φ).isSufficient Finset.empty
```

This theorem is proven by showing both directions:

- If φ is a tautology, then the empty coordinate set is sufficient
- If the empty coordinate set is sufficient, then φ is a tautology

The proof verifies that the utility construction in `reductionProblem` creates the appropriate decision structure where:

- At reference states, `accept` is optimal with utility 1
- At assignment states, `accept` is optimal iff $\varphi(a) = \text{true}$

A.6 Economic Results

The hardness distribution theorems (Section 9) are fully formalized:

```
theorem simplicityTax_conservation (P : SpecificationProblem)
    (S : SolutionArchitecture P) :
    S.centralDOF + simplicityTax P S ≥ P.intrinsicDOF

theorem simplicityTax_grows (P : SpecificationProblem)
    (S : SolutionArchitecture P) (n₁ n₂ : ℕ)
    (hn : n₁ < n₂) (htax : simplicityTax P S > 0) :
    totalDOF S n₁ < totalDOF S n₂

theorem native_dominates_manual (P : SpecificationProblem) (n : Nat)
```

```
    (hn : n > P.intrinsicDOF) :
    totalDOF (nativeTypeSystem P) n < totalDOF (manualApproach P) n

theorem totalDOF_eventually_constant_iff_zero_distributed
    (S : SolutionArchitecture P) :
    IsEventuallyConstant (fun n => totalDOF S n) ↔ S.distributedDOF = 0

theorem no_positive_slope_linear_represents_saturating
    (c d K : ℕ) (hd : d > 0) :
    ¬ (∀ n, c + n * d = generalizedTotalDOF c (saturatingSiteCost K) n)
```

Identifier note. Lean identifiers retain internal naming (`intrinsicDOF`, `simplicityTax_conservation`); in paper terminology these correspond to *baseline hardness* and the *redistribution lower-bound identity*, respectively.

A.7 Complete Claim Coverage Matrix

Paper handle	Status	Lean support
cor:exact-identifiability	Full	DQ.Sigma2PHardness.exactlyIdentifiesRelevant_iff_sufficient_and_subset_relevantFinset
cor:exact-no-competence-zero-certified	Full	DQ.IntegrityCompetence.rlff_maximizer_has_evidence
cor:gap-externalization	Full	DQ.HardnessDistribution.saturatingSiteCost_eventually_constant, DQ.HardnessDistribution.totalDOF_ge_intrinsic
cor:gap-minimization-hard	Full	DQ.Sigma2PHardness.min_representationGap_zero_iff_relevant_card
cor:generalized-eventual-dominance	Full	DQ.HardnessDistribution.generalized_right_dominates_wrong_of_bounded_vs_identity_lower
cor:hardness-exact-certainty-inflation	Full	DQ.ClaimClosure.exact_certainty_inflation_under_hardness_core
cor:hardness-raw-only-exact	Full	DQ.ClaimClosure.exact_raw_only_of_no_exact_admissible_core, DQ.IntegrityCompetence.competence_implies_integrity
cor:information-barrier-query	Full	DQ.ClaimClosure.information_barrier_opt_oracle_core, DQ.ClaimClosure.information_barrier_state_batch_core, DQ.ClaimClosure.information_barrier_value_entry_core
cor:integrity-universal	Full	DQ.ClaimClosure.integrity_universal_applicability_core
cor:interior-singleton-certificate	Full	DQ.InteriorVerification.interior_verification_tractable_certificate
cor:linear-positive-no-saturation	Full	DQ.HardnessDistribution.native_dominates_manual
cor:no-auto-minimize	Full	DQ.ClaimClosure.no_auto_minimize_of_p_neq_conp
cor:no-uncertified-exact-claim	Full	DQ.ClaimClosure.no_uncertified_exact_claim_core
cor:outside-excuses-no-exact-report	Full	DQ.ClaimClosure.no_exact_claim_under_declared_assumptions_unless_excused_core
cor:overmodel-diagnostic-implication	Full	DQ.Sigma2PHardness.representationGap_eq_zero_iff
cor:physical-counterexample-core-failure	Full	CT.no_physical_counterexample_of_core_theorem, CT.physical_counterexample_invalidates_core_rule, CT.physical_counterexample_yields_core_counterexample
cor:physics-no-universal-exact-claim	Full	DQ.ClaimClosure.no_exact_claim_admissible_under_hardness_core
cor:practice-bounded	Full	agent_transfer_licensed_iff_snapshot
cor:practice-diagnostic	Full	DQ.Sigma2PHardness.min_representationGap_zero_iff_relevant_card
cor:practice-structured	Full	anchor_sigma2p_complete_conditional
cor:practice-tree	Full	anchor_sigma2p_reduction_core
cor:practice-unstructured	Full	ExactPhysicalClaimWellTyped
cor:query-obstruction-bool	Full	DQ.ClaimClosure.query_obstruction_boolean_corollary, RegimeSimulation
cor:right-wrong-hardness	Full	DQ.HardnessDistribution.no_positive_slope_linear_represents_saturating

cor:rlff-abstain-no-certs	Full	DQ.IntegrityCompetence.exactCertaintyInflation_iff_no_exact_competence
cor:type-system-threshold	Full	DQ.HardnessDistribution.linear_lt_exponential_plus_constant_eventually
prop:abstain-guess-self-signal	Full	DQ.IntegrityCompetence.signal_no_evidence_forces_zero_certified
prop:adq-ordering	Full	DQ.ClaimClosure.adq_ordering
prop:attempted-competence-matrix	Full	DQ.IntegrityCompetence.RLFFWeights, DQ.IntegrityCompetence.ReportSignal, DQ.IntegrityCompetence.evidence_of_claim_admissible
prop:bounded-slice-meta-irrelevance	Full	DQ.ClaimClosure.meta_coordinate_irrelevant_of_invariance_on_declared_slice, DQ.ClaimClosure.meta_coordinate_not_relevant_on_declared_slice
prop:bridge-failure-horizon	Full	DQ.ClaimClosure.horizonTwoWitness_immediate_empty_sufficient, DQ.ClaimClosure.horizon_gt_one_bridge_can_fail_on_sufficiency
prop:bridge-failure-stochastic	Full	DQ.ClaimClosure.standard_assumption_ledger_unpack
prop:bridge-failure-transition	Full	DQ.ClaimClosure.tractable_tree_core
prop:bridge-transfer-scope	Full	DQ.ClaimClosure.one_step_bridge
prop:budgeted-crossover	Full	DQ.ClaimClosure.physical_crossover_core, DQ.PhysicalBudgetCrossover.CrossoverAt
prop:certainty-inflation-iff-inadmissible	Full	DQ.IntegrityCompetence.ReportBitModel, DQ.IntegrityCompetence.claim_admissible_of_evidence
prop:certified-confidence-gate	Full	DQ.IntegrityCompetence.rlffBaseReward, DQ.IntegrityCompetence.rlffReward
prop:crossover-above-cap	Full	DQ.ClaimClosure.physical_crossover_above_cap_core, DQ.PhysicalBudgetCrossover.explicit_infeasible_succinct_feasible_of_crossover
prop:crossover-not-certification	Full	DQ.ClaimClosure.physical_crossover_hardness_core
prop:crossover-policy	Full	DQ.ClaimClosure.physical_crossover_policy_core
prop:decision-unit-time	Full	DQ.Physics.DecisionTime.decision_event_iff_eq_tick, DQ.Physics.DecisionTime.decision_taking_place_is_unit_of_time, DQ.Physics.DecisionTime.tick_increments_time, DQ.Physics.DecisionTime.tick_is_decision_event
prop:declared-contract-selection-validity	Full	DQ.ClaimClosure.explicit_assumptions_required_of_not_excused_core, DQ.ClaimClosure.no_exact_claim_under_declared_assumptions_unless_excused_core, DQ.ClaimClosure.tree_structure_detectable
prop:dominance-modes	Full	DQ.HardnessDistribution.centralized_higher_leverage
prop:empty-sufficient-constant	Full	DP.anchorSufficient
prop:evidence-admissibility-equivalence	Full	DQ.IntegrityCompetence.certifiedTotalBits, DQ.IntegrityCompetence.certifiedTotalBits_of_evidence, DQ.IntegrityCompetence.certifiedTotalBits_of_no_evidence
prop:exact-requires-evidence	Full	DQ.IntegrityCompetence.no_completion_fraction_without_declared_bound, DQ.IntegrityCompetence.overModelVerdict_rational_iff
prop:fraction-defined-under-bound	Full	DQ.IntegrityCompetence.signal_consistent_of_claim_admissible
prop:generalized-assumption-boundary	Full	DQ.HardnessDistribution.generalizedTotal_with_saturation_eventually_constant, DQ.HardnessDistribution.generalized_dominance_can_fail_without_right_boundedness
prop:hardness-conservation	Full	DQ.HardnessDistribution.totalDOF_eventually_constant_iff_zero_distributed
prop:hardness-efficiency-interpretation	Full	DQ.HardnessDistribution.generalized_right_eventually_dominates_wrong
prop:heuristic-reusability	Full	DQ.ClaimClosure.bounded_actions_detectable, DQ.ClaimClosure.reusable_heuristic_of_detectable, DQ.ClaimClosure.separable_detectable, DQ.ClaimClosure.transition_coupled_bridge_can_fail_on_sufficiency
prop:insufficiency-counterexample	Full	DP.classMonotoneOn, DP.constant_opt_all_sufficient
prop:integrity-competence-separation	Full	DQ.IntegrityCompetence.certifiedTotalBits_ge_raw, DQ.IntegrityCompetence.epsilon_competence_implies_integrity
prop:integrity-resource-bound	Full	DQ.ClaimClosure.integrity_resource_bound_for_sufficiency, DQ.IntegrityCompetence.completion_fraction_defined_of_declared_bound, DQ.IntegrityCompetence.evidence_nonempty_iff_claim_admissible

prop:interior-one-sidedness	Full	DQ.InteriorVerification.interior_certificate_implies_non_rejection, DQ.InteriorVerification.interior_dominance_not_full_sufficiency
prop:interior-universal-non-rejection	Full	DQ.InteriorVerification.interiorParetoDominates
prop:interior-verification-tractable	Full	DQ.InteriorVerification.agreeOnSet, DQ.InteriorVerification.interior_dominance_implies_universal_non_rejection
prop:law-instance-objective-bridge	Full	CT.law_gap_physical_claim_holds, CT.no_law_gap_counterexample
prop:minimal-relevant-equiv	Full	DQ.DecisionProblem.relevantSet_is_minimal, DQ.DecisionProblem.sufficient_implies_selectorSufficient
prop:no-evidence-zero-certified	Full	DQ.IntegrityCompetence.rlff_abstain_strictly_prefers_no_certificates
prop:no-fraction-without-bound	Full	DQ.IntegrityCompetence.signal_certified_positive_implies_admissible
prop:one-step-bridge	Full	DQ.ClaimClosure.one_step_bridge
prop:oracle-lattice-strict	Full	ExplicitHardnessAssumptions, L.eval
prop:oracle-lattice-transfer	Full	DQ.ClaimClosure.oracle_lattice_transfer_as_regime_simulation, bounded_actions_detectable
prop:outside-excuses-explicit-assumptions	Full	DQ.ClaimClosure.explicit_assumptions_required_of_not_excused_core
prop:physical-claim-transport	Full	CT.physical_claim_lifts_from_core, CT.physical_claim_lifts_from_core_conditional
prop:physics-no-universal-exact	Full	DQ.ClaimClosure.declared_physics_no_universal_exact_certifier_core
prop:query-finite-state-generalization	Full	StandardComplexityAssumptions, adq_ordering
prop:query-randomized-robustness	Full	OneStepSequentialObjective, TwoStepObjective, adq
prop:query-randomized-weighted	Full	bridgeTransferLicensed, bridge_boundary_represented_family
prop:query-regime-obstruction	Full	DQ.ClaimClosure.query_obstruction_finite_state_core, StandardComplexityAssumptions, adq_ordering
prop:query-state-batch-lb	Full	StochasticCriterionObjective, agentBridgeClass
prop:query-subproblem-transfer	Full	DQ.ClaimClosure.regime_simulation_transfers_hardness, DQ.ClaimClosure.stochastic_objective_bridge_can_fail_on_sufficiency, DQ.ClaimClosure.subproblem_hardness_lifts_to_full
prop:query-tightness-full-scan	Full	TransitionCoupledObjective
prop:query-value-entry-lb	Full	StructureDetectable, boundaryCharacterized_iff_exists_sufficient_subset
prop:query-weighted-transfer	Full	bounded_actions_reusable_heuristic, bridgeFailureWitness
prop:raw-certified-bit-split	Full	DQ.ClaimClosure.certified_total_bits_split_core, DQ.IntegrityCompetence.admissible_irrational_strictly_more_than_rational, DQ.IntegrityCompetence.admissible_matrix_counts, DQ.IntegrityCompetence.certaintyInflation_iff_not_admissible, DQ.IntegrityCompetence.certificationOverheadBits, DQ.IntegrityCompetence.certificationOverheadBits_of_evidence, DQ.IntegrityCompetence.certificationOverheadBits_of_no_evidence
prop:rlff-maximizer-admissible	Full	DQ.IntegrityCompetence.exact_raw_only_of_no_exact_admissible, DQ.IntegrityCompetence.integrity_forces_abstention
prop:run-time-accounting	Full	DQ.Physics.DecisionTime.decisionTrace_length_eq_ticks, DQ.Physics.DecisionTime.decision_count_equals_elapsed_time, DQ.Physics.DecisionTime.run_elapsed_time_eq_ticks, DQ.Physics.DecisionTime.run_time_exact
prop:selector-separation	Full	DQ.ClaimClosure.selectorSufficient_not_implies_setSufficient
prop:self-confidence-not-certification	Full	DQ.IntegrityCompetence.signal_exact_no_competence_forces_zero_certified
prop:set-to-selector	Full	DQ.ClaimClosure.DecisionProblem.sufficient_iff_zeroEpsilonSufficient
prop:snapshot-process-typing	Full	DQ.ClaimClosure.agent_transfer_licensed_iff_snapshot, DQ.ClaimClosure.process_bridge_failure_witness, DQ.ClaimClosure.snapshot_vs_process_typed_boundary
prop:steps-run-scalar	Full	DQ.IntegrityCompetence.rlff_maximizer_is_admissible, DQ.IntegrityCompetence.self_reflected_confidence_not_certification

prop:substrate-unit-time	Full	DQ.Physics.DecisionTime.substrate_step_is_time_unit, DQ.Physics.DecisionTime.substrate_step_realizes_decision_event, DQ.Physics.DecisionTime.time_unit_law_substrate_invariant
prop:sufficiency-char	Full	DQ.ClaimClosure.sufficiency_conp_reduction_core, DQ.ClaimClosure.sufficiency_iff_dq_ratio
prop:thermo-conservation-additive	Full	DQ.ClaimClosure.sufficiency_iff_projectedOptCover_eq_opt
prop:thermo-hardness-bundle	Full	DQ.ClaimClosure.thermo_eventual_lift_core
prop:thermo-lift	Full	DQ.ClaimClosure.thermo_conservation_additive_core, DQ.ClaimClosure.thermo_energy_carbon_lift_core
prop:thermo-mandatory-cost	Full	DQ.ClaimClosure.thermo_hardness_bundle_core
prop:time-discrete	Full	DQ.Physics.DecisionTime.time_coordinate_falsifiable, DQ.Physics.DecisionTime.time_is_discrete
prop:typed-claim-admissibility	Full	DQ.ClaimClosure.tree_structure_detectable
prop:universal-solver-framing	Full	DQ.ClaimClosure.typed_static_class_completeness
prop:zero-epsilon-competence	Full	DQ.IntegrityCompetence.certifiedTotalBits_gt_raw_of_evidence, DQ.IntegrityCompetence.integrity_not_competent_of_nonempty_scope
prop:zero-epsilon-reduction	Full	DQ.ClaimClosure.DecisionProblem.epsOpt_zero_eq_opt, DQ.DecisionProblem.minimalSufficient_iff_relevant
thm:amortization	Full	DQ.HardnessDistribution.centralized_higher_leverage
thm:bridge-boundary-represented	Full	DQ.ClaimClosure.bridge_boundary_represented_family, DQ.ClaimClosure.bridge_failure_witness_non_one_step, DQ.ClaimClosure.bridge_transfer_iff_one_step_class
thm:centralization-dominance	Full	DQ.HardnessDistribution.centralization_dominance_bundle, DQ.HardnessDistribution.centralization_step_saves_n_minus_one
thm:claim-integrity-meta	Full	DQ.ClaimClosure.explicit_assumptions_required_of_not_excused_core
thm:config-reduction	Full	DQ.ConfigReduction.config_sufficiency_iff_behavior_preserving
thm:cost-asymmetry-eth	Full	DQ.ClaimClosure.cost_asymmetry_eth_conditional, DQ.HardnessDistribution.isWrongHardness
thm:dichotomy	Full	DQ.ClaimClosure.dichotomy_conditional, DQ.ClaimClosure.explicit_state_upper_core, DQ.ClaimClosure.hard_family_all_coords_core
thm:exact-certified-gap-iff-admissible	Full	DQ.ClaimClosure.epsilon_admissible_iff_raw_lt_certified_total_core, DQ.ClaimClosure.exact_admissible_iff_raw_lt_certified_total_core, DQ.ClaimClosure.exact_raw_eq_certified_iff_certainty_inflation_core
thm:generalized-dominance	Full	DQ.HardnessDistribution.generalized_dominance_can_fail_without_wrong_growth
thm:generalized-saturation-possible	Full	DQ.HardnessDistribution.gap_conservation_card, DQ.HardnessDistribution.right_dominates_wrong
thm:linear-saturation-iff-zero	Full	DQ.HardnessDistribution.simplicityTax_grows
thm:overmodel-diagnostic	Full	DQ.ClaimClosure.boundaryCharacterized_iff_exists_sufficient_subset, DQ.ClaimClosure.no_exact_identifier_implies_not_boundary_characterized
thm:physical-bridge-bundle	Full	CT.physical_bridge_bundle
thm:regime-coverage	Full	DQ.ClaimClosure.declaredRegimeFamily_complete, DQ.ClaimClosure.regime_core_claim_proved
thm:tax-conservation	Full	DQ.HardnessDistribution.complete_model_dominates_after_threshold
thm:tax-grows	Full	DQ.HardnessDistribution.totalDOF_ge_intrinsic
thm:tractable	Full	DQ.ClaimClosure.thermo_mandatory_cost_core, DQ.ClaimClosure.tractable_bounded_core, DQ.ClaimClosure.tractable_separable_core, DQ.ClaimClosure.tractable_subcases_conditional
thm:typed-completeness-static	Full	DQ.ClaimClosure.typed_claim_admissibility_core

Auto summary: mapped 113/113 (full=113, derived=0, unmapped=0).

A.8 Proof Hardness Index (Auto)

Paper handle	Hardness profile	Regime tags	Lean support
thm:amortization	cost-growth	LC	HD3
thm: centralization-dominance	cost-growth	AR,LC	HD1, HD2
thm:tax-conservation	cost-growth	AR,LC	HD4
thm:tax-grows	cost-growth	LC	HD21
thm:cost-asymmetry-eth	exp-lb-conditional	S+ETH	CC12, HD13
prop:typed-claim-admissibility	exp-lb-conditional, query-lb	AR,DC,Qf,S+ETH	CC75
thm:overmodel-diagnostic	succinct-hard	S	CC6, CC40
thm:regime-coverage	succinct-hard	CR,S	CC14, CC51
thm:typed-completeness-static	succinct-hard	AR,CR,S	CC76
prop: integrity-resource-bound	succinct-hard,exp-lb-conditional	AR,DC,S,S+ETH	CC30, IC24, IC26
cor: physical-counterexample-core-failure	succinct-hard,exp-lb-conditional, query-lb,tractable-structured	AR,E,Qb,Qf,S,S+ETH	CT3, CT7, CT8
prop:decision-unit-time	succinct-hard,exp-lb-conditional, query-lb,tractable-structured	AR,E,Qb,Qf,S,S+ETH	DQ535, DQ537, DQ545, DQ546
prop: oracle-lattice-strict	succinct-hard,exp-lb-conditional, query-lb,tractable-structured	AR,E,Qb,Qf,S,S+ETH	L3, L4
prop: oracle-lattice-transfer	succinct-hard,exp-lb-conditional, query-lb,tractable-structured	AR,E,Qb,Qf,S,S+ETH	CC43, L20
prop: physical-claim-transport	succinct-hard,exp-lb-conditional, query-lb,tractable-structured	AR,E,Qb,Qf,S,S+ETH	CT5, CT6
prop: query-regime-obstruction	succinct-hard,exp-lb-conditional, query-lb,tractable-structured	AR,E,Q_fin,Qb,Qf,S,S+ETH	CC50, L7, L13
prop: query-state-batch-lb	succinct-hard,exp-lb-conditional, query-lb,tractable-structured	AR,E,Qb,Qf,S,S+ETH	L8, L14
prop:query-subproblem-transfer	succinct-hard,exp-lb-conditional, query-lb,tractable-structured	AR,E,Qb,Qf,S,S+ETH	CC52, CC58, CC59
prop: query-value-entry-lb	succinct-hard,exp-lb-conditional, query-lb,tractable-structured	AR,E,Qb,Qf,S,S+ETH	L9, L19
prop:run-time-accounting	succinct-hard,exp-lb-conditional, query-lb,tractable-structured	AR,E,Qb,Qf,S,S+ETH	DQ533, DQ534, DQ539, DQ540
prop:substrate-unit-time	succinct-hard,exp-lb-conditional, query-lb,tractable-structured	AR,E,Qb,Qf,S,S+ETH	DQ541, DQ542, DQ549
prop:time-discrete	succinct-hard,exp-lb-conditional, query-lb,tractable-structured	AR,E,Qb,Qf,S,S+ETH	DQ547, DQ548

thm:dichotomy	succinct-hard,exp-lb-conditional, query-lb,tractable-structured	AR,E,Q_fin,Qb,Qf, RG,S,S+ETH	CC16, CC23, CC24
thm:tractable	succinct-hard,exp-lb-conditional, query-lb,tractable-structured	AR,E,Qb,Qf,RG,S, S+ETH,STR,TA	CC69, CC70, CC71, CC72
thm:config-reduction	succinct-hard, tractable-structured	E,S,TA	CR1
cor: exact-identifiability	unspecified	-	S2P1
cor: exact-no-competence-zero-certified	unspecified	AR,DC	IC41
cor:gap-externalization	unspecified	-	HD18, HD21
cor: gap-minimization-hard	unspecified	-	S2P2
cor:generalized-eventual-dominance	unspecified	-	HD9
cor:hardness-exact-certainty-inflation	unspecified	CR	CC19
cor: hardness-raw-only-exact	unspecified	AR	CC21, IC23
cor:information-barrier-query	unspecified	-	CC27, CC28, CC29
cor:integrity-universal	unspecified	TR	CC31
cor:interior-singleton-certificate	unspecified	AR	IV9
cor:linear-positive-no-saturation	unspecified	-	HD15
cor:no-auto-minimize	unspecified	-	CC37
cor:no-uncertified-exact-claim	unspecified	AR	CC41
cor:outside-excuses-no-exact-report	unspecified	CR,DC	CC39
cor:overmodel-diagnostic-implication	unspecified	-	S2P6
cor:physics-no-universal-exact-claim	unspecified	AR,CR,DC	CC38
cor:practice-bounded	unspecified	-	L15
cor:practice-diagnostic	unspecified	-	S2P2
cor:practice-structured	unspecified	-	L16
cor:practice-tree	unspecified	-	L17
cor: practice-unstructured	unspecified	-	L1
cor: query-obstruction-bool	unspecified	-	CC49, L6
cor:right-wrong-hardness	unspecified	-	HD16
cor: rlff-abstain-no-certs	unspecified	AR	IC30
cor: type-system-threshold	unspecified	-	HD14
prop:abstain-guess-self-signal	unspecified	AR,DC	IC46
prop:adq-ordering	unspecified	-	CC2
prop:attempted-competence-matrix	unspecified	AR	IC6, IC7, IC27
prop:bounded-slice-meta-irrelevance	unspecified	AR	CC32, CC33
prop: bridge-failure-horizon	unspecified	-	CC25, CC26
prop:bridge-failure-stochastic	unspecified	-	CC57
prop:bridge-failure-transition	unspecified	-	CC73

prop: bridge-transfer-scope	unspecified	-	CC42
prop:budgeted-crossover	unspecified	-	CC45, PBC1
prop: certainty-inflation-iff-inadmissible	unspecified	CR	IC8, IC22
prop:certified-confidence-gate	unspecified	AR,DC	IC38, IC39
prop:crossover-above-cap	unspecified	-	CC44, PBC2
prop:crossover-not-certification	unspecified	-	CC46
prop:crossover-policy	unspecified	-	CC47
prop:declared-contract-selection-validity	unspecified	-	CC22, CC39, CC75
prop:dominance-modes	unspecified	-	HD3
prop:empty-sufficient-constant	unspecified	AR,DC,DM	DP6
prop: evidence-admissibility-equivalence	unspecified	AR,DC	IC17, IC20, IC21
prop: exact-requires-evidence	unspecified	AR,DC	IC35, IC36
prop:fraction-defined-under-bound	unspecified	AR,DC	IC45
prop:generalized-assumption-boundary	unspecified	-	HD6, HD7
prop: hardness-conservation	unspecified	-	HD20
prop: hardness-efficiency-interpretation	unspecified	-	HD10
prop: heuristic-reusability	unspecified	-	CC7, CC53, CC55, CC74
prop:insufficiency-counterexample	unspecified	AR,DC,DM	DP7, DP8
prop:integrity-competence-separation	unspecified	AR	IC18, IC25
prop: interior-one-sidedness	unspecified	AR	IV6, IV8
prop:interior-universal-non-rejection	unspecified	AR	IV5
prop:interior-verification-tractable	unspecified	AR	IV4, IV7
prop:law-instance-objective-bridge	unspecified	AR	CT1, CT2
prop: minimal-relevant-equiv	unspecified	-	DP2, DP3
prop:no-evidence-zero-certified	unspecified	AR,DC	IC40
prop:no-fraction-without-bound	unspecified	AR,DC	IC44
prop:one-step-bridge	unspecified	-	CC42
prop:outside-excuses-explicit-assumptions	unspecified	CR,DC	CC22
prop:physics-no-universal-exact	unspecified	AR,CR,DC	CC15
prop:query-finite-state-generalization	unspecified	-	L7, L13
prop:query-randomized-robustness	unspecified	-	L5, L11, L12
prop:query-randomized-weighted	unspecified	-	L23, L24
prop:query-tightness-full-scan	unspecified	-	L10
prop: query-weighted-transfer	unspecified	-	L21, L22
prop: raw-certified-bit-split	unspecified	AR	CC11, IC10, IC11, IC13, IC14, IC15, IC16

prop:rlff-maximizer-admissible	unspecified	AR	IC31, IC32
prop:selector-separation	unspecified	-	CC54
prop:self-confidence-not-certification	unspecified	AR,DC	IC47
prop:set-to-selector	unspecified	-	DP5
prop:snapshot-process-typing	unspecified	RA	CC3, CC48, CC56
prop:steps-run-scalar	unspecified	AR,DC	IC42, IC43
prop:sufficiency-char	unspecified	AR	CC62, CC63
prop:thermo-conservation-additive	unspecified	AR	CC64
prop:thermo-hardness-bundle	unspecified	AR	CC67
prop:thermo-lift	unspecified	AR	CC65, CC66
prop:thermo-mandatory-cost	unspecified	AR	CC68
prop:universal-solver-framing	unspecified	TR	CC77
prop:zero-epsilon-competence	unspecified	-	IC19, IC33
prop:zero-epsilon-reduction	unspecified	-	DP4, DP1
thm:bridge-boundary-represented	unspecified	-	CC8, CC9, CC10
thm:claim-integrity-meta	unspecified	AR,CR,DC	CC22
thm:exact-certified-gap-iff-admissible	unspecified	AR	CC17, CC18, CC20
thm:generalized-dominance	unspecified	-	HD8
thm:generalized-saturation-possible	unspecified	-	HD5, HD17
thm:linear-saturation-iff-zero	unspecified	-	HD19
thm:physical-bridge-bundle	unspecified	AR	CT4

Auto summary: indexed 113 claims by hardness profile (cost-growth=4; exp-lb-conditional=1; exp-lb-conditional,query-lb=1; succinct-hard=3; succinct-hard,exp-lb-conditional=1; succinct-hard,exp-lb-conditional,query-lb,tractable-structured=14; succinct-hard,tractable-structured=1; unspecified=88).

A.9 Claims Not Fully Mechanized

Status: all theorem/proposition/corollary handles in this paper now have Lean backing. Entries marked **Full (conditional)** are explicitly mechanized transfer theorems that depend on standard external complexity facts (e.g., source-class completeness or ETH assumptions), with those dependencies represented as hypotheses in Lean.

A.10 Module Structure

- `Basic.lean` – Core definitions (DecisionProblem, sufficiency, optimality)
- `Sufficiency.lean` – Sufficiency checking algorithms and properties
- `Reduction.lean` – TAUTOLOGY reduction construction and correctness
- `Hardness/ConfigReduction.lean` – Sentinel-action configuration reduction theorem
- `Complexity.lean` – Polynomial-time reduction definitions using mathlib

- `HardnessDistribution.lean` – Simplicity Tax redistribution and amortization theorems
- `IntegrityCompetence.lean` – Solver integrity vs regime competence separation
- `ClaimClosure.lean` – Mechanized closure of paper-level bridge and diagnostic claims
- `Tractability/` – Bounded actions, separable utilities, tree structure

A.11 Verification

The proofs compile with Lean 4 and contain no `sorry` placeholders. Run `lake build` in the proof directory to verify.

References

[1] Hirotugu Akaike. A new look at the statistical model identification. *IEEE Transactions on Automatic Control*, 19(6):716–723, 1974.

[2] Robert M. Alaniz, Josh Brunner, Michael Coulombe, Erik D. Demaine, Jenny Diomidova, Timothy Gomez, Elise Grizzell, Ryan Knobel, Jayson Lynch, Andrew Rodriguez, Robert Schweller, and Tim Wylie. Complexity of reconfiguration in surface chemical reaction networks. *ArXiv preprint*, 2023.

[3] Edoardo Amaldi and Viggo Kann. On the approximability of minimizing nonzero variables or unsatisfied relations in linear systems. *Theoretical Computer Science*, 209(1-2):237–260, 1998.

[4] Jakob L. Andersen, Christoph Flamm, Daniel Merkle, and Peter F. Stadler. Maximizing output and recognizing autocatalysis in chemical reaction networks is np-complete. *Journal of Systems Chemistry*, 3(1):1–15, 2012.

[5] Sanjeev Arora and Boaz Barak. *Computational Complexity: A Modern Approach*. Cambridge University Press, 2009.

[6] Len Bass, Paul Clements, and Rick Kazman. *Software Architecture in Practice*. Addison-Wesley Professional, 3rd edition, 2012.

[7] Charles H. Bennett. Notes on landauer's principle, reversible computation, and maxwell's demon. *Studies in History and Philosophy of Science Part B: Studies in History and Philosophy of Modern Physics*, 34(3):501–510, 2003.

[8] Richard E. Blahut. Computation of channel capacity and rate-distortion functions. *IEEE Transactions on Information Theory*, 18(4):460–473, 1972.

[9] Avrim L. Blum and Pat Langley. Selection of relevant features and examples in machine learning. *Artificial Intelligence*, 97(1-2):245–271, 1997.

[10] Frederick P. Brooks. No silver bullet: Essence and accidents of software engineering. *Computer*, 20(4):10–19, 1987.

[11] Harry Buhrman and Ronald de Wolf. Complexity measures and decision tree complexity: A survey. *Theoretical Computer Science*, 288(1):21–43, 2002.

[12] Luca Cardelli and Peter Wegner. On understanding types, data abstraction, and polymorphism. *ACM Computing Surveys*, 17(4):471–523, 1985.

[13] David Maxwell Chickering, David Heckerman, and Christopher Meek. Large-sample learning of Bayesian networks is NP-hard. *Journal of Machine Learning Research*, 5:1287–1330, 2004.

[14] Thomas M. Cover and Joy A. Thomas. *Elements of Information Theory*. Wiley-Interscience, 2nd edition, 2006.

[15] Imre Csiszár and János Körner. *Information Theory: Coding Theorems for Discrete Memoryless Systems*. Cambridge University Press, 2nd edition, 2011.

[16] Leonardo de Moura, Soonho Kong, Jeremy Avigad, Floris van Doorn, and Jakob von Raumer. The Lean theorem prover (system description). In *Automated Deduction – CADE-25*, volume 9195 of *Lecture Notes in Computer Science*, pages 378–388. Springer, 2015.

[17] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, pages 625–635. Springer, 2021.

[18] Shahar Dobzinski and Jan Vondrák. From query complexity to computational complexity. In *Proceedings of the 44th ACM Symposium on Theory of Computing*, pages 1107–1116, 2012.

[19] Rolf Fagerberg, Christoph Flamm, Daniel Merkle, Philipp Peters, and Peter F. Stadler. On the complexity of reconstructing chemical reaction networks. *Mathematical Biosciences*, 242(2):86–96, 2013.

[20] Ronald A. Fisher. On the mathematical foundations of theoretical statistics. *Philosophical Transactions of the Royal Society of London. Series A*, 222:309–368, 1922.

[21] Yannick Forster, Fabian Kunze, and Maxi Wuttke. Verified programming of Turing machines in Coq. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, pages 114–128. ACM, 2020.

[22] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design patterns: elements of reusable object-oriented software*. Addison-Wesley, 1994.

[23] Shafi Goldwasser, Silvio Micali, and Charles Rackoff. The knowledge complexity of interactive proof systems. *SIAM Journal on Computing*, 18(1):186–208, 1989.

[24] Peter D. Grünwald. *The Minimum Description Length Principle*. MIT Press, 2007.

[25] Isabelle Guyon and André Elisseeff. An introduction to variable and feature selection. *Journal of Machine Learning Research*, 3:1157–1182, 2003.

[26] Maximilian P. L. Haslbeck and Tobias Nipkow. Verified analysis of list update algorithms. In *36th IARCS Annual Conference on Foundations of Software Technology and Theoretical Computer Science (FSTTCS 2016)*, volume 65 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 49:1–49:15. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2016.

[27] Ronald A. Howard. Information value theory. *IEEE Transactions on Systems Science and Cybernetics*, 2(1):22–26, 1966.

[28] Russell Impagliazzo and Ramamohan Paturi. On the complexity of k-sat. *Journal of Computer and System Sciences*, 62(2):367–375, 2001.

[29] Emir Kamenica and Matthew Gentzkow. Bayesian persuasion. *American Economic Review*, 101(6):2590–2615, 2011.

[30] Daphne Koller and Nir Friedman. *Probabilistic Graphical Models: Principles and Techniques*. MIT Press, 2009.

[31] János Körner. Coding of an information source having ambiguous alphabet and the entropy of graphs. In *Transactions of the 6th Prague Conference on Information Theory, Statistical Decision Functions, Random Processes*, pages 411–425, 1973.

[32] Fabian Kunze, Gert Smolka, and Yannick Forster. Formal small-step verification of a call-by-value lambda calculus machine. In *Asian Symposium on Programming Languages and Systems*, pages 264–283. Springer, 2018.

[33] Rolf Landauer. Irreversibility and heat generation in the computing process. *IBM Journal of Research and Development*, 5(3):183–191, 1961.

[34] Xavier Leroy. Formal verification of a realistic compiler. *Communications of the ACM*, 52(7):107–115, 2009.

[35] Ming Li and Paul Vitányi. *An Introduction to Kolmogorov Complexity and Its Applications*. Springer, 3rd edition, 2008.

[36] Barbara H Liskov and Jeannette M Wing. A behavioral notion of subtyping. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 16(6):1811–1841, 1994.

[37] Michael L. Littman, Judy Goldsmith, and Martin Mundhenk. The computational complexity of probabilistic planning. *Journal of Artificial Intelligence Research*, 9:1–36, 1998.

[38] László Lovász. On the shannon capacity of a graph. *IEEE Transactions on Information Theory*, 25(1):1–7, 1979.

[39] Ross M. McConnell, Kurt Mehlhorn, Stefan Näher, and Pascal Schweitzer. Certifying algorithms. *Computer Science Review*, 5(2):119–161, 2011.

[40] Paul Milgrom and John Roberts. Price and advertising signals of product quality. *Journal of Political Economy*, 94(4):796–821, 1986.

[41] Martin Mundhenk, Judy Goldsmith, Christopher Lusena, and Eric Allender. The complexity of finite-horizon markov decision process problems. *Journal of the ACM*, 47(4):681–720, 2000.

[42] Roger B. Myerson. Incentive compatibility and the bargaining problem. *Econometrica*, 47(1):61–73, 1979.

[43] George C. Necula. Proof-carrying code. In *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, pages 106–119. ACM, 1997.

[44] Tobias Nipkow and Gerwin Klein. *Concrete Semantics with Isabelle/HOL*. Springer, 2014.

[45] Tobias Nipkow, Lawrence C. Paulson, and Markus Wenzel. *Isabelle/HOL: A Proof Assistant for Higher-Order Logic*. Springer, 2002.

[46] Jose Nelson Onuchic, Zaida Luthey-Schulten, and Peter G. Wolynes. Theory of protein folding: The energy landscape perspective. *Annual Review of Physical Chemistry*, 48:545–600, 1997.

[47] Christos H. Papadimitriou. *Computational Complexity*. Addison-Wesley, 1994. Classic textbook; publisher DOI not in common use for this edition.

[48] Christos H. Papadimitriou and John N. Tsitsiklis. The complexity of markov decision processes. *Mathematics of Operations Research*, 12(3):441–450, 1987.

[49] David L Parnas. On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, 15(12):1053–1058, 1972.

[50] Judea Pearl. *Causality: Models, Reasoning, and Inference*. Cambridge University Press, 2nd edition, 2009.

[51] Benjamin C Pierce. *Types and programming languages*. MIT press, 2002.

[52] Howard Raiffa and Robert Schlaifer. *Applied Statistical Decision Theory*. Division of Research, Graduate School of Business Administration, Harvard University, Boston, MA, 1961. Library records list this imprint; avoid using JSTOR DOI 10.2307/2282452 which refers to a journal item, not the book.

[53] John C Reynolds. Types, abstraction and parametric polymorphism. In *Information processing*, volume 83, pages 513–523, 1983.

[54] Jorma Rissanen. Modeling by shortest data description. *Automatica*, 14(5):465–471, 1978.

[55] Leonard J. Savage. *The Foundations of Statistics*. John Wiley & Sons, 1954.

[56] Gideon Schwarz. Estimating the dimension of a model. *The Annals of Statistics*, 6(2):461–464, 1978.

[57] Claude E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(3):379–423, 1948.

[58] Claude E. Shannon. Zero-error capacity of a noisy channel. *IRE Transactions on Information Theory*, 2(3):8–19, 1956.

[59] Claude E. Shannon. Coding theorems for a discrete source with a fidelity criterion. *IRE National Convention Record*, 7:142–163, 1959. Part 4.

[60] Mary Shaw and David Garlan. *Software Architecture: Perspectives on an Emerging Discipline*. Prentice Hall, 1996.

[61] Ilya Shpitser and Judea Pearl. Identification of joint interventional distributions in recursive semi-Markovian causal models. In *Proceedings of the Twenty-First National Conference on Artificial Intelligence (AAAI-06)*, pages 1219–1226. AAAI Press, 2006.

[62] Jeremy G Siek and Walid Taha. Gradual typing for functional languages. *Scheme and Functional Programming Workshop*, 6:81–92, 2006.

[63] Tristan Simas. Identification capacity and rate-query tradeoffs in classification systems, 2026.

[64] David Slepian and Jack K. Wolf. Noiseless coding of correlated information sources. *IEEE Transactions on Information Theory*, 19(4):471–480, 1973.

[65] Michael Spence. Job market signaling. *The Quarterly Journal of Economics*, 87(3):355–374, 1973.

[66] Mervyn Stone. Cross-validatory choice and assessment of statistical predictions. *Journal of the Royal Statistical Society: Series B (Methodological)*, 36(2):111–133, 1974.

[67] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, pages 367–381. Association for Computing Machinery, 2020.

[68] David J. Wales. *Energy Landscapes: Applications to Clusters, Biomolecules and Glasses*. Cambridge University Press, Cambridge, 2003.

[69] David J. Wales. The energy landscape as a unifying theme in molecular science. *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 363(1827):1711–1733, 2005.

[70] David J. Wales. Exploring energy landscapes. *Annual Review of Physical Chemistry*, 69:401–425, 2018.

[71] Gregory Wheeler. Bounded rationality. The Stanford Encyclopedia of Philosophy, 2024. First published 2018; substantive revision 13 Dec 2024.

[72] Jiayi Yuan, Hao Li, Xinheng Ding, Wenya Xie, Yu-Jhe Li, Wentian Zhao, Kun Wan, Jing Shi, Xia Hu, and Zirui Liu. Understanding and mitigating numerical sources of nondeterminism in LLM inference, 2025.